HARDWARE AND SYSTEMS

Hardware and low-level systems from first principles

UNIT 4

Systems Software and the Product

Kernels, Linux, drivers, virtualisation, data paths and the product

4.1 Write a kernel

4.2 Linux kernel internals

4.3 Linux device drivers

4.4 Virtualisation and hypervisors

4.5 Storage and network data paths

4.6 Product engineering

4.7 Capstone: Module Zero

Second edition, October 2026

Contents

About this unit 2
- Before you start 2
- Kit for this unit 2

4.1 Write a kernel 3
- Core ideas 3
- Reading 6
- Labs 7
- Problem set 9
- Deliverables and stretch 10

4.2 Linux kernel internals 11
- Core ideas 11
- Reading 14
- Labs 14
- Problem set 17
- Deliverables and stretch 18

4.3 Linux device drivers 19
- Core ideas 19
- Reading 22
- Labs 22
- Problem set 27
- Deliverables and stretch 28

4.4 Virtualisation and hypervisors 29
- Core ideas 29
- Reading 32
- Labs 33
- Problem set 36
- Deliverables and stretch 37

4.5 Storage and network data paths 38
- Core ideas 38
- Reading 41
- Labs 41
- Problem set 44
- Deliverables and stretch 45

4.6 Product engineering 46
- Core ideas 46
- Reading 50
- Labs 50
- Problem set 53
- Deliverables and stretch 53

4.7 Capstone: Module Zero 54
- The demonstration 54
- Architecture 55
- Weekly plan 57
- Acceptance criteria 59
- Measurements to record 59
- Retrospective 60
- Deliverables and stretch 60
- After the course 60

Sign-off 61

UNIT 4

About this unit

Unit 4 covers the software a server company runs on its hardware, and the product built from both. You write a 64-bit SMP kernel with an NVMe driver, then learn Linux from the inside: building, tracing, bisecting and hardening it. You write Linux drivers for TI parts that mainline lacks, build a KVM virtual machine monitor with your own virtio devices, and measure storage and network data paths from `io_uring` and the kernel stack to SPDK and DPDK. Then you write the documents that turn a prototype into a product, and assemble Module Zero: two nodes under your BMC that attest, form a quorum, run tenants and survive a pulled cable.

Chapter	You build	Weeks
4.1	A 64-bit SMP kernel with paging, user mode, an NVMe driver and a filesystem, on QEMU and the server	5
4.2	Linux built, traced, bisected and hardened on the server and the Pi; the node kernels	3.5
4.3	Mainline TI drivers by overlay, your own OPT4048 and ADS1220 drivers, fan control, PCIe and VFIO	5
4.4	A KVM VMM with virtio devices, VFIO pass-through and live migration; a bare-metal hypervisor	5
4.5	Storage and network paths measured; SPDK; Reed-Solomon; NVMe over Fabrics; XDP and DPDK; a Blocks prototype	2.5
4.6	Module Zero's specification, threat model, update architecture, test plan, compliance plan and cost model	2
4.7	Module Zero, demonstrated from a cold start and recorded	6 to 8

Before you start

The server, the Pi, Board A and Board B from Chapter 3.4, and the lab network from the Handbook must all be working. Chapter 4.5 boots guests in your Chapter 4.4 VMM, so it follows 4.4, and 4.6 can be read alongside the capstone's first two weeks.

Kit for this unit

The Handbook's kit part lists every item for this unit with its part number, seller and price. The totals, for items this unit needs first:

	Required	Optional
UK sellers, VAT included (ex-VAT prices × 1.2)	£173	£0
Overseas sellers, before import VAT and delivery	none	none
Items with no checked price	1	none

Not priced: Intel SSD DC P4510 1 TB U.2, used.

CHAPTER 4.1

Write a kernel

TIME	5 weeks
YOU BUILD	A 64-bit SMP kernel for x86-64, started by both of your Chapter 2.7 loaders on QEMU and the lab server: its own consoles, GDT, IDT and page tables, a buddy allocator and slab heap, a per-CPU scheduler on every core, user processes with copy-on-write `fork` and validated system calls, a libc and shell, PCI enumeration, an NVMe driver with polled and MSI-X completion, and a read-only FAT32 filesystem on a real drive.
NEEDS	QEMU with SeaBIOS and OVMF, your Lab 2.7.1 and 2.7.2 loaders, an `x86_64-elf` cross compiler, the lab server with serial-over-LAN, and the Kingston NV3 as the sacrificial drive.

WHAT THIS BUYS OMH

OMH will ship Linux, configured and patched by you and occasionally bisected while a customer's Storage module is down. Writing your own kernel first makes the mechanisms familiar. You know what a page-fault handler must do because yours does it, why a scheduler keeps per-CPU run queues because yours deadlocked without them, and what an NVMe completion looks like because you polled for one. Storage and Blocks are NVMe products, so the driver lab is the one you may not skip. The target is x86-64, which OMH modules run. Chapter 4.4 runs this kernel as a guest of your VMM and then turns it into a hypervisor.

SKIP TEST

1. A user process touches an unmapped stack page. Trace the path from the CPU raising the exception to the process resuming, naming every data structure the kernel consults.
2. Two cores each hold a spinlock and want the other's. How did they get there, and what discipline prevents it? Why must a spinlock that an interrupt handler also takes disable interrupts?
3. Describe the state saved on a context switch between two kernel threads, and why it is less than the state saved on a trap from user mode.
4. Give the steps that bring up a second x86 core: what the bootstrap processor sends, what state the application processor starts in, and what must exist before it can run C.
5. Walk an NVMe read: what the host writes where, how the device finds the data buffer, and how the host learns that the command completed.

If all five are easy, do Labs 4.1.3 and 4.1.5 only.

Core ideas

What a kernel is. A kernel is the program that owns the privileged state of Chapter 2.5 and multiplexes the machine among programs that do not. It does this through memory protection by paging, time sharing by timer interrupts and context switches, controlled entry by system calls, and device access by drivers. Policies such as scheduling and allocation sit on top and can change independently.

The handoff. Your Lab 2.7.2 loader leaves the CPU in long mode with boot services exited and passes a boot-information structure: memory map, framebuffer, RSDP and command line. Your Lab 2.7.1 loader can pass the same structure once its stage two collects the E820 map. A kernel that boots unchanged from both loaders depends on that structure alone. The kernel's first jobs are a serial console for `printk`, a `panic` that dumps registers and halts, and its own GDT, IDT and page tables.

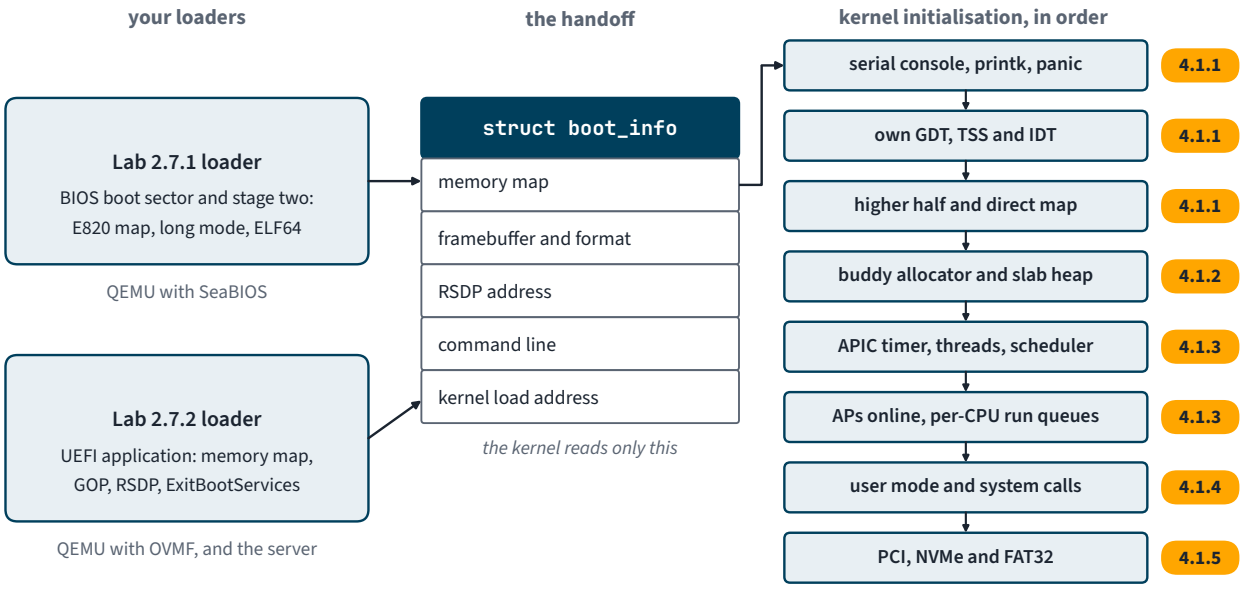


Figure 4.1.1. The kernel's boot flow from either loader. Both loaders fill the same boot-information structure, and the initialisation order on the right is the order of Labs 4.1.1 to 4.1.5.

Physical memory. Parse the map into usable ranges, minus the kernel image, the loader's structures, ACPI tables and firmware runtime regions. A buddy allocator is fast and returns the contiguous power-of-two blocks that DMA needs; build it in preference to a bitmap.

Virtual memory. The kernel lives in the higher half and user programs in the lower. A direct map of all physical memory at a fixed offset turns physical-to-virtual translation into an addition. Four-level tables need map, unmap, protect and walk, each change followed by `invlpg`. Every process's PML4 shares its upper 256 entries, so the kernel half is identical everywhere. A slab heap sits on the page allocator, and the image links in the top 2 GiB so that `-mcmodel=kernel` code reaches it with 32-bit displacements.

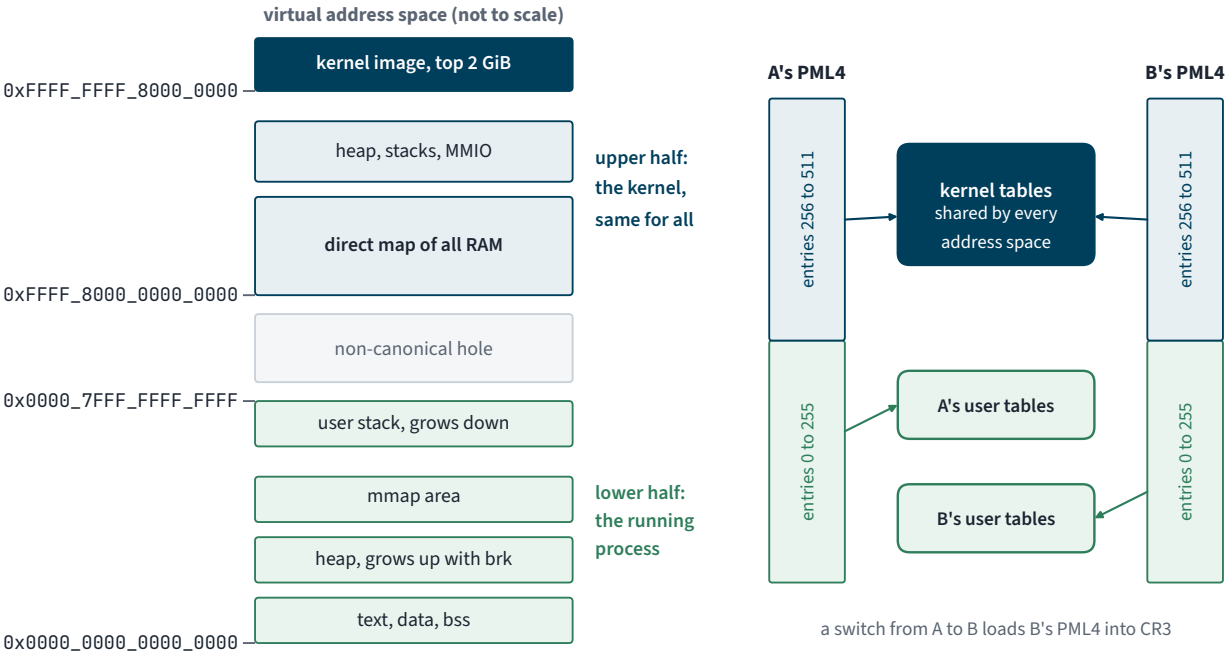


Figure 4.1.2. One possible layout for your kernel. Every PML4 points its upper 256 entries at the same kernel tables, so a context switch changes CR3 without changing the kernel's view of memory.

Interrupts and time. The GDT, TSS, IDT and APIC code of Lab 2.5.3 carries over. The local APIC timer is the scheduler tick; calibrate it and the TSC against the PIT or the HPET, and build a monotonic `ktime_ns()` on the TSC. Handlers stay short and defer work to a queue or a thread.

Threads and the context switch. A thread is a kernel stack plus saved registers. To switch, push the callee-saved registers, save the stack pointer, load the next thread's, pop and return. That is about a dozen instructions, plus a CR3 load between processes. A new thread's first run is faked with a stack that looks as if it had been switched away from.

Scheduling. Build round-robin first, then priorities, then per-CPU run queues with idle balancing. Wait queues block threads on time or I/O, and the idle thread runs `hlt`. Preemption happens on the timer interrupt's return path, and a per-CPU `preempt_disable` count holds it off in critical regions.

SMP. The MADT lists the application processors (APs) as local APIC or x2APIC entries. The bootstrap processor sends each an INIT inter-processor interrupt, then two STARTUP IPIs whose vector is the page number of a real-mode trampoline below 1 MiB. The AP climbs from real mode to long mode through code you already have, loads its own GDT, TSS, IDT and stack, finds its per-CPU structure through the GS base and reports in. Spinlocks (`xchg` or `lock cmpxchg`, with `pause` in the loop) must disable interrupts when a handler takes the same lock. Changing a mapping that other CPUs may cache needs a TLB shootdown: an IPI to each of them and an acknowledgement from each.

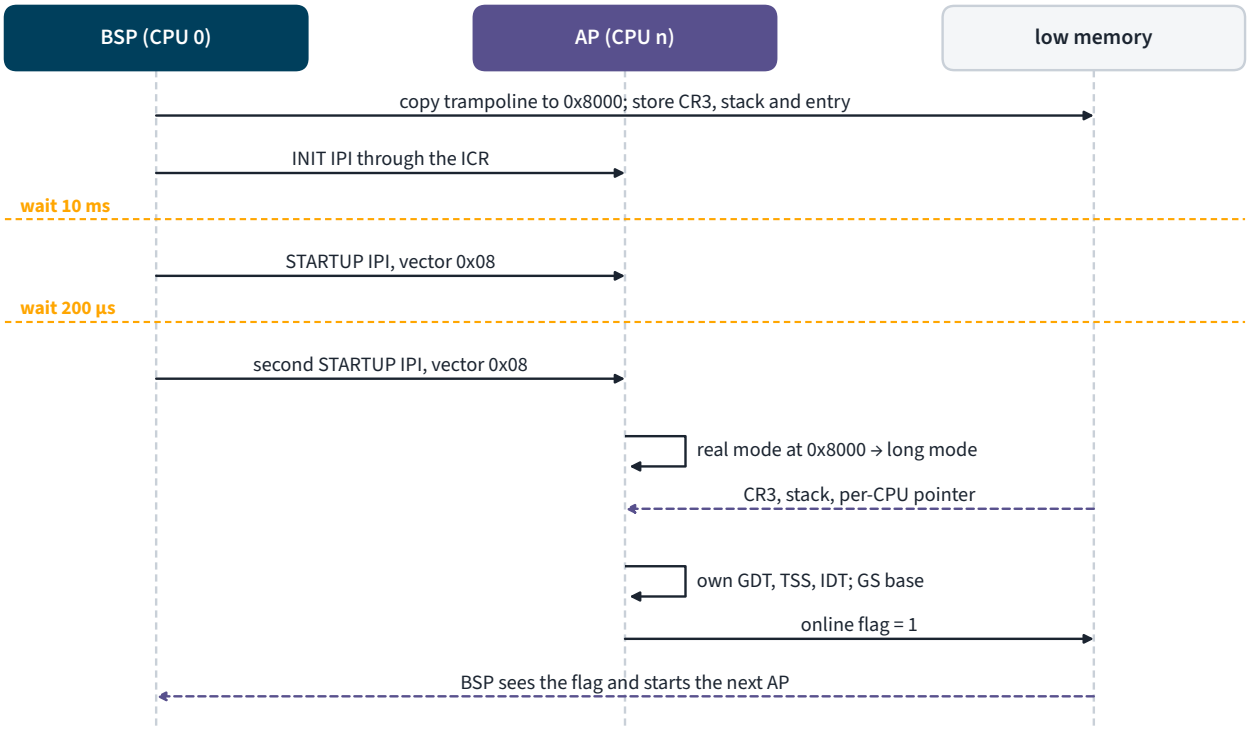


Figure 4.1.3. Bringing up one application processor. The trampoline's data area carries the page-table root and stack that the AP needs to reach long mode and C.

User mode. Load an ELF64 by mapping each segment with its permissions into a fresh address space, build a user stack with `argv` and `envp`, and enter ring 3 with `iretq` or `sysretq`. A `syscall` arrives at your `LSTAR` entry, which uses `swapgs` to reach the kernel stack, saves user registers, dispatches by number and validates every user pointer (in range, mapped, permitted) before copying. `fork` shares frames read-only with reference counts and copies on the first write fault; `execve` replaces the address space, `exit` frees it and `waitpid` reaps. Lazy allocation, copy-on-write and stack growth all live in the page-fault handler, which fixes the mapping or kills the process.

Devices and the block path. PCI enumeration over the configuration space of Lab 2.4.4 finds the NVMe controller's BAR0. Bring-up is a fixed sequence: clear `CC.EN` and wait for `CSTS.RDY` to fall; place the admin queues in memory and program `AQA`, `ASQ` and `ACQ`; set `CC.EN` and wait for ready; issue Identify; create an I/O queue pair. A command is 64 bytes in a submission queue followed by a write of the new tail to its

doorbell, with data buffers given as physical addresses in PRP entries. A completion is 16 bytes whose phase bit inverts on each pass round the ring; the host polls for it or takes an MSI-X interrupt, then writes the head doorbell.

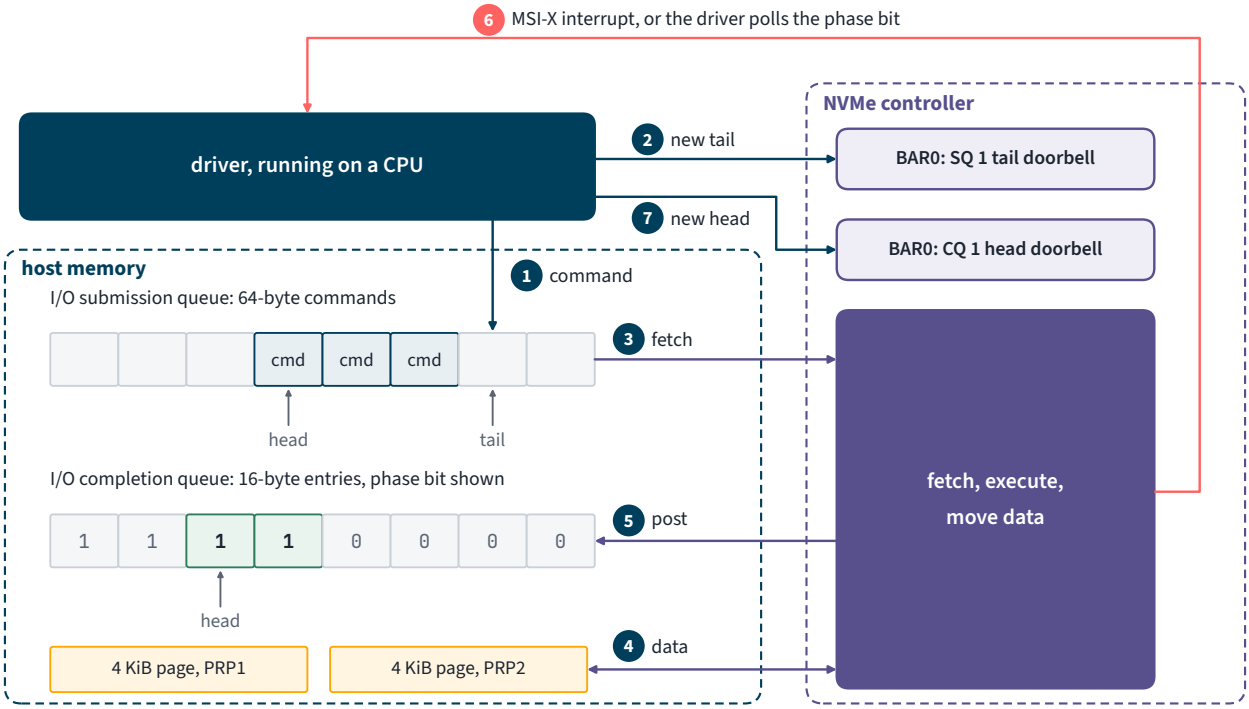


Figure 4.1.4. An NVMe read on one queue pair. The driver owns the submission queue's tail and the completion queue's head, the controller the other two; the phase bit tells the driver which completions are new on this pass.

Filesystems. Build a block cache, then a read-only FAT32 or ext2 parser that loads programs from the drive. A VFS of inode, file and directory operation tables lets the console, the disk and a pipe share one interface.

Debugging a kernel. In QEMU, use the gdbstub (`-s -S`) with your symbols, `-d int,cpu_reset` and `-d guest_errors`, `-no-reboot` so that a triple fault stops QEMU instead of restarting the guest, and the monitor's `info registers`, `info mem` and `info tlb`. On the server you have only the SOL console and the framebuffer through the BMC's virtual console.

Reading xv6. xv6 is under ten thousand lines, complete, and documented by a book. Read it fully over two weekends near the start and again once your kernel works, noting what it omits (it runs on several harts but has no NVMe driver) and what it keeps simple (the scheduler and the trap path).

Reading

- Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*: the Virtualisation and Concurrency parts, and Persistence chapters 36 to 40.
- Cox, Kaashoek and Morris, *xv6: a simple, Unix-like teaching operating system* (RISC-V edition) and its source, both in full.
- OSDev wiki: Bare Bones, Higher Half Kernel, Page Frame Allocation, Paging, APIC, APIC Timer, Symmetric Multiprocessing, System Calls, ELF, PCI, NVMe and FAT. Check each fact against the SDM or the specification, because an error in the wiki can cost days.
- Intel SDM volume 3A: paging, interrupt and exception handling, multiple-processor management (the MP initialisation protocol), the APIC and memory cache control; volume 3B's debug and performance-monitoring chapters as needed.
- NVM Express Base Specification (revision 2.4 at the time of writing): queues, controller initialisation, queue entries, PRPs, Identify and the Create I/O Queue commands. The NVMe over PCIe Transport and NVM Command Set specifications for doorbells and for Read, Write and Flush.

- Anderson and Dahlin, *Operating Systems: Principles and Practice*, chapters 4 to 7, for a second view of concurrency.
- Linux `Documentation/arch/x86/boot.rst`, if you boot Linux in Lab 4.1.6.

Labs

Each lab is a weekly milestone; commit and tag a working kernel at each. Compile with your `x86_64-elf` cross compiler and `-ffreestanding`, `-mno-red-zone`, `-mgeneral-regs-only` and `-mcmodel=kernel`. Interrupts would overwrite a red zone, and the SSE registers hold user state that you do not save on entry. Run `qemu-system-x86_64` with `-machine q35 -smp 4 -m 4G`, `-serial stdio` and `-no-reboot`, with OVMF for the UEFI loader and SeaBIOS for the BIOS one.

LAB 4.1.1 Boot, console and exceptions — 1 weekend

1. From the Lab 2.7.2 handoff, write a serial driver with `printk` and your own formatter (`%d %u %x %p %s %c`), a scrolling framebuffer console, and a `panic` that dumps registers and halts. Take the server's console port from the ACPI SPCR table: SOL is usually COM2 at 0x2F8, where QEMU's first port is at 0x3F8.
2. Install your own GDT with a TSS and your own IDT: 32 exception handlers that dump the frame, and stubs for vectors 32 to 255 that log and send an EOI.
3. Print the memory map and usable RAM; walk the RSDP and XSDT to the MADT and count the CPUs.
4. Build page tables for the higher half and the direct map, load CR3, unmap the loader's identity map and keep running.
5. Make your Lab 2.7.1 stage two collect the E820 map through interrupt 15h into the same structure. Boot one kernel binary from both loaders in QEMU, then from the UEFI loader on the server, signed with your db key if Lab 2.7.3 left Secure Boot on.

DONE WHEN

- [ ] The kernel boots from both loaders in QEMU and from the UEFI loader on the server, printing its memory map and CPU count.
- [ ] Each provoked exception (`#DE`, `#UD`, `#GP`, `#PF`) dumps a correct frame.

EVIDENCE FOR THE NOTEBOOK

- Serial logs from QEMU and the server; the boot-information structure; one annotated exception dump.

LAB 4.1.2 Memory — 1 weekend

1. Write a buddy allocator for orders 0 to 10 over the memory map, with `alloc_pages(order)`, `free_pages` and boot-time self-tests behind `CONFIG_SELFTEST` that allocate everything, free everything and check that blocks coalesce back to the starting counts.
2. Write `map(as, va, pa, flags)`, `unmap`, `protect` and `walk` with `invlpg`; create and destroy address spaces that share the kernel half.
3. Build a slab heap with size classes from 16 to 4,096 bytes (`kmalloc`, `kfree`), a path for large allocations, and poisoning of freed memory in debug builds.
4. Put an unmapped guard page under every kernel stack. An overflow faults while pushing the page-fault frame and becomes a double fault, so give `#DF` its own IST stack and make its handler name the thread and the address in CR2.

DONE WHEN

- [ ] The self-tests pass on every boot.

- [ ] A deliberate stack overflow is reported with the thread's name instead of corrupting memory.

EVIDENCE FOR THE NOTEBOOK
- Allocator statistics around the self-test; the overflow report.

LAB 4.1.3 Time, threads, scheduling and SMP — 2 weekends

1. Calibrate the local APIC timer and the TSC against the PIT or the HPET, comparing with CPUID leaf 0x15 where the processor reports it; tick at 1 kHz and implement a monotonic `ktime_ns()`.
2. Write kernel threads: `kthread_create(fn, arg)`, a context switch in assembly, a run queue, round-robin preemption from the tick, `yield`, `sleep_ms` on a timer wheel or sorted list, and wait queues with `wake_up`.
3. Add spinlocks that save and restore the interrupt flag, a lock-order assertion in debug builds, and a mutex built on a wait queue.
4. Copy a trampoline below 1 MiB, send INIT, STARTUP and STARTUP to each AP in the MADT, and bring each to long mode with its own GDT, TSS, stack and per-CPU structure behind the GS base, running the idle loop. Add per-CPU run queues with simple work stealing, and show N threads spinning on N cores in a `top`-style display.
5. Implement a TLB-shootdown IPI for kernel-half unmaps.

DONE WHEN
- [ ] On the server every core is online and threads migrate between cores.
- [ ] Hundreds of threads sleeping and waking run for an hour without a lockup.

EVIDENCE FOR THE NOTEBOOK
- Calibration numbers; the per-CPU display; the stress-test log with its duration.

LAB 4.1.4 User mode — 2 weekends

1. Load an ELF64 into a new address space with an `argv` stack and enter it through `iretq`.
2. Enter system calls through `syscall` and `LSTAR` with `swapgs` and the per-CPU kernel stack, numbered by your own ABI: `write`, `read`, `exit`, `getpid`, `fork` (copy-on-write with frame reference counts), `execve`, `waitpid`, anonymous `mmap`, `brk`, console `open` and `close`, and `sleep`.
3. Route every system call through one user-pointer validator, and test it with a program that passes a kernel address and must receive `-EFAULT` while the kernel carries on.
4. Serve lazy anonymous mappings, copy-on-write and stack growth in the page-fault handler, and kill the process with a clear message for anything else.
5. Build a userland: a small libc (`crt0`, system-call stubs, `printf`, `malloc` on `brk`), `init`, a shell that forks and execs commands from an in-memory table, and a few ported programs.

DONE WHEN
- [ ] The shell runs on QEMU and on the server.
- [ ] A fork bomb leaves the kernel responsive, refusing new processes cleanly when memory runs out, and the `-EFAULT` test passes.

EVIDENCE FOR THE NOTEBOOK
- Your ABI document; a shell session log; the page-fault kill messages.

LAB 4.1.5 PCI, NVMe and a filesystem — 2 weekends

Kit. QEMU with `-device nvme`; the server with the Kingston NV3 on its ICY DOCK adapter.

1. Enumerate PCI through ECAM, from the MCFG table as in Lab 2.5.4: every bus, device and function, with vendor, device, class, BARs and MSI-X capabilities. Match `lspci -nn` on the server.
2. Write the NVMe driver: map BAR0, reset and enable the controller through `CC`, `AQA`, `ASQ` and `ACQ`, issue Identify Controller and Identify Namespace, create one I/O queue pair, and read and write LBA ranges with PRP lists. Complete first by polling the phase bit, then by MSI-X with a wait queue. Test on QEMU's NVMe device first.
3. Print each controller's model and serial number, and record whether the NV3 asks for a host memory buffer in the `HMPRE` and `HMMIN` fields of Identify Controller.
4. Add a block cache with LRU replacement, a read-only FAT32 (or ext2) driver, and a VFS with `open`, `read` and `readdir` over the FAT volume and the console. Load user programs from the drive.
5. Measure sequential 4 KiB reads polled against interrupt-driven, and at queue depth 1 against 32; relate them to Little's law (Chapter 2.4) and keep them for Chapters 4.2 and 4.5.

> **SAFETY**
> The P4510 in the same server holds data you keep, and enumeration order follows slot numbering. Make the driver refuse every write unless Identify Controller's model number matches the NV3, and test that refusal before the first write. Partition and format the NV3 as FAT32 from Linux first.

DONE WHEN

- [ ] `ls` in your shell lists the FAT filesystem on the NV3 in the server.
- [ ] The table of polling against interrupts and queue depth 1 against 32 is in the notebook.

EVIDENCE FOR THE NOTEBOOK

- `lspci -nn` beside your enumeration; the decoded Identify data; the throughput table.

LAB 4.1.6 Choose one — 2 weekends

- **Networking:** a virtio-net or e1000 driver in QEMU, ARP, IPv4, UDP and a `udp_echo` program.
- **A RISC-V port:** the kernel on QEMU's `virt` machine over OpenSBI, with Sv39, the PLIC, the timer through SBI and `virtio-blk`, plus a page on what was specific to x86.
- **Booting Linux:** your Lab 2.7.2 loader also boots a Linux `bzImage` through the x86 boot protocol, with an initrd and a command line, as your Chapter 4.4 VMM will inside a virtual machine.

DONE WHEN

- [ ] The chosen option works and has a notebook page documenting it.

Problem set

1. Design your system-call ABI: register assignments, the error convention, how a call with seven arguments works, and how you would add a call without breaking old binaries. Compare with Linux and justify each difference.
2. A 64 GiB machine with 4 KiB pages: how much memory do the page tables take if every page is mapped once? With 2 MiB pages? Why does the direct map use 1 GiB pages where it can?
3. Write the TLB-shootdown protocol for unmapping a kernel-half page on a 32-core machine: what is sent, who acknowledges, what the sender may do meanwhile, and what happens if a target has interrupts disabled for a long time.

4. Why must interrupts be disabled between saving the outgoing thread's state and loading the incoming thread's stack pointer? Where can they be re-enabled, and what goes wrong one instruction earlier?
5. Copy-on-write `fork`: give the reference-count protocol for a frame shared by three processes as each writes in turn. What happens if one of them exits, and if one `mmap`s over the region?
6. NVMe completion queues use a phase bit instead of a valid flag. Explain why, and what the host must do when the queue wraps. A 4 KiB read takes 90 µs at queue depth 1: what IOPS does Little's law predict at depths 1 and 32 if nothing saturates, and why will your Lab 4.1.5 table disagree at 32?
7. Your spinlock spins on `xchg`. Why add `pause`? Why is a plain store with a compiler barrier enough for the unlock? Which ordering does x86 give you for free here that RISC-V does not?
8. A user program passes `write` a 4 KiB buffer that spans a mapped page and an unmapped one. Show the check sequence that avoids both a kernel fault and a time-of-check-to-time-of-use race with a thread that `munmap`s concurrently.
9. Measure your kernel's interrupt latency and context-switch cost on the server with the TSC, and explain each contribution.

Deliverables and stretch

Deliverables. A tagged kernel per milestone that boots on QEMU and the server, with a README of what works, what does not and how to run the self-tests; the NVMe throughput table; your ABI document.

Stretch. POSIX signals, delivered on return to user mode through a trampoline, so that `SIGSEGV` reaches a handler instead of killing the process. A filesystem write path with a journaling scheme you design, and a crash test that kills QEMU mid-write and checks consistency on remount. Make your loader measure the kernel into PCR 9 through the EFI TCG2 protocol, as GRUB does for the files it loads, and check that your Lab 2.7.4 event-log replay still matches.

COMPLETION CHECKLIST

- ☐ **Lab 4.1.1:** one kernel boots from both loaders in QEMU and from the UEFI loader on the server, with correct exception frames.
- ☐ **Lab 4.1.2:** allocator self-tests pass on every boot; a stack overflow is reported by name.
- ☐ **Lab 4.1.3:** every server core online with threads migrating; an hour of sleep-and-wake stress without a lockup.
- ☐ **Lab 4.1.4:** the shell on QEMU and the server; a fork bomb survived; `-EFAULT` returned.
- ☐ **Lab 4.1.5:** `ls` of the FAT volume on the NV3; the polling, interrupt and queue-depth table.
- ☐ **Lab 4.1.6:** the chosen option working and documented.
- ☐ **Problem set:** all nine answered.
- ☐ **You can explain**, without notes: the page-fault path, why spinlocks disable interrupts, what a context switch saves, how an AP comes up, and an NVMe read from doorbell to completion.

CHAPTER 4.2

Linux kernel internals

TIME	3.5 weeks
YOU BUILD	Kernels built, booted and debugged on QEMU, the lab server and the Raspberry Pi 5, and a crash dump analysed; traces from `write(2)` to a UART and from sysfs to the TMP117; a character-device module with an RCU list, clean under KASAN, KCSAN and lockdep; scheduler and memory measurements; two bisections, one on hardware; and the signed node kernels that Chapter 4.7 ships, with OMH's kernel policy.
NEEDS	The lab server with the P4510 and the NV3, the Pi 5 with its Lab 2.7.5 TMP117 overlay and a 3.3 V USB-to-serial adapter, QEMU, the long-term and Raspberry Pi kernel trees, and `fio`, `perf`, `bpftrace`, `trace-cmd`, `rt-tests`, `virtme-ng` and `crash`.

WHAT THIS BUYS OMH

Every OMH module ships a Linux kernel that you configure, harden, patch, sign and update for years. Compute needs the scheduler's isolation and the memory manager's huge pages and NUMA policy. Storage and Blocks run in the block layer and the NVMe driver, Clusters is built on namespaces and cgroups, and Signals makes `perf`, ftrace and eBPF into a product. Chapter 4.1 showed the mechanisms in a kernel you wrote. This chapter teaches you to read a large codebase efficiently, trace it live on both of Module Zero's architectures, find the commit that broke something, and write code that runs safely inside it. Its last lab builds the node kernels that Chapter 4.7 ships.

SKIP TEST

1. Trace `write(2)` on a tty from the system-call entry to the UART driver, naming the subsystems crossed.
2. What does RCU guarantee to readers, what does it require of writers, and why is it everywhere in the networking and VFS code?
3. Describe blk-mq's queue model and why it matches NVMe better than the old single-queue block layer.
4. A page fault hits a file-backed `mmap` region. What does the kernel consult, in what order, and when does it touch the disk?
5. A vendor's out-of-tree module taints the kernel. What does that mean technically, and what are the consequences for OMH shipping one?

If all five are easy, do Labs 4.2.2 to 4.2.4 and 4.2.7.

Core ideas

The source tree. `arch/` holds entry code, paging and interrupt controllers, and `kernel/` the scheduler, time, locking, RCU and tracing. The other main directories are `mm/`, `fs/`, `block/`, `drivers/`, `net/`, `include/linux/` and `Documentation/`. Browse the code with `clangd` over `make compile_commands.json`, and learn why code is as it is from `git log -S`, `git blame` and LWN's kernel index. Kconfig configures (`menuconfig`, `localmodconfig`, `scripts/config`); Kbuild produces the image, modules and an optional initramfs.

Processes and threads. `task_struct` represents both; a thread shares `mm`, files and signal handlers through `clone` flags. `fork` copies on write and `execve` loads through a `binfmt` handler. Namespaces virtualise global

resources and cgroup v2 limits CPU, memory and I/O; with a root filesystem they make a container, which is all a Clusters node runs.

Scheduling. Classes in priority order are stop, deadline, real-time, fair and idle, and since 6.12 `sched_ext` lets a BPF program supply one more. The fair class has been EEVDF since 6.6, running the eligible task with the earliest virtual deadline. Per-CPU run queues balance across scheduling domains built from the topology. `isolcpus`, `nohz_full`, `rcu_nocbs`, IRQ affinity and cpusets isolate latency-critical work; Compute will pin vCPU threads with them.

Memory. Learn nodes and zones, the buddy allocator (`alloc_pages`), SLUB (`kmalloc`, `kmem_cache_create`), `vmalloc`, the page cache, and VMAs with the fault path through `handle_mm_fault`. Then learn reclaim through the multi-generational LRU, transparent huge pages, NUMA policy, and memory cgroups with the OOM killer, and read `/proc/meminfo` and `/proc/vmstat` fluently. The Pi's kernel uses 16 KiB pages, so its huge page is 32 MiB and every page-size-dependent number changes.

VFS and the block layer. The VFS is built from `super_block`, `inode`, `dentry` and `file` with their operations tables, with the page cache behind `address_space`. A `bio` enters blk-mq in the submitting CPU's software context, which maps to a hardware context whose `queue_rq` hands it to the driver; for NVMe a hardware context is one queue pair with its own MSI-X vector. An I/O scheduler (`none`, `mq-deadline`, `bfq`) may sit between them, device-mapper stacks targets above (dm-crypt is "encrypted at rest"), and io_uring's rings mirror NVMe's queues.

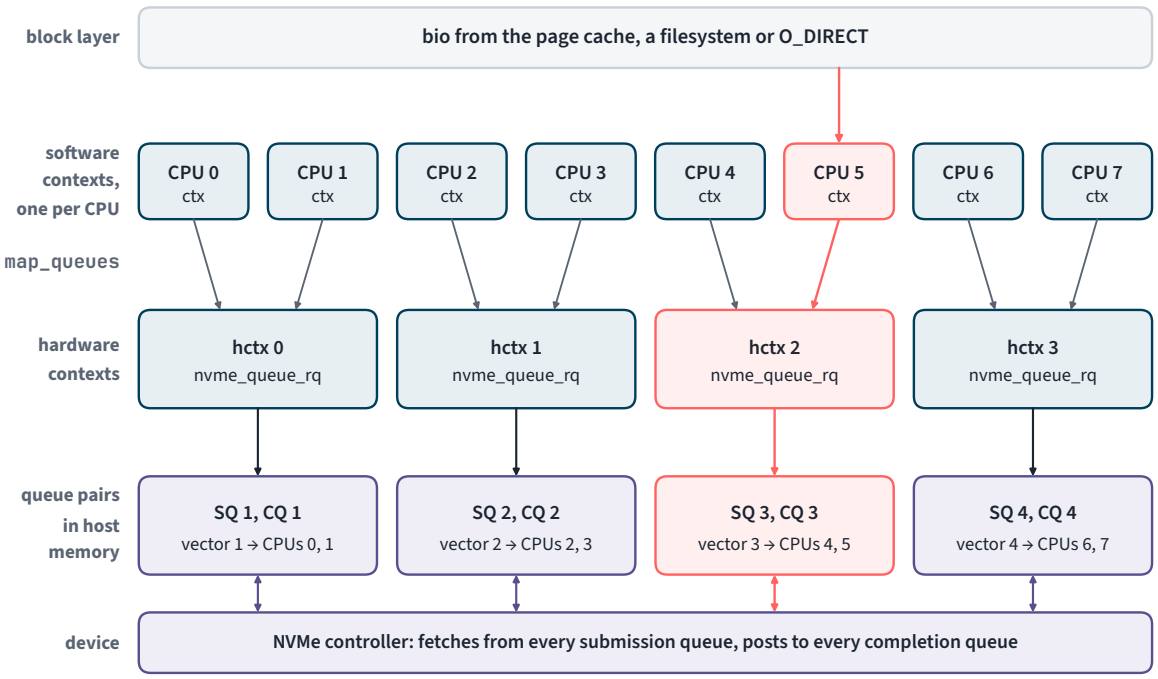


Figure 4.2.1. blk-mq with eight CPUs and four NVMe queue pairs. The highlighted request is submitted on CPU 5 and completes on CPU 4 or 5 through vector 3.

Interrupts and deferred work. Hard IRQ handlers run with local interrupts disabled and stay short. Softirqs (networking, block, timers, RCU) run on interrupt return or in `ksoftirqd`, and tasklets are giving way to BH workqueues. Ordinary workqueues and threaded IRQs run in process context and may sleep; NAPI mixes interrupts with polling. `/proc/interrupts` and `/proc/softirqs` show where the time went.

Concurrency. The primitives are spinlocks (`spin_lock_irqsave` when a hard IRQ shares the data, `spin_lock_bh` when a softirq does), sleeping mutexes, reader-writer locks, seqlocks, per-CPU variables, atomics and RCU. An RCU reader brackets its walk with `rcu_read_lock()` at almost no cost; a writer publishes with `rcu_assign_pointer()` and frees the old version only after every reader that might hold it has finished (`synchronize_rcu()` or `call_rcu()`). For ordering, `smp_load_acquire()` with `smp_store_release()` is usually what you want. Lockdep checks lock order, KCSAN finds data races and KASAN finds use-after-free; KCSAN and KASAN exclude each other, so keep two debug configurations.

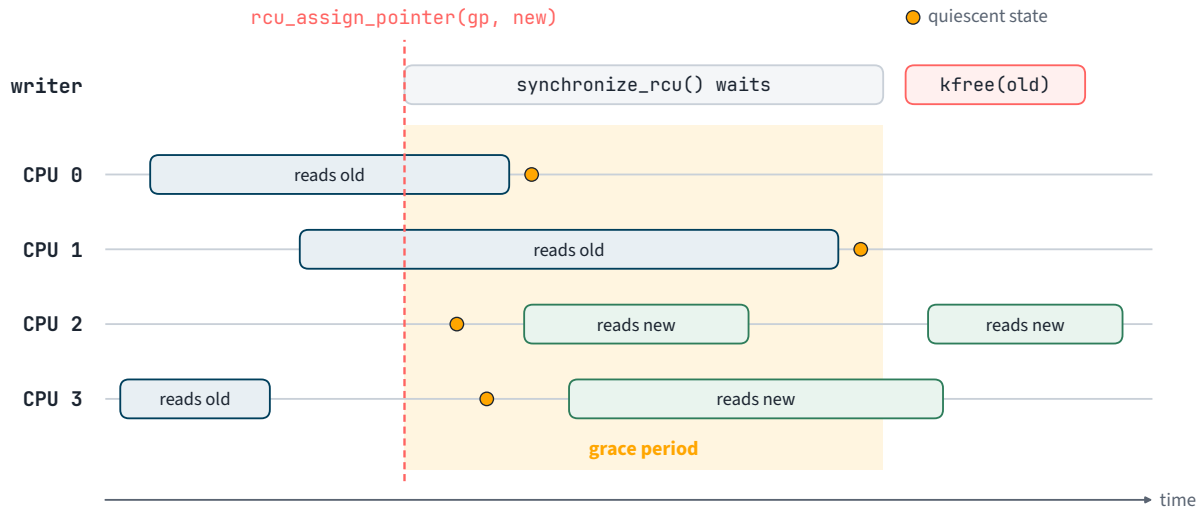


Figure 4.2.2. An RCU grace period. The writer waits only for readers already running when it published; later readers see the new version and do not extend the wait.

Time. The kernel keeps time with clocksources (TSC, HPET, the Arm generic timer), clockevents, hrtimers, tickless idle, `ktime_get()` and jiffies. Timekeeping is subtle, so use the kernel's functions.

System calls and entry. Read `entry_SYSCALL_64` in `arch/x86/entry/entry_64.S`, the system-call table, `SYSCALL_DEFINE`, `copy_from_user()` with SMAP (PAN on arm64), the vDSO for `clock_gettime`, and the `ptrace` and `seccomp` hooks on the path.

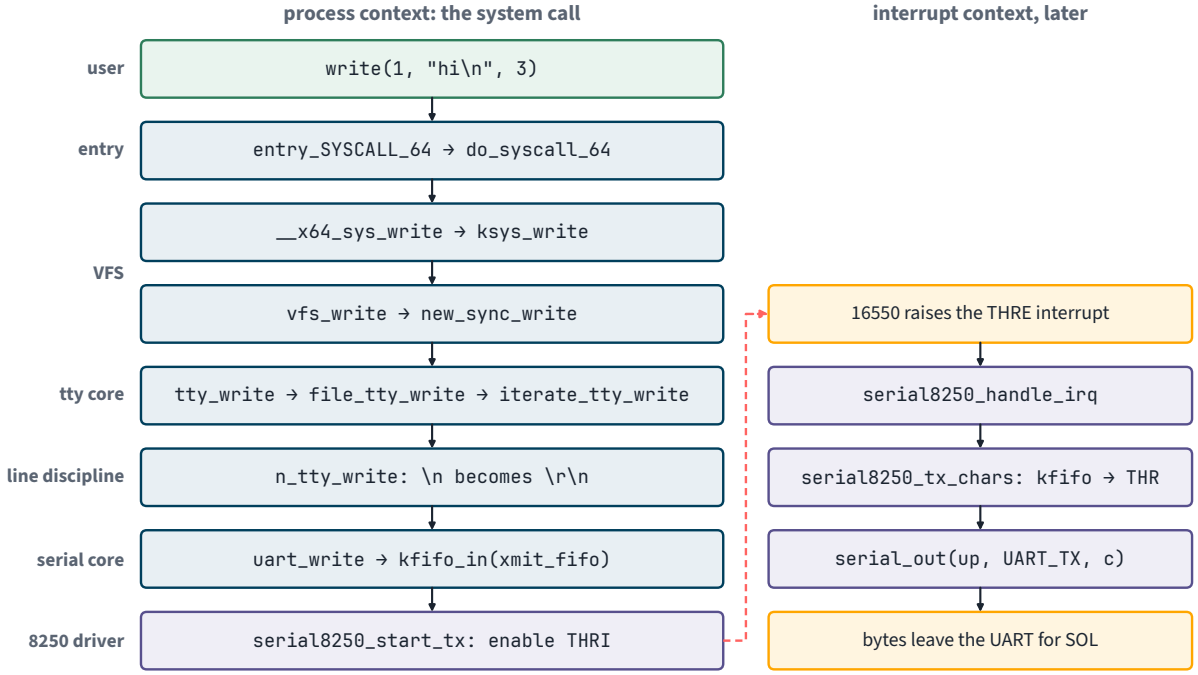


Figure 4.2.3. `echo hi > /dev/ttyS1` on the server. The system call only queues bytes and enables the transmit interrupt; the 8250 interrupt handler moves them into the UART.

The device model. Each `kobject` has sysfs as its view. A `bus_type` matches each `device` to a `device_driver` and probes it, and `udev` reacts to the resulting uevents. Chapter 4.3 works in this layer.

Modules. Modules are loadable objects linked against exported symbols (`EXPORT_SYMBOL`, `EXPORT_SYMBOL_GPL`), with parameters, init and exit functions, version magic and optional signatures. Out-of-tree, unsigned and proprietary modules taint the kernel with flags O, E and P, and upstream and distributions decline bug

reports from tainted kernels. A vendor module must be rebuilt for every kernel update and signed when the boot chain enforces signatures.

Observability. The tools are `printk`, `dynamic_debug`, ftrace (`function_graph`, tracepoints, `trace-cmd`), kprobes and uprobes, eBPF through `bpftrace` and libbpf, `perf`, kgdb, and kdump with `crash`. Signals is a product built from these.

Security. The mechanisms are Linux Security Modules (SELinux, AppArmor, Landlock), IMA (file hashes extended into the TPM, Chapter 2.7), lockdown, seccomp, and hardening options such as `CONFIG_HARDENED_USERCOPY`, `CONFIG_RANDOMIZE_BASE`, `CONFIG_STACKPROTECTOR_STRONG` and `CONFIG_INIT_ON_ALLOC_DEFAULT_ON`. Mainline does not enable lockdown under Secure Boot (distributions patch that in), so your node kernels force it. On a storage appliance, every kernel configuration choice affects security.

The process. Development runs through mailing lists, `git format-patch`, `scripts/checkpatch.pl`, `MAINTAINERS`, the two-week merge window, `-rc` releases, and stable and long-term branches (6.18 is the newest long-term branch at the time of writing). The Raspberry Pi kernel follows them with its own board-support patch stack, the position OMH will be in. `git bisect` finds a regression among N commits in about $\log_2 N$ builds, and the fix's `Fixes:` tag names the culprit. A small patch stack on one long-term branch keeps this work bounded.

Reading

- Love, *Linux Kernel Development*, 3rd ed., quickly, in the first week. Its details date from 2.6.34, and its account of the kernel's structure still holds.
- Bootlin's "Linux kernel and driver development" slides on build, debugging and concurrency.
- 0xAX, *Linux Insides*: the booting and interrupts chapters, for the x86 entry path.
- In `Documentation/`: `core-api/`, the EEVDF and cgroup v2 pages, `admin-guide/mm/`, `RCU/whatisRCU.rst`, `memory-barriers.txt`, `trace/ftrace.rst`, the KASAN and KCSAN pages in `dev-tools/`, the guide to verifying and bisecting regressions in `admin-guide/`, and `process/submitting-patches.rst`.
- McKenney, *Is Parallel Programming Hard, And, If So, What Can You Do About It?*: deferred processing (RCU) and memory ordering.
- Gregg, *BPF Performance Tools*: chapters 1 to 5 and 9 (disk I/O).
- LWN.net's kernel index on EEVDF, the multi-generational LRU, io_uring, blk-mq, `sched_ext` and RCU. Subscribe; it is the trade journal.

Labs

Use the newest long-term branch on the server and the Raspberry Pi tree's default branch on the Pi, cross-compiling the Pi's kernels on the server (`ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-`, `bcm2712_defconfig`). The server's SOL console is usually `ttyS1`; the Pi's is `ttyAMA10` on its debug header, or GPIO 14 and 15 with `enable_uart=1`.

LAB 4.2.1 Build and boot a kernel, then analyse a crash dump

1 weekend

Kit. The server, the Pi 5 with a USB-to-serial adapter, QEMU.

1. Build the long-term kernel from `defconfig` with `CONFIG_DEBUG_INFO_DWARF5` and `CONFIG_GDB_SCRIPTS`, timing the build. Boot it in QEMU with a BusyBox initramfs, `-append "console=ttyS0 nokaslr"` and `-s -S`; in `gdb vmlinux` run `lx-symbols`, break on `vfs_open`, take a backtrace and print `$lx_current().comm`.
2. Boot it on the server with `console=ttyS1,115200`. Enable kdump (`crashkernel=`, `kdump-tools`), crash the machine with `echo c > /proc/sysrq-trigger`, and find the panicking task's backtrace with `crash`.
3. Build the Pi tree on the server and on the Pi, timing both. Name the new image with `kernel=` in a copy of `config.txt` called `tryboot.txt` and boot it with `sudo reboot '0 tryboot'`, which falls back to `config.txt` after a hang. Check that `getconf PAGESIZE` prints 16384.

DONE WHEN

- [ ] Your kernels boot in QEMU under GDB, on the server over SOL and on the Pi over serial.
- [ ] One crash dump is analysed with `crash` down to the panicking task's backtrace.

EVIDENCE FOR THE NOTEBOOK

- The three build times; the GDB session; the `crash` output.

LAB 4.2.2 Tracing the kernel

1 weekend

1. On the server, trace one `echo hi > /dev/ttyS1` with `function_graph` and `set_graph_function` set to `ksys_write`, and annotate each layer of the graph, as in the write-path figure; then trace `serial8250_tx_chars` to catch the interrupt that drains the buffer.
2. On the Pi, trace a read of the TMP117's `in_temp_raw` from `iio_read_channel_info` through `tmp117_read_raw` and the I²C core to `i2c_dw_xfer` and its completion interrupt; the DesignWare controller sits in RP1, across PCIe. Enable `CONFIG_FUNCTION_GRAPH_TRACER` if it is missing.
3. With `bpftrace`, histogram the time from `block:block_rq_issue` to `block:block_rq_complete` under `fio` 4 KiB random reads at queue depths 1, 8 and 32, on the NV3 beside your Lab 4.1.5 table and then on the P4510.
4. Run `perf record -g` under `fio` and under `iperf3` across the X520 (ports joined by a DAC, one in its own network namespace), and explain three top kernel symbols. Count context switches, migrations, page faults and cache misses with `perf stat` on a kernel build, with and without `taskset`.
5. Record `sched_switch` and `sched_wakeup` with `trace-cmd` during `cyclictest`, and find the worst latency and its cause in KernelShark.

DONE WHEN

- [ ] The annotated write path from the server and TMP117 path from the Pi are in the notebook.
- [ ] The histograms sit beside your Lab 4.1.5 numbers, and the KernelShark finding is explained.

EVIDENCE FOR THE NOTEBOOK

- Trimmed, annotated traces; the histograms; `perf` output with your explanations.

LAB 4.2.3 Modules, from hello to a character device under the sanitisers

2 weekends

1. Write a `hello.ko` that uses `module_param`, `MODULE_LICENSE` and `pr_info`; load it, read its parameters under `/sys/module/hello/` and decode the taint mask in `/proc/sys/kernel/tainted`. Add a procfs file and a sysfs attribute for a read-write counter.
2. Write a character device (`alloc_chrdev_region`, `cdev`, and `file_operations` with `open`, `read`, `write`, `unlocked_ioctl` and `poll`) with per-open data, a kernel thread filling a ring buffer, `poll` waking readers through a wait queue, and a rate `ioctl` numbered with `_IOW` and `_IOR` in a uAPI header.
3. Make it SMP-safe with a spinlock, then make the readers lockless with `smp_load_acquire` and `smp_store_release`. Stress it with many readers and writers on the server and on the Pi, whose weaker ordering (Lab 2.4.3) exposes missing barriers that have no visible effect on x86.
4. Build one kernel with KASAN and lockdep and one with KCSAN and lockdep. Plant a use-after-free on close, a lock-order inversion and an unlocked counter update; read each tool's report, then fix all three.

DONE WHEN

- [ ] The stress test runs clean under both debug kernels and on the Pi.
- [ ] The notebook holds the KASAN, KCSAN and lockdep reports from the planted bugs.

EVIDENCE FOR THE NOTEBOOK
- The module, uAPI header and stress test; the three reports.

LAB 4.2.4 RCU in your module — 1 weekend

1. Add an RCU-protected list of subscribers that readers walk with `list_for_each_entry_rcu` inside `rcu_read_lock` and writers change under a spinlock, freeing entries with `kfree_rcu`.
2. Stress it from user space, adding and removing subscribers while readers iterate, and confirm with `perf` that readers take no lock.
3. Swap RCU for a reader-writer lock, and plot reader throughput for both at 1, 4 and 16 readers on the server and at 1 and 4 on the Pi.
4. Read the RCU requirements in `Documentation/RCU/Design/` and the header comment of `kernel/rcu/tree.c` until you can explain why `synchronize_rcu` can take milliseconds.

DONE WHEN
- [ ] The plot shows RCU readers scaling where reader-writer-lock readers do not.
- [ ] You can explain a grace period in one paragraph.

EVIDENCE FOR THE NOTEBOOK
- The plot and its raw numbers; the paragraph.

LAB 4.2.5 Scheduler and memory experiments — 1 weekend

1. Isolate four server cores with `isolcpus`, `nohz_full` and `rcu_nocbs`, move IRQ affinities off them, and compare the maximum `cyclictest` latency there and on a busy core.
2. Pointer-chase 8 GiB with transparent huge pages set to `always`, `madvise` and `never`, counting `dTLB-load-misses` with `perf stat`. Repeat over 2 GiB on the Pi, where a huge page is 32 MiB.
3. Run the benchmark under a `memory.max` of half its footprint and watch `memory.stat`, reclaim and the OOM kill; then use `memory.high` and watch throttling.
4. Compare `numactl --cpunodebind=0 --membind=1` with local placement for memory bandwidth, and read `numastat`.
5. Give two kernel builds cgroup v2 CPU weights of 100 and 900 and measure their wall times.

DONE WHEN
- [ ] The latency, TLB, cgroup and NUMA measurements are in the notebook, each with an explanation.

EVIDENCE FOR THE NOTEBOOK
- The `cyclictest` histograms; the measurement tables.

LAB 4.2.6 Bisect a change, then a regression on hardware — 1 weekend

Kit. The server, QEMU, `virtme-ng`.

1. Write a `git bisect run` script that builds with `vng --build` (with `CONFIG_SCHED_DEBUG` set once through `vng --kconfig --configitem`), boots with `vng --user root`, lists `/sys/kernel/debug/sched/` after mounting debugfs, and exits 0 (good), 1 (bad) or 125 (skip).
2. Between v6.5 and v6.6 the fair class became EEVDF and the CFS tunable `latency_ns` left that directory. Bisect for its removal, timing each step. Confirm the commit with `git log -S '"latency_ns"' v6.5..v6.6` on `kernel/sched/debug.c`, and explain why bisection is still needed when no string names a change.

3. Script a branch of 40 commits from your long-term tree plus one, at a random position recorded in a file you leave unopened, that adds `udelay(200)` to `nvme_queue_rq`. Bisect it on the server: build, install and `kexec` into each kernel to skip POST, then compare 10 seconds of `fio` 4 KiB random reads at queue depth 1 on the NV3 with a threshold. Under Secure Boot, sign each image with your db key and use `kexec -s`.

DONE WHEN

- [ ] `git bisect run` finds the commit that removed `latency_ns`, matching `git log -S`.
- [ ] The hardware bisection finds the planted commit, confirmed against the recorded position.

EVIDENCE FOR THE NOTEBOOK

- Both `git bisect log` outputs; the scripts; the time per step.

LAB 4.2.7 The node kernel and the patch process

2 weekends

Kit. The server, the Pi 5, your Lab 2.7.3 Secure Boot keys and Lab 2.7.5 FIT key.

1. Configure the server from `defconfig` plus `make localmodconfig`, and the Pi from `bcm2712_defconfig` plus `localmodconfig` with `ARCH=arm64` and `LSMOD=` naming the Pi's saved `lsmod` output. Trim both toward a headless node for Chapters 4.4 and 4.5 (keep KVM, VFIO, NVMe, the NICs and the TPM; drop sound, graphics and wireless), keep each as a `merge_config.sh` fragment, and justify ten options added and ten removed. The Pi's fragment also serves Board B, whose Compute Module 5 has the same BCM2712.
2. Harden both: set `CONFIG_LOCK_DOWN_KERNEL_FORCE_INTEGRITY` and `CONFIG_MODULE_SIG_FORCE` with your own signing key, add the options above and IMA for your Lab 2.7.4 verifier, and drop `/dev/mem`. Decide whether the arm64 node keeps 16 KiB pages, and justify every `kernel-hardening-checker -c` failure you accept.
3. On the server, boot under Secure Boot a unified kernel image built with `ukify` and signed with your db key; on the Pi, boot a FIT signed with your Lab 2.7.5 key through U-Boot. On both, check that `/sys/kernel/security/lockdown` shows `[integrity]` and that `insmod` refuses an unsigned module.
4. Pick a driver and fix one legitimate warning from `scripts/checkpatch.pl`, a documentation error or a missing `static`. Write the message with a rationale and `Signed-off-by:`, run `git format-patch` and `scripts/get_maintainer.pl`, and read in `Documentation/process/` how it would be merged. Sending it is encouraged.
5. Write OMH's one-page kernel policy: the branch per architecture, the patch stack, signing and fleet rollout of kernels and modules, CVE handling, and who bisects a regression.

DONE WHEN

- [ ] Both node kernels boot signed, in integrity lockdown, and refuse unsigned modules.
- [ ] Fragments, change logs and the hardening report are committed for Chapter 4.7's image.
- [ ] The patch is formatted and the policy is committed.

EVIDENCE FOR THE NOTEBOOK

- Fragments and change logs; the checker output; the refusals; the patch; the policy.

Problem set

1. List each layer, and its key function or structure, from a cold `read(2)` on ext4 over NVMe to the doorbell.
2. Why can RCU readers never block a writer, what breaks if one sleeps under `rcu_read_lock()`, and what does SRCU change?
3. Map blk-mq's software and hardware contexts onto NVMe queue pairs and MSI-X vectors for 56 logical CPUs and a drive offering 32 I/O queue pairs. Where is the mapping one to one, and where is it not?

4. A process `mmap`s a 1 GiB file and touches every page. Describe the fault path, page-cache lookups, readahead, and major against minor faults. Count the faults with 4 KiB and with 16 KiB pages, without fault-around and with the default 64 KiB `fault_around_bytes`. What does `MAP_POPULATE` change?
5. What does taint mean for a bug report, for kdump analysis and for a support contract? How would you take an OMH driver upstream, and what would you ship meanwhile?
6. Give a concrete deadlock for each wrong choice among `spin_lock`, `spin_lock_bh` and `spin_lock_irqsave`.
7. Explain what `nohz_full` does to a core, what still interrupts it, and why a pinned vCPU thread benefits. With a 250 Hz tick costing 2 µs, how much time per second does it return to each isolated core?
8. A memory cgroup at `memory.max` runs out. Walk what reclaim tries before the OOM killer, and explain why `memory.high` behaves better on a multi-tenant node.
9. Bisecting 2,900 commits: how many steps, and how long at 6 minutes per build and 4 per boot and test, with and without the 3 minutes of POST that `kexec` saves? What if a step will not build?

Deliverables and stretch

Deliverables. The node-kernel fragments and change logs; the RCU module with its uAPI header, stress test and sanitiser reports; the Lab 4.2.2 and 4.2.5 traces and tables; both bisection logs; the patch and policy.

Stretch. A libbpf program exporting per-queue NVMe latency from the driver's tracepoints, a Signals prototype. An upstream fix backported to your long-term branch, resolving the conflict by hand. A `sched_ext` scheduler, written or adapted from `tools/sched_ext/`, that pins chosen tasks to isolated cores.

COMPLETION CHECKLIST

- ☐ **Lab 4.2.1:** kernels under GDB in QEMU, on the server and on the Pi; a crash dump analysed.
- ☐ **Lab 4.2.2:** both paths annotated; histograms beside Lab 4.1.5; the KernelShark finding.
- ☐ **Lab 4.2.3:** the module clean under both debug kernels and on the Pi; the three reports.
- ☐ **Lab 4.2.4:** RCU readers scaling where reader-writer-lock readers do not.
- ☐ **Lab 4.2.5:** latency, TLB, cgroup and NUMA measurements explained.
- ☐ **Lab 4.2.6:** the EEVDF commit and the planted regression found by bisection.
- ☐ **Lab 4.2.7:** both node kernels signed and locked down; patch and policy committed.
- ☐ **Problem set:** all nine answered.
- ☐ **You can explain**, without notes: the `write(2)` and NVMe paths, RCU, lock variants, taint and lockdown.

CHAPTER 4.3

Linux device drivers

TIME	5 weeks
YOU BUILD	Mainline TI drivers bound by overlay and read line by line; an upstream-ready OPT4048 IIO driver; an ADS1220 driver streaming the load cell through a data-ready trigger; an EMC2101 hwmon driver through which the kernel runs a fan curve; a regmap driver that makes your FPGA a gpiochip, interrupt controller and character device; a PCIe driver with MSI and DMA; a minimal 82599 driver and a VFIO userspace driver; an NVMe debugfs patch.
NEEDS	The Raspberry Pi 5 and its kernel tree from Chapter 4.2; TMP117, INA228 and OPT4048 (Adafruit 6335) breakouts; the ADS1220 load-cell rig from Lab 1.2.4; the EMC2101 breakout (Adafruit 4808) with the Noctua NF-A8 PWM fan and a 12 V bench supply; the ULX3S with Lab 2.2.5's bitstream; logic analyser; QEMU; the lab server with the X520-DA2, its DAC and the P4510.

WHAT THIS BUYS OMH

An OMH module is a Linux machine surrounded by hardware Linux must drive: fans and thermal sensors, power monitors, PSUs on PMBus, drive-bay signals, the management board, NICs, NVMe drives and FPGA glue logic. Some of it has upstream drivers; the rest will be OMH's to write and maintain. This chapter presents your FPGA, whose register map you began in Lab 2.2.4, as standard kernel devices, and makes the TI sensors from Chapter 1.3 into IIO and hwmon devices that OpenBMC can read. The PCIe and VFIO labs are the base of Compute's device passthrough in Chapter 4.4.

SKIP TEST

1. For an I²C sensor on the Pi, which `bus_type` is involved, how did the device come to exist, how was the driver matched, and what does `probe` receive?
2. When is `dma_alloc_coherent` correct and when `dma_map_single`, what does the IOMMU change, and what bug appears on a non-coherent Arm SoC that x86 hides?
3. Write a device-tree node for an SPI ADC with its data-ready line on a GPIO. What validates the binding, and what validates the node?
4. MSI, MSI-X and INTx: what does the driver do differently for each, and why does a multi-queue device need MSI-X?
5. Why must an `ioremap`ped BAR be accessed with `readl` and `writel` instead of through the pointer, and what does `writel` do on Arm that a plain store does not?

If all five are easy, do Labs 4.3.2, 4.3.5 and 4.3.8 only.

Core ideas

What a driver does. A driver connects a bus to a subsystem. The bus (platform, I²C, SPI, PCI) discovers a device or is told about it and calls a matching driver's `probe`; the subsystem (IIO, hwmon, gpio, net, block) presents the device to userspace through a standard ABI. Chapter 4.2 showed the machinery. The driver supplies ID tables (`of_match_table`, `i2c_device_id`, `pci_device_id`) and a `probe` function. Resources taken through the managed `devm_*` calls need no unwinding, and `probe` returns `-EPROBE_DEFER` while a regulator, clock or IIO channel it needs is missing. Good drivers are short. The mainline TMP117 driver is about 250 lines.

Device tree and bindings. Undiscoverable hardware is a device-tree node with `compatible`, `reg`, `interrupts`, `*-supply`, `*-gpios` and its own properties, linked to other nodes by phandle. Each binding is a YAML schema under `Documentation/devicetree/bindings/`; `make dt_binding_check` checks schemas and `make dtbs_check` checks trees against them. On the Pi, an overlay compiled with `dtc -@` and loaded by `dtoverlay=` in `config.txt`, or from U-Boot as in Lab 2.7.5, targets the board's `i2c_arm`, `spi0` and `gpio` labels. On an ACPI server the device arrives through `_HID` and `_CRS`, or `PRP0001` with a `_DSD` carrying the same `compatible`, so a driver using `device_property_*` serves both.

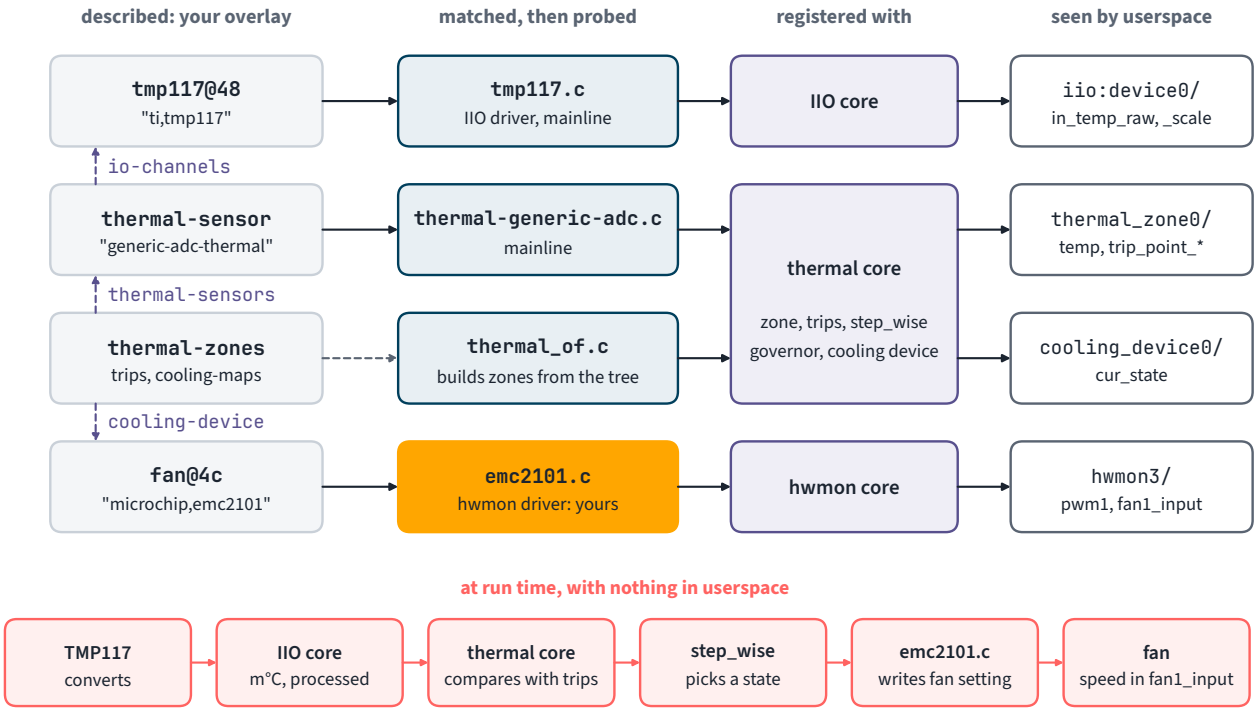


Figure 4.3.1. One overlay, three mainline drivers and yours, linked by phandles; at run time the kernel alone runs the fan curve. Lab 4.3.4 builds this.

Buses and regmap. I²C offers `i2c_transfer` and SMBus helpers; SPI offers `spi_sync` over a list of transfers. Above both, you declare to regmap the register and value widths, flag bits, padding, volatile registers and cache, and it does the framing, locking, caching and a debugfs dump. `regmap-irq` builds an interrupt controller from status and mask registers and `gpio-regmap` builds a gpiochip, so a driver for simple glue logic can be a few dozen lines.

Interrupts. `request_threaded_irq` splits a handler into a hard part that runs with interrupts disabled and a thread that may sleep. A chip behind I²C or SPI is reachable only from the thread, so an interrupt controller behind such a bus records mask changes, writes them in `irq_bus_sync_unlock`, and dispatches with `handle_nested_irq`. An IRQ domain maps its hardware interrupt numbers to Linux ones, so another driver, or `gpiomon`, can take an interrupt from an FPGA pin.

Subsystems and their ABIs. An IIO channel has a type, an optional modifier such as `IIO_MOD_X`, and attributes such as `raw`, `scale` and `processed`, so userspace computes real units without knowing the part. Buffered capture adds a trigger whose top half stores a timestamp and whose thread reads the device and pushes a scan with `iio_push_to_buffers_with_ts`. hwmon uses a fixed vocabulary (`temp1_input` in millidegrees, `fan1_input` in RPM, `pwm1` from 0 to 255). `sensors` and OpenBMC's sensor daemons read it, and Redfish reports through those daemons. The PMBus core makes a PMBus device into hwmon in a few dozen lines. The thermal framework joins sensors to cooling devices through zones and trip points in the device tree. When no subsystem fits, a misc character device with a versioned uAPI header is the fallback. Userspace may rely on sysfs, which is a stable ABI, but not on debugfs.

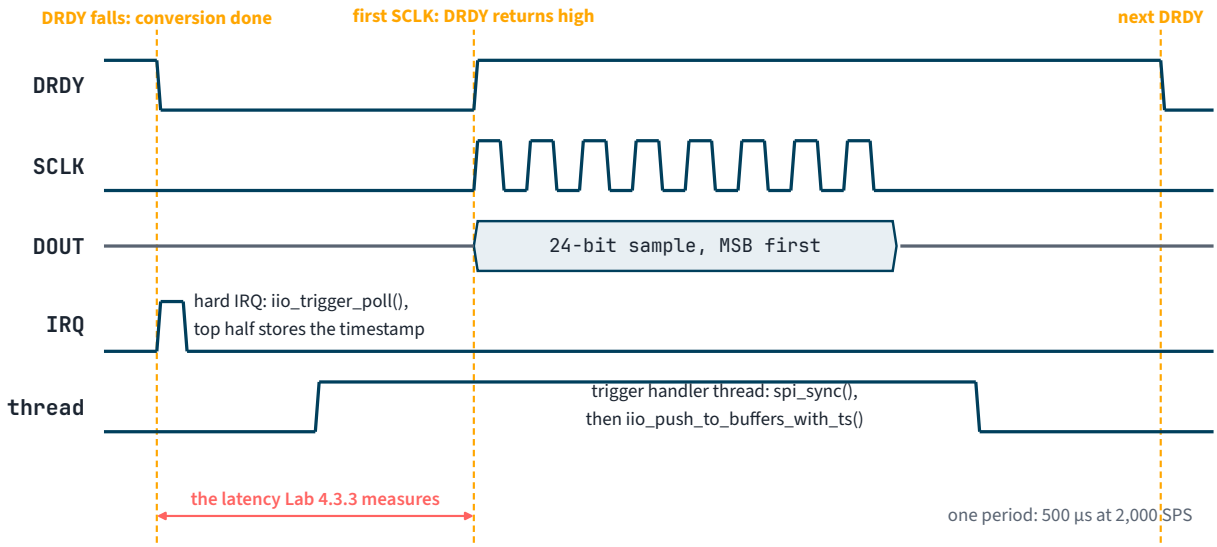


Figure 4.3.2. A data-ready trigger on the ADS1220. The hard interrupt only takes a timestamp; the delay before the first clock grows with load, and one longer than a period loses a sample.

DMA and MMIO. Devices see bus addresses, which are IOVAs behind an IOMMU. `dma_alloc_coherent` suits long-lived structures that both sides write, such as descriptor rings. `dma_map_single` and `dma_map_sg` suit per-transfer buffers and hand ownership over explicitly, because on a non-coherent SoC the map, unmap and sync calls are where caches are cleaned and invalidated. x86 keeps DMA coherent in hardware, so a missing sync works there and corrupts data on a non-coherent Arm SoC. `dma_set_mask_and_coherent` declares the address bits a device drives. Registers are `ioremap`ped and accessed with `readl` and `writel`, which are volatile accesses with barriers. On Arm, `writel` orders an earlier descriptor write before the doorbell. PCIe writes are posted, so only a read-back proves a `writel` arrived.

PCI, network, block and userspace drivers. A `pci_driver` enables the device, maps its BARs, enables bus mastering, sets its DMA mask and asks `pci_alloc_irq_vectors` for MSI-X, MSI or INTx. MSI-X gives each queue of a multi-queue device its own vector, steerable to the CPU that owns the queue. `pci_error_handlers` recover from AER events, and `pci_enable_sriov` creates virtual functions, each with BARs and MSI-X vectors for a VM to own, as on the X520's 82599. A network driver is a `net_device` with descriptor rings in coherent memory, NAPI polling under load, offloads and `ethtool` operations; `e1000e` shows the shape and `ixgbe` how much of a production driver is error handling. A block driver implements blk-mq's `queue_rq`; `drivers/nvme/host/pci.c` is the production version of your Chapter 4.1 driver. UIO serves simple devices; VFIO hands a whole PCI device to a process (regions to `mmap`, interrupts as eventfds, DMA through an IOMMU domain it controls) through the container and group interface or through iommufd and `/dev/vfio/devices/vfioN`. QEMU, DPDK and SPDK take over devices this way, and so will your VMM in Chapter 4.4 and your data paths in Chapter 4.5.

Robustness and upstreaming. Time out every wait for hardware, write a reset path and exercise it, and bounds-check every value a device returns. A PCIe device that has dropped off the bus reads `0xFFFFFFFF`, and a prototype on SPI can return garbage for one transaction. KUnit tests logic in the kernel, kselftest tests behaviour from userspace, and QEMU device models and `i2c-stub` stand in for missing hardware. Upstream work uses `checkpatch.pl`, `get_maintainer.pl`, `make W=1` and sparse, and follows an order: the binding before the driver, and a cover letter saying how you tested. Upstream drivers are the cheapest to maintain, because whoever changes a kernel API must fix every in-tree caller.

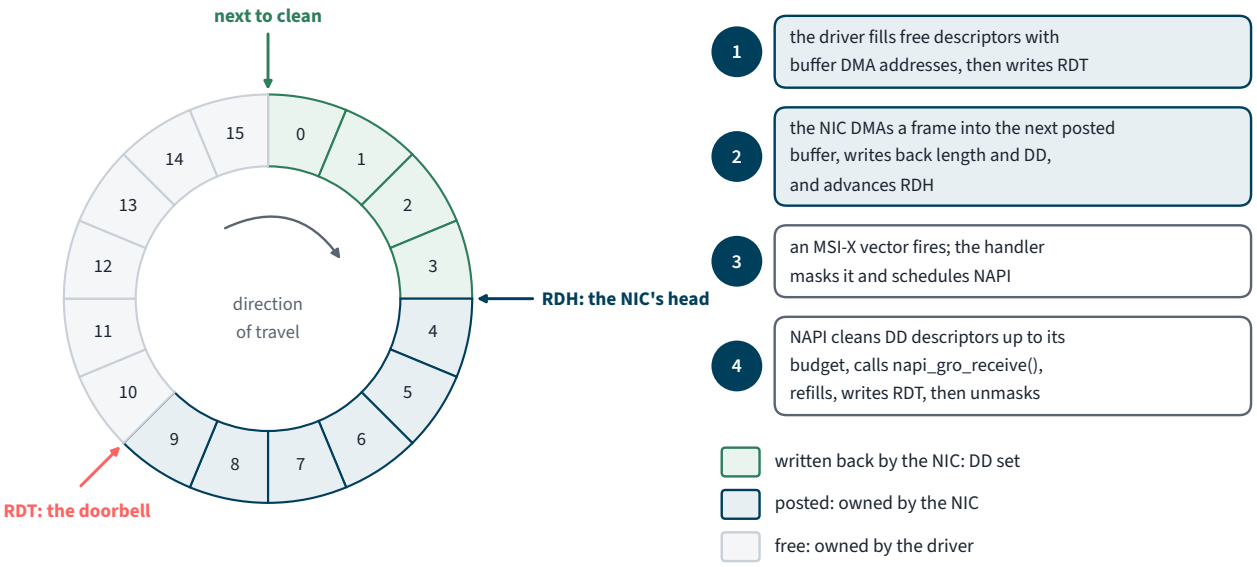


Figure 4.3.3. An RX descriptor ring in the 82599's terms: the driver posts buffers by advancing RDT, the NIC fills them and advances RDH, and NAPI cleans and refills. Lab 4.3.8 drives one.

Reading

- Corbet, Rubini and Kroah-Hartman, *Linux Device Drivers*, 3rd ed. (free): chapters 1 to 3, 5 to 10, 12, 14 and 15, for concepts. Bootlin's free driver labs on I²C, platform drivers, interrupts and DMA.
- Madieu, *Linux Device Driver Development*, 2nd ed.: the chapters on the device tree, I²C and SPI drivers, regmap, the IRQ framework, DMA and IIO.
- In the kernel's `Documentation/`: the IIO driver API with its triggered-buffer page, the hwmon kernel API and sysfs interface, the DMA API how-to, the PCI and MSI how-tos, VFIO, writing binding schemas, and submitting patches.
- In the kernel's source: `tmp117.c`, `ina238.c`, `opt4060.c`, `ti-ads131e08.c`, `emc2305.c`, `pmbus_core.c`, `regmap-irq.c`, the `e1000e` driver and the NVMe driver's `pci.c`. QEMU's `hw/misc/edu.c` with `docs/specs/edu.rst`.
- The OPT4048 (SBOSA84), ADS1220 (SBAS501) and EMC2101 datasheets; the Intel 82599 datasheet's sections on descriptors, MSI-X and SR-IOV; Emmerich et al., "User Space Network Drivers" (ANCS 2019), and its ixy code, an 82599 driver in about a thousand lines of C.

Labs

The Pi labs build in the kernel tree you configured in Chapter 4.2, whose kernel refuses unsigned modules. Build in the tree, where `make modules_install` signs them, or sign them with `scripts/sign-file`.

LAB 4.3.1 Mainline TI drivers by overlay — 6 h

Kit. Pi 5, TMP117 and INA228 breakouts, the fan and bench supply from Lab 1.3.3.

1. Add this chapter's options to your Chapter 4.2 configuration fragments, merge them, rebuild and install:

```
CONFIG_TMP117=m
CONFIG_SENSORS_INA238=m
CONFIG_GENERIC_ADC_THERMAL=m
CONFIG_IIO_HRTIMER_TRIGGER=m
CONFIG_I2C_STUB=m
CONFIG_PMBUS=m
```

2. Extend your Lab 2.7.5 overlay: give the TMP117 the `vcc-supply` its binding requires, and add the INA228 (`"ti,ina228"`) with `shunt-resistor` in micro-ohms from your four-wire measurement in Lab 1.3.3 and `ti,shunt-gain` for your range. Find both in `dmesg`.
3. Read the TMP117's `in_temp_raw` and `in_temp_scale` and the INA228 through `sensors`, and explain why `i2cget -y 1 0x48 0x0F w` is refused. Unbind both drivers, run your Python drivers from Labs 1.3.1 and 1.3.3 with the fan at three duties, rebind, and tabulate kernel against Python.
4. Annotate every function of `tmp117.c` and `ina238.c`. Find the SHUNT_CAL the driver writes and the current LSB it gives with your shunt, compare them with your Lab 1.3.3 calibration, and list what your Python drivers did that these do not, such as the TMP117's alert and the INA228's charge register.

DONE WHEN

- [ ] Both devices bind from your overlay, and kernel and Python readings agree within the parts' resolution or each difference is explained.
- [ ] The annotations give the current LSB the driver uses with your shunt and every feature each driver omits.

EVIDENCE FOR THE NOTEBOOK

- Overlay, comparison table and annotated sources.

LAB 4.3.2 An upstream-ready IIO driver for the OPT4048

2 weekends

Kit. Pi 5, OPT4048 breakout (Adafruit 6335), a desk lamp, a box that encloses both.

1. Learn from the datasheet (SBOSA84) the register map, each channel's exponent, 20-bit mantissa, counter and CRC, and the channel-1 lux equation. Its address table gives ADDR tied to VDD and to SCL the same address, so check yours with `i2cdetect`, then decode one `i2ctransfer` result by hand.
2. Write `ti,opt4048.yaml` with `compatible`, `reg`, `vdd-supply` and an optional `interrupts` until `make dt_binding_check DT_SCHEMA_FILES=ti,opt4048.yaml` is clean. Add the node to your overlay and check it with `make dtbs_check`.
3. Write `drivers/iio/light/opt4048.c` from the datasheet alone: 16-bit big-endian regmap; a device-ID check that fails `probe` clearly; continuous conversion with automatic range; raw `IIO_INTENSITY` channels with `IIO_MOD_X`, `_Y`, `_Z` and `IIO_MOD_LIGHT_CLEAR`; `integration_time`; a processed `IIO_LIGHT` channel in lux; CRC and stale-counter checks; `debugfs_reg_access`. Add a KUnit test of the decode and CRC that includes a corrupted word.
4. Only now read `opt4060.c` and `as73211.c`. The OPT4060's register map is close to the OPT4048's and a maintainer will ask why a new driver is needed, so decide between a new driver and extending `opt4060.c`, and make the series implement your decision.
5. Prepare the series (binding, driver, MAINTAINERS entry) clean under `checkpatch.pl --strict`, `make W=1` and `make C=1`, with recipients from `get_maintainer.pl` and a cover letter giving your testing and your decision. Make it with `git format-patch --cover-letter`, mail it to yourself, and apply it to a clean tree with `git am`.

DONE WHEN

- [ ] The binding passes `dt_binding_check` and your node passes `dtbs_check`.
- [ ] In the closed box, the four raw channels match hand-decoded reads within 1 %, and the lux value equals the datasheet's conversion of the channel-1 code.
- [ ] The KUnit test passes, and the series passes all three checks and applies with `git am`.

EVIDENCE FOR THE NOTEBOOK

- The series as an mbox; the box measurements; the KUnit log; the register-map comparison.

LAB 4.3.3 The ADS1220 over SPI: a data-ready trigger and a buffer — 1 weekend

Kit. Pi 5, the ADS1220 load-cell rig from Lab 1.2.4, logic analyser.

1. Write a binding for `ti,ads1220` with `spi-cpha` (only SPI mode 1 works), `interrupts` for DRDY and a `vref-supply` for the bridge excitation, and an overlay on `&spi0` chip select 0 that disables the `spidev` node there.
2. Write the driver. Regmap covers RREG and WREG with `read_flag_mask = 0x20`, `write_flag_mask = 0x40` and `reg_shift = REGMAP_UPSHIFT(2)`; other commands are plain SPI transfers. Expose your Lab 1.2.4 input with `raw`, a `scale` from the reference voltage and gain, `hardwaregain` and `sampling_frequency`, plus the internal temperature sensor; a direct read runs a single-shot conversion and waits for DRDY with a timeout.
3. Add buffered capture modelled on `ti-ads131e08.c`: continuous conversion from `preenable`, a trigger fired from the DRDY interrupt with `iio_trigger_poll`, and a handler that clocks out the three data bytes (no RDATA is needed after DRDY) and calls `iio_push_to_buffers_with_ts`. Keep the trigger to this device with `iio_trigger_validate_own_device`; under any other trigger, read with RDATA.
4. At 1,000 SPS, compare an hrtimer trigger made through configfs with DRDY: repeated and missed conversions and timestamp jitter, explained by the two clocks.
5. Stream the load cell with `iio_readdev` at 20 SPS with mains rejection and at 2,000 SPS in turbo mode, and compare the noise in grams with your Python driver's.
6. At 2,000 SPS, histogram the delay from DRDY falling to the first SCLK edge over a minute of logic-analyser capture, half of it under `stress-ng --cpu 4`.

DONE WHEN

- [ ] At 2,000 SPS the scans read in a minute equal the DRDY pulses captured, and ten minutes of stream show no timestamp gap over 1.5 periods.
- [ ] The noise at 20 SPS matches Lab 1.2.4 within 10 %, both latency histograms are plotted, the triggers are compared in numbers, and the binding passes `dt_binding_check`.

EVIDENCE FOR THE NOTEBOOK

- Stream logs, histograms and the trigger comparison.

LAB 4.3.4 Fan control: a hwmon driver, a thermal zone and PMBus — 2 weekends

Kit. Pi 5, EMC2101 breakout (Adafruit 4808), Noctua NF-A8 PWM fan, 12 V bench supply, the TMP117 and INA228 from Lab 4.3.1, logic analyser.

Mainline Linux has no EMC2101 driver, so you write it.

> **SAFETY**
> The breakout must never see the fan's 12 V. Feed the fan from the bench supply through the INA228 with a 0.5 A limit, connect only its PWM and tachometer wires to FAN and TACH (with the TACH pull-up jumper soldered), and join the grounds.

1. Write `drivers/hwmon/emc2101.c`. `probe` checks the product ID at 0xFD (0x16, or 0x28 for the EMC2101-R) and the manufacturer ID at 0xFE (0x5D). Make the temperature, tachometer and status registers volatile, and read pairs in the datasheet's interlock order: diode high byte first, tachometer low byte first.
2. Register `temp1_input`, `temp2_input`, `temp2_fault`, `fan1_input` (5,400,000 divided by the tachometer count), `fan1_min`, `pwm1` from the 6-bit fan setting, `pwm1_enable` (1 manual, 2 the chip's look-up table) and `pwm1_freq`, and the look-up table as `pwm1_auto_point[1-8]_temp` and `_pwm` through `extra_groups`, since the info API has no type for it. Set the PWM for the 25 kHz 4-wire fans expect and record the duty resolution left.

3. Register a cooling device (`#cooling-cells = <2>`) and build the thermal zone of the figure above: the TMP117 through `generic-adc-thermal` , two active trips and a cooling map to your fan. The TMP117 node then needs `#io-channel-cells = <0>` , which its binding forbids. Write the one-property binding patch that `dtbs_check` asks for.
4. Warm the TMP117 by hand, then with a hair dryer on low, and log temperature, cooling state, duty, RPM and fan power with no program of yours running. Check `fan1_input` against the tachometer on the logic analyser (two pulses per revolution), and run the look-up table alone through the forced-temperature register.
5. Exercise PMBus without a PMBus device. Load `i2c-stub chip_addr=0x40` , preload READ_VIN (0x88), READ_IOUT (0x8C) and READ_TEMPERATURE_1 (0x8D) in LINEAR11 with `i2cset` , and read `pmbus_core.c` . Write a client driver whose `pmbus_driver_info` declares them on one page and whose `read_word_data` hook fixes a planted quirk (a READ_IOUT exponent one too small), and instantiate it with `new_device` .

DONE WHEN

- ☐ `fan1_input` matches the tachometer within 2 % at three duties, `pwm1` controls the fan, and the chip's look-up table runs it alone.
- ☐ The thermal zone drives the fan from the TMP117 with no userspace program, logged with the fan's power; with your patch, `dtbs_check` is clean.
- ☐ `sensors` shows the stubbed PMBus values decoded correctly through your hook.

EVIDENCE FOR THE NOTEBOOK

- The binding patch; the thermal log; the PMBus driver and its `sensors` output.

LAB 4.3.5 Your FPGA over SPI, through regmap

2 weekends

Kit. Pi 5, the ULX3S with Lab 2.2.5's bitstream, jumper wires, logic analyser.

1. Write a binding for `omh,fpga-glue` on `&spi0` chip select 1 (the ADS1220 holds 0) with `interrupts` , `gpio-controller` , `interrupt-controller` and `gpio-line-names` , adding `omh` to your tree's `vendor-prefixes.yaml` .
2. Describe Lab 2.2.5's transaction format to regmap (widths, the read or write flag, `pad_bits` for turnaround, `reg_stride` , volatile status, input and timer registers, `REGCACHE_MAPLE` for the rest), or supply `reg_read` and `reg_write` . Fail `probe` cleanly on a wrong ID.
3. Write a `gpio_chip` by hand until `gpioset` , `gpioget` and `gpioinfo` work on FPGA pins. Add an `irq_chip` and IRQ domain on the pattern in the core ideas, with a threaded handler that reads and acknowledges the status register, until `gpiomon` sees FPGA edges. Rewrite both halves with `gpio-regmap` and `regmap-irq` , test both versions with one script, and compare line counts.
4. Expose the timer as a misc device: an `ioctl` for the compare value, defined in a versioned uAPI header, and `poll` that wakes on the compare interrupt.
5. Add a `gpio-leds` node on an FPGA pin with the `netdev` trigger on `eth0` , and find from the callers of `ledtrig_disk_activity()` why `disk-activity` stays dark for NVMe.
6. Toggle an FPGA output as fast as you can while `gpiomon` watches an input wired to it; find the rate at which events are lost, and where the time goes, with `trace-cmd record -e irq -e spi` .

DONE WHEN

- ☐ `gpioset` drives an FPGA LED, `gpiomon` sees FPGA edges through your IRQ domain, and the compare interrupt wakes a process blocked in `poll` .
- ☐ Both driver versions pass the same tests, with line counts recorded.
- ☐ The LED follows Ethernet traffic, and the stress ceiling is measured with its bottleneck named.

EVIDENCE FOR THE NOTEBOOK
- Both driver versions, the uAPI header and the stress trace.

LAB 4.3.6 PCIe with MSI and DMA in QEMU — 1 weekend

Kit. QEMU with `-device edu`.

1. Read QEMU's `docs/specs/edu.rst` and write the register map in your notebook. Then write a `pci_driver` for 1234:11e8 that enables the device, maps BAR0, checks its identification register, sets the DMA mask and requests one MSI vector. The device clamps DMA addresses to 28 bits (its `dma_mask` property), so declare exactly that.
2. Compute factorials on the device behind a character-device `ioctl`, waiting for its interrupt.
3. DMA a coherent buffer into the device's 4 KiB buffer at 0x40000 and back, and verify it. Repeat from a user buffer with `pin_user_pages` and `dma_map_sg`.
4. Add timeouts and a reset path, patch `hw/misc/edu.c` to skip an interrupt now and then, and show the driver recovers.
5. Write a KUnit test for the pure logic and a kselftest script for the character device.

DONE WHEN
- ☐ DMA round trips verify from both kinds of buffer, injected faults never hang the driver, and both test suites pass.

EVIDENCE FOR THE NOTEBOOK
- The QEMU patch; the log of injected faults and recoveries.

LAB 4.3.7 NVMe, in the tree — 6 h

Kit. The server with the P4510, its root file system elsewhere so the NVMe modules can be reloaded.

1. Read `drivers/nvme/host/core.c` and `pci.c` beside your Chapter 4.1 driver, map your steps onto the kernel's, and list what it adds: namespaces, multipath, APST, reset work, polled queues, SGLs, the controller memory buffer.
2. Add a debugfs file showing each queue's depth and completion count, rebuild and reload only the NVMe modules, and watch it while `fio` runs random reads.
3. Run `trace-cmd record -e nvme` during `fio`, pair submissions with completions by queue and command ID, and compute the latency distribution.

DONE WHEN
- ☐ Your debugfs file shows live queue state, and your latency distribution agrees with a `bpftrace` histogram of the same workload, made as in Chapter 4.2.

EVIDENCE FOR THE NOTEBOOK
- The patch; the trace script and its plot.

LAB 4.3.8 The 82599 in the kernel, then in userspace — 3 weekends

Kit. The server booted with `intel_iommu=on` and reachable through its on-board ports; the X520's ports joined by the DAC.

1. Read `e1000e`'s probe, ring setup, transmit and NAPI poll, and draw the ring lifecycle.
2. Leave port 1 on `ixgbe` in a network namespace so that traffic crosses the cable, unbind port 0, and write a minimal driver for it from the 82599 datasheet (QEMU's `e1000` model is a gentler start): reset, link, one RX and one TX ring in coherent memory, MSI-X, `ndo_open`, `ndo_stop`, `ndo_start_xmit` and a NAPI poll calling `napi_gro_receive`, with no offloads.

3. `ping` across the cable, watch `/proc/interrupts`, and add a few `ethtool -S` counters. Compare single-stream `iperf3` with `ixgbe` and explain the gap through offloads, ring sizes and interrupt moderation.
4. Bind port 0 to `vfio-pci` and check its IOMMU group; if port 1 shares it, bind both and transmit from your own program. In C, open the container and group, set the IOMMU type, get the device, `mmap` BAR0 and read the registers you read in Lab 2.4.4.
5. Map a buffer with `VFIO_IOMMU_MAP_DMA`, bring up one RX queue whose descriptors and buffers live in it (ixy does this in a few hundred lines), and receive a frame. Route an MSI-X vector to an eventfd with `VFIO_DEVICE_SET_IRQS` and block on it.
6. Port the program to iommufd (`/dev/vfio/devices/vfioN`, `VFIO_DEVICE_BIND_IOMMUFD`, `IOMMU_IOAS_ALLOC`, `VFIO_DEVICE_ATTACH_IOMMUFD_PT`, `IOMMU_IOAS_MAP`). Write a page on what the kernel did for you (IOMMU domain, interrupt remapping, reset on release) and what would happen without an IOMMU.

DONE WHEN

- [ ] `ping` crosses the cable through your driver, and the throughput gap is explained in writing.
- [ ] A frame reaches your buffer by DMA through the IOMMU, signalled by an eventfd, with both VFIO interfaces, and the page is written.

EVIDENCE FOR THE NOTEBOOK

- The ring drawing; the throughput table; both VFIO programs with a hex dump of the frame; the page.

Problem set

1. Give a case where `dma_alloc_coherent` is wrong (large per-request buffers on a non-coherent SoC) and one where `dma_map_single` is wrong (a ring the device writes asynchronously), and what fails in each.
2. A driver polls an `ioremap`ped status register through a plain pointer. What does GCC do with the loop? With `volatile` but no barrier on Arm, what can still go wrong with a DMA buffer read after the status bit changes?
3. An OPT4048 channel returns EXPONENT = 6, RESULT_MSB = 0x123 and RESULT_LSB = 0x45. Compute the mantissa, the linear code and, for channel 1, the illuminance. How many bits must the raw value hold at the largest exponent, and how do you express the lux conversion as an IIO scale without floating point?
4. The ADS1220 runs at 2,000 SPS with a 4 MHz SPI clock. What fraction of a period does the 24-bit read take? A scan is a 32-bit sample and a 64-bit timestamp: how much buffer does a second need? At what DRDY-to-SCLK delay do samples start to be lost, and what does a lost sample look like in the buffer?
5. The EMC2101's PWM frequency is 360 kHz divided by $2 \times \text{PWM_F} \times \text{PWM_D}$, with a duty resolution of $1/(2 \times \text{PWM_F})$. Choose register values for a 25 kHz fan, give the resolution left, and compute the speed for a tachometer count of 2,700. Then write the device-tree node for an EMC2101 at 0x4C with its ALERT/TACH pin as an interrupt on GPIO 17, its YAML binding, and the same device in ACPI ASL.
6. Order `probe`'s resource acquisitions for a PCIe device with MSI-X, DMA and a character device so that managed resources alone make every failure path correct. Which resource still needs explicit cleanup, and why?
7. Explain, from what each vector's handler does and which CPU runs it, why a multi-queue NIC needs MSI-X instead of MSI. Trace one RX interrupt through NAPI to re-enabling the interrupt, and say what changes under sustained load.
8. What can a malicious guest that owns an 82599 VF do to the PF's traffic and to other VFs, and what do the hardware and the `ixgbe` PF driver prevent?
9. Design an `ioctl` ABI for your FPGA device that can grow (versioning, size fields, reserved space) and add a field without breaking a compiled program; compare DRM and io_uring. If the prototype returns

0xFFFFFFFF from every register for one transaction, what in your driver should notice, and what should it do?

Deliverables and stretch

Deliverables. Every driver and binding with its README, test script and recorded output; the OPT4048 series as an mbox ready to send, and the TMP117 binding patch; both VFIO programs and their page; the latency histograms, throughput comparison and NVMe trace analysis.

Stretch. Write a QEMU model of your FPGA peripheral that talks over a socket to the Verilator simulation of its RTL, so driver and hardware develop together. Add the TMP117's ALERT pin to the mainline driver as IIO threshold events. Add SR-IOV to your NIC driver. Write an `nvme-lite` blk-mq driver for QEMU's NVMe device. Send the OPT4048 series, the binding patch or the EMC2101 driver upstream.

COMPLETION CHECKLIST

- ☐ **Lab 4.3.1:** TMP117 and INA228 bound by overlay and matched to your Python drivers; both sources annotated.
- ☐ **Lab 4.3.2:** OPT4048 verified in the box; KUnit passing; the series clean and applying.
- ☐ **Lab 4.3.3:** 2,000 SPS with no lost samples; noise within 10 % of Lab 1.2.4; latency histograms.
- ☐ **Lab 4.3.4:** fan speed within 2 % of the tachometer; the fan curve run by the kernel; PMBus through your hook.
- ☐ **Lab 4.3.5:** FPGA gpiochip, IRQ domain and pollable timer, in both versions; the stress ceiling explained.
- ☐ **Lab 4.3.6:** edu DMA from both kinds of buffer; injected faults survived.
- ☐ **Lab 4.3.7:** live queue state in debugfs; trace latencies matching `bpftrace`.
- ☐ **Lab 4.3.8:** `ping` through your 82599 driver; a frame received through the IOMMU with both VFIO interfaces.
- ☐ **Problem set:** all nine answered; problems 3 to 5 checked numerically.
- ☐ **You can explain**, without notes: how a device-tree node becomes a probed driver, when DMA should be coherent and when streaming, why an interrupt controller behind SPI writes masks in `irq_bus_sync_unlock`, and what VFIO and the IOMMU each contribute.

CHAPTER 4.4

Virtualisation and hypervisors

TIME	5 weeks
YOU BUILD	A KVM VMM that boots Linux with your own 16550, virtio-blk and virtio-net, two vCPUs, a passed-through 82599 virtual function and live migration; your Chapter 4.1 kernel as its guest; a bare-metal VT-x hypervisor running Linux; Compute's VMM API; a hardened multi-tenant host; an Arm port.
NEEDS	The lab server with VT-x, VT-d and SR-IOV enabled in its firmware, the X520-DA2 with its DAC and the P4510; QEMU, Firecracker and Cloud Hypervisor; the Raspberry Pi 5; your Chapter 4.1 kernel, your Lab 2.7.2 loader and your VFIO program from Lab 4.3.8.

WHAT THIS BUYS OMH

Compute is OMH's hypervisor product. Every tenant instance OMH sells runs on a virtual machine monitor that must isolate tenants from each other and from the host, give them fast storage and networking, and move them between modules during rolling updates. It must never let a guest reach the keys Modulus holds. A founder who cannot read a VM-exit reason, or say why a guest's disk write is slow, cannot specify the product or price it. This chapter builds the stack twice: on KVM, the way you will ship it, and on bare metal with VT-x, so that you understand everything KVM does.

SKIP TEST

1. A guest executes `cpuid`. List, in order, every hardware and software step from that instruction to the guest receiving a result, on Intel with KVM.
2. Name the two levels of translation under EPT and who owns each. What does a guest page fault that the guest kernel handles look like to the hypervisor?
3. Why does a virtio driver need a memory barrier between writing a descriptor and writing the available index, and what does the packed ring layout change?
4. A passed-through NIC DMAs to guest physical address 0x1000. Name every table consulted before a byte moves, and what keeps the DMA out of another tenant's memory.
5. In live migration, what is dirty-page tracking, what happens in the final stop-and-copy phase, and why does a passed-through device make migration hard?

If all five are easy, do Labs 4.4.2, 4.4.4 and 4.4.6 only.

Core ideas

Trap and emulate, and why it needed hardware. Popek and Goldberg showed in 1974 that a machine is virtualisable if every sensitive instruction traps when run with reduced privilege. Classic x86 was not, because `popf` and `sgdt`, among others, behaved differently in ring 3 without trapping. VMware translated binaries around the gap and Xen modified the guest (paravirtualisation). Intel's VT-x (2005) and AMD's SVM (2006) then added a mode above ring 0 and a hardware structure, the VMCS on Intel and the VMCB on AMD, that says which guest actions cause a VM exit. With them, a hypervisor is a loop: enter the guest with `vmlaunch` or `vmresume`, run until an exit, read the exit reason, emulate or fix up, and enter again.

VMX in detail. `vmxon` enters VMX root operation. The VMCS holds guest state (loaded on entry), host state (loaded on exit), execution controls (which events exit: CPUID, HLT, port I/O, MSR access, control-register writes, external interrupts, EPT violations), exit information (reason, qualification, guest linear and physi-

cal address) and entry controls such as event injection. Software touches it only through `vmread` and `vmwrite` by field encoding. CPUID, INVD and `vmcall` always exit; everything else is a control bit, constrained by the allowed-0 and allowed-1 settings in the `IA32_VMX_*` capability MSRs. A wrong field produces "VM-entry failure due to invalid guest state" with no further detail, so you debug it by working through the SDM's list of checks on guest state.

Memory: EPT. Without hardware help a hypervisor shadows the guest's page tables and traps every change to them. Extended Page Tables (Intel) and nested page tables (AMD) add a second translation: guest virtual to guest physical by the guest's tables, then guest physical to host physical by the hypervisor's. A TLB miss becomes a two-dimensional walk of up to 24 memory reads, which is why huge pages matter more under virtualisation. An EPT violation is the hypervisor's page fault, and lazy allocation, MMIO emulation, dirty tracking and ballooning all live in its handler. VPIDs tag TLB entries per guest so that switching need not flush them, and `invept` drops cached EPT translations.

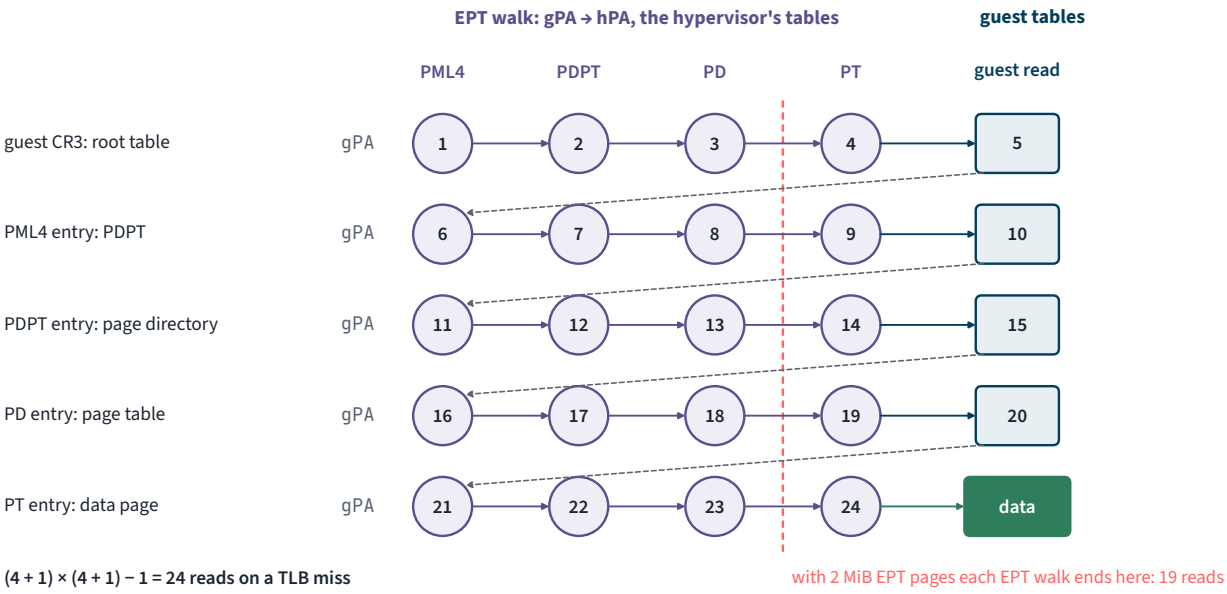


Figure 4.4.1. The two-dimensional walk. Every guest page-table pointer is a guest physical address the EPT must translate first, so four guest levels over four EPT levels cost 24 reads.

Interrupts and timers. A guest expects a local APIC, an I/O APIC, a PIT, and an HPET or TSC-deadline timer. Emulating the APIC in software costs an exit per end-of-interrupt write; APICv and posted interrupts deliver interrupts into a running guest in hardware. Injection on entry, interrupt windows while the guest has interrupts masked, and the bookkeeping of pending interrupts are where hypervisor bugs cluster. Guest timers are host timers that inject on expiry, and the TSC is offset per guest and may be scaled.

Devices: emulated, paravirtual, passed through. Emulating a real device such as an e1000 is slow because its driver touches many registers per operation and each access exits. virtio devices are designed to be emulated: descriptor tables in guest memory, an available ring the driver writes, a used ring the device writes, a notification each way and feature negotiation, over PCI or MMIO. vhost moves the data path into a kernel thread, vhost-user into another process (DPDK, SPDK), and vDPA into hardware behind a vendor control path. Passthrough hands a real device to the guest through VFIO: the IOMMU translates its DMA with the same guest-physical-to-host map as the EPT, and interrupt remapping stops it signalling an arbitrary vector. SR-IOV makes passthrough scale. Each port of the X520's 82599 can present virtual functions, each its own PCI function with MSI-X vectors and usually its own IOMMU group, while the PF driver keeps the MAC and VLAN filters and anti-spoofing.

KVM. KVM makes Linux the hypervisor. `KVM_CREATE_VM` on `/dev/kvm` returns a VM descriptor, and `KVM_CREATE_VCPU` returns one per vCPU. `KVM_SET_USER_MEMORY_REGION` maps host memory into guest physical space. `KVM_RUN` runs a vCPU until an exit KVM cannot handle itself, which it reports in the shared `kvm_run` structure as port I/O, MMIO, halt, shutdown or an internal error. The in-kernel interrupt controller, `KVM_IRQFD` and `KVM_IOEVENTFD` let devices signal and be signalled without a round trip to userspace. The

VMM (QEMU, Firecracker, Cloud Hypervisor, crosvm, kvmtool) owns the descriptors, emulates devices and handles what KVM hands back. On Arm, KVM runs at EL2 with stage-2 translation and nearly the same ioctls, which is why Firecracker runs on both.

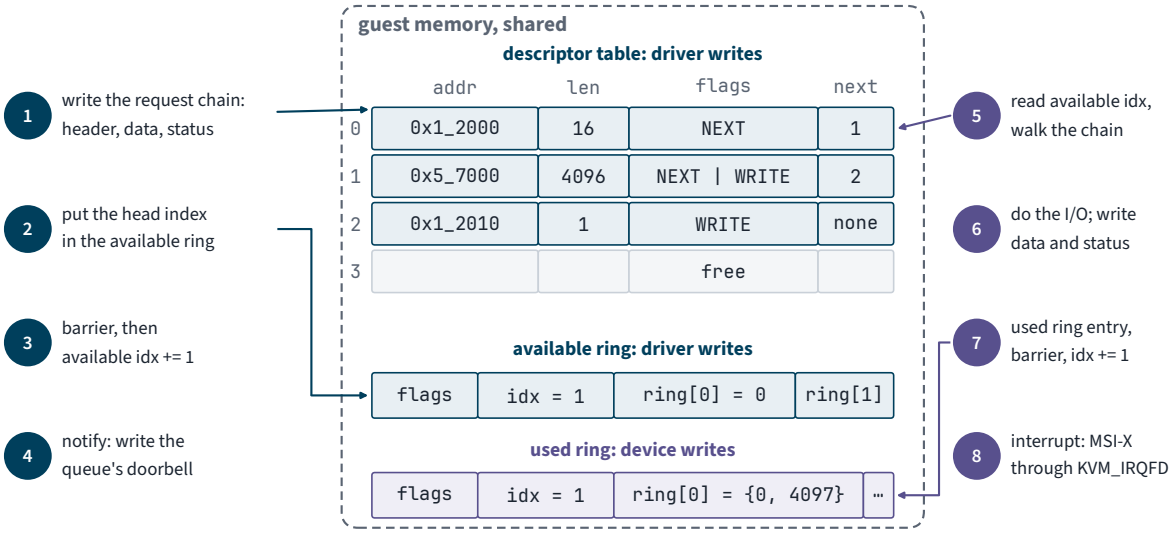


Figure 4.4.2. A virtio-blk read on a split virtqueue. The barriers stop either side seeing an index before the data it covers.

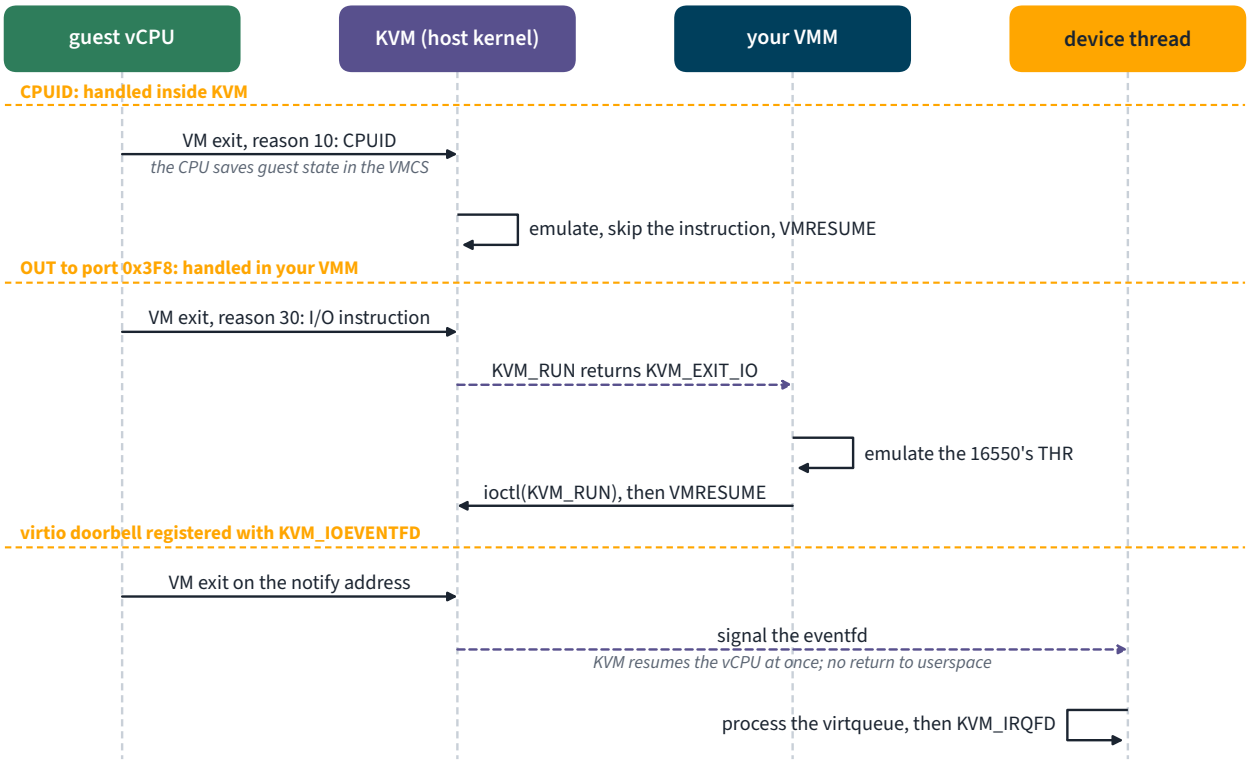


Figure 4.4.3. Where an exit is handled decides its cost: CPUID never leaves the kernel, port I/O round-trips through your VMM, and an ioeventfd doorbell wakes a device thread while the vCPU carries on.

Boot inside a VM. A VMM decides how much firmware the guest gets. With none, it loads a bzImage through the Linux boot protocol, fills `boot_params` with an E820 map and a command line, and starts the vCPU at the 32-bit entry point. The alternatives are a minimal firmware such as qboot, or full UEFI with OVMF. Direct boot lets a microVM boot in about 125 ms, which a functions service needs. Get the 16550 serial port at 0x3F8 working first, because until it works you cannot see what the guest is doing.

Isolation and security. The hypervisor is the trust boundary between tenants. Its attack surface is device emulation, where most historical VM escapes were found, the instruction emulator, shared memory, and side channels such as Spectre, L1TF and MDS that leak between threads on one core. The defences are

small device models, seccomp and namespaces around each VMM process, core scheduling so that sibling hyperthreads never run different tenants, and buffer and cache flushes on the transitions where the CPU needs them. Memory encryption (AMD SEV-SNP, Intel TDX, Arm CCA) removes the hypervisor from the guest's trust base; the lab server's Xeon supports none of these, so study what each protects on paper.

Live migration and snapshots. Pre-copy marks all guest memory dirty and copies it while the guest runs, then repeatedly copies the pages dirtied since the last pass, tracked through write-protected EPT entries, the CPU's page-modification log or KVM's dirty ring. When the remainder is small it pauses the guest, sends the rest with the vCPU and device state, and resumes on the destination. A guest that dirties memory faster than the link drains it never converges, so the hypervisor throttles it or switches to post-copy, which runs the guest on the destination and fetches pages on demand. A snapshot is a migration to a file, which is how Firecracker restores a microVM faster than it boots one. A passed-through device breaks migration unless it can save its state through VFIO's migration interface, so Compute must choose between passthrough speed and migratable instances, or sell both.

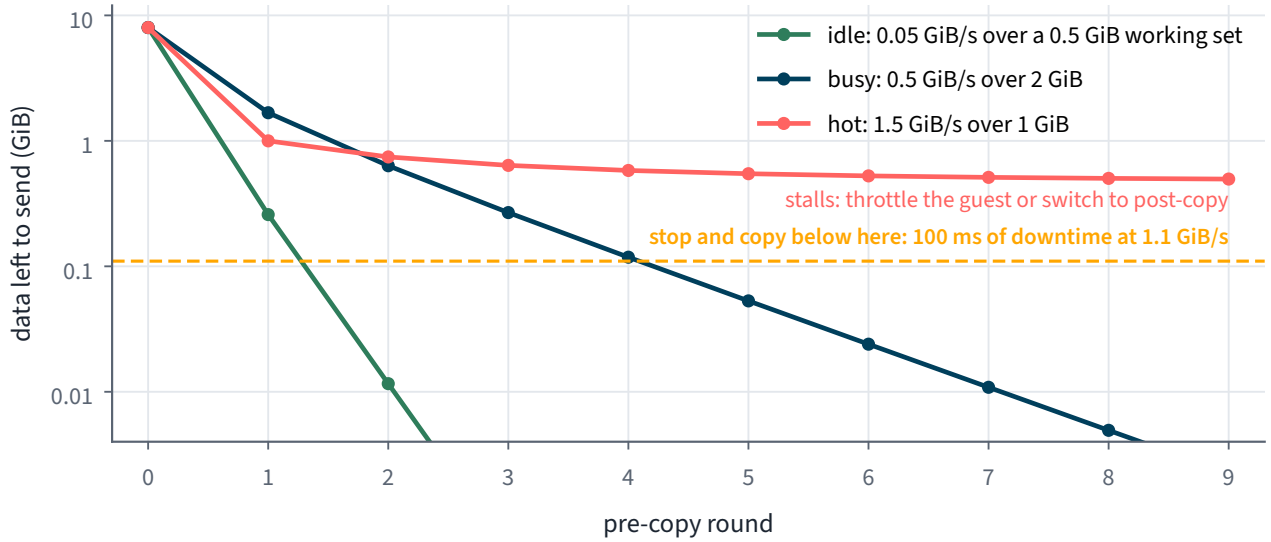


Figure 4.4.4. Pre-copy for an 8 GiB guest on a 10 Gb/s link, in a model where each round's dirty pages saturate a working set. A hot guest stalls above the downtime budget until it is throttled.

Nested virtualisation and the management plane. KVM can run KVM: the L1 hypervisor's VMX instructions trap to L0, which emulates them with shadow VMCSs and EPT on EPT. It is slow, but it lets you develop a hypervisor inside a virtual machine. Above the VMM sits the management plane, libvirt in general or a purpose-built agent with an API like Firecracker's. Compute's control plane will talk to a VMM API on each module, and this chapter decides its shape: create, configure devices, boot, pause, snapshot, migrate, destroy and report metrics.

Arm virtualisation. Arm provides EL2 with stage-2 translation, the GIC's virtual CPU interface, the generic timer's virtual counter, and VHE, which lets the host kernel run at EL2. The Pi 5 adds two practical details. Its default kernel uses 16 KiB pages, so guest memory regions must be 16 KiB aligned. Its GIC-400 is a GICv2 whose 8 KiB virtual CPU interface is not aligned to a 16 KiB page, so KVM traps guest accesses to it instead of mapping them. The exit vocabulary differs (HVC calls, data aborts for MMIO, WFI) while the VMM code barely changes.

Reading

- Bugnion, Nieh and Tsafrir, *Hardware and Software Support for Virtualization* (Morgan & Claypool, 2017): all of it; it is short and the best single text on the subject.
- Intel SDM volume 3: the VMX chapters from "Introduction to Virtual Machine Extensions" to "VMX Instruction Reference" (VMCS, VM entries and exits, EPT, APIC virtualisation), read closely before Lab 4.4.4, with the appendix of basic exit reasons as a desk reference. The VT-d specification's chapters on DMA remapping and interrupt remapping.

- The virtio 1.3 specification: chapter 2 (basic facilities), 4.1 (PCI transport), 5.1 (network device) and 5.2 (block device). Draw the split ring on paper before Lab 4.4.2.
- Linux `Documentation/virt/kvm/api.rst` for the ioctls in Lab 4.4.1; `Documentation/arch/x86/boot.rst` and `zero-page.rst` for direct boot; `Documentation/arch/arm64/booting.rst` for Lab 4.4.7.
- kvmtool, small enough to read entirely; Firecracker's `src/vmm`; Cloud Hypervisor's `vmm/src/cpu.rs`; QEMU's `accel/kvm/kvm-all.c` and `hw/virtio/virtio.c`.
- Agache et al., "Firecracker: Lightweight Virtualization for Serverless Applications" (NSDI 2020); Clark et al., "Live Migration of Virtual Machines" (NSDI 2005); Popek and Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures" (CACM, 1974).

Labs

Write the VMM in C or C++ with nothing beyond libc and the kernel's headers. It is the one you extend for the rest of the course: Chapter 4.5 adds a vhost-user disk to it.

LAB 4.4.1 A first guest through /dev/kvm

1 weekend

Kit. The lab server.

1. Open `/dev/kvm`, check that `KVM_GET_API_VERSION` returns 12, create a VM, `mmap` 64 MiB and register it at guest physical address 0 with `KVM_SET_USER_MEMORY_REGION`. Create a vCPU and `mmap` its `kvm_run`, sized by `KVM_GET_VCPU_MMAP_SIZE`. Note which of `ept`, `vpid`, `enable_apicv`, `pml` and `nested` are set in `/sys/module/kvm_intel/parameters/`.
2. Run a 16-bit real-mode guest of a few bytes that `out`s a string to port 0x3F8 and halts. Set registers with `KVM_SET_REGS` and `KVM_SET_SREGS`, loop on `KVM_RUN`, print on `KVM_EXIT_IO` and stop on `KVM_EXIT_HLT`.
3. Run a 64-bit guest: build page tables and a GDT in guest memory, set CR0, CR3, CR4, EFER and the segment registers for long mode, and run a flat binary compiled from C that writes to the serial port and reads an MMIO address you do not back with memory. Handle `KVM_EXIT_MMIO`.
4. Create the in-kernel irqchip (`KVM_CREATE_IRQCHIP`) and PIT (`KVM_CREATE_PIT2`), and inject an interrupt from a host timer thread with `KVM_IRQ_LINE`. The guest installs an IDT and counts ticks.
5. Run `perf kvm stat` while the guest runs, report exits per second by reason, and measure the cost of one round trip through `KVM_RUN` in nanoseconds.

DONE WHEN

- [ ] A 64-bit guest you built from nothing runs, takes MMIO exits and receives injected interrupts.
- [ ] The notebook has an exit-cost table by reason.

EVIDENCE FOR THE NOTEBOOK

- The program; the exit statistics and round-trip timing.

LAB 4.4.2 Boot Linux with your own virtio-blk and virtio-net

2 weekends

1. Direct boot: load a bzImage as `boot.rst` describes, fill `boot_params` (the E820 map from `zero-page.rst`, command line, initrd, `loadflags`) and start the vCPU in protected mode at the 32-bit entry. Emulate a 16550 at 0x3F8 well enough for `earlyprintk` and then the 8250 driver: THR, RBR, IER, IIR and LSR, with an interrupt on receive. Boot a minimal kernel and a Buildroot initramfs to a shell.
2. virtio-blk over PCI: emulate configuration space through ports 0xCF8 and 0xCFC with one device carrying the virtio vendor ID and the block device ID, its common, notify, ISR and device capabilities in a BAR served from MMIO exits, feature negotiation, the split virtqueue with the barriers the specification requires, and MSI-X or INTx on completion. Back it with a file and boot with `root=/dev/vda`.

3. virtio-net on a `tap` device, with a TX and an RX queue and the virtio-net header. Give the guest an address, `ping` the host and run `iperf3`.
4. Move notifications off the vCPU thread with `KVM_IOEVENTFD` for doorbells, `KVM_IRQFD` for completions and a thread per virtqueue, and measure the change in exits and throughput.
5. Add a second vCPU: describe it in an MP table or ACPI MADT, run it on its own thread, and let the in-kernel irqchip carry IPIs. Confirm that `nproc` prints 2 and that a parallel build keeps both busy.

DONE WHEN

- [ ] Linux boots from your virtio-blk to a shell on your serial port, runs `ping` and `iperf3` over your virtio-net, and keeps both vCPUs busy.
- [ ] The time from the first `KVM_RUN` to the shell prompt is recorded.

EVIDENCE FOR THE NOTEBOOK

- The boot log with timing; the measurements before and after ioeventfd and irqfd.

LAB 4.4.3 Your kernel as a guest, then a passed-through virtual function — 2 weekends

Kit. The server with SR-IOV enabled, the X520's ports joined by the DAC.

1. Boot your Chapter 4.1 kernel in your VMM: load its ELF and build the boot-information structure as your Lab 2.7.2 loader does. Fix what breaks, recording whether each fault was the kernel's or the VMM's. Its NVMe driver will find no controller, so give it a virtio-blk driver or emulate enough of an NVMe controller (admin queue, one I/O queue) for it to run. The second option teaches you what QEMU's `hw/nvme` does and is worth the week.
2. Create four virtual functions on port 0, still on `ixgbe`, by writing 4 to its `device/sriov_numvfs` in sysfs. Set VF 0's MAC address and VLAN from the host with `ip link set <port> vf 0`, and bind VF 0 to `vfio-pci`.
3. Pass it through with your Lab 4.3.8 VFIO code: present its configuration space on your PCI bus with the values VFIO reports; map the regions VFIO marks mmap-capable into guest physical space, and forward the rest, including the MSI-X table, from MMIO exits with `pread` and `pwrite`; map all guest RAM for DMA with guest physical addresses as IOVAs; and route each MSI-X vector's eventfd into `KVM_IRQFD` through GSI routing.
4. Boot Linux in your VMM, let `ixgbevf` bind in the guest, and run `iperf3 -P 4` through the VF to port 1 in a host network namespace, then through your virtio-net.
5. Kill the VMM under load and confirm the VF recovers, since VFIO resets it on release. Then leave part of guest RAM out of the IOMMU mapping and show that the host logs a DMA fault where memory would otherwise have been corrupted.

DONE WHEN

- [ ] Your Chapter 4.1 kernel runs as a guest of your VMM.
- [ ] A Linux guest drives the VF through the IOMMU at over 9 Gb/s, and the deliberate DMA fault is caught.

EVIDENCE FOR THE NOTEBOOK

- The passthrough code; the throughput table against virtio-net; the fault log.

LAB 4.4.4 A bare-metal VT-x hypervisor — 3 weekends

Your Chapter 4.1 kernel becomes a hypervisor. Develop it under QEMU with `-enable-kvm -cpu host`, which offers nested VMX, then move to the server.

1. Check CPUID and `IA32_FEATURE_CONTROL`, set CR4.VMXE, allocate the VMXON region and a VMCS with the revision ID, and run `vmxon`, `vmclear` and `vmptrld`. Derive every control field from the `IA32_VMX_*` capability MSRs, which say which bits must be 0 and which must be 1.
2. Fill the VMCS for a guest running a flat 64-bit binary in memory you allocate: guest state; host state (your kernel's segments, CR3, a host stack, the RIP of your exit handler); execution controls with CPUID, HLT, port I/O and MSR exits; and EPT with four-level tables you build. When `vmlaunch` fails, `vmread` the VM-instruction error field, or for a failure on guest state the exit reason, 33, and find the check you broke in the SDM.
3. In the exit handler, save the guest's registers and switch on the exit reason: run CPUID on the host and hide the VMX bit; send port I/O to 0x3F8 to your console; idle on HLT; on an EPT violation allocate a frame or emulate an MMIO device you define; on anything else dump the VMCS and stop.
4. Run Linux as the guest with Lab 4.4.2's direct boot, a 16550 and virtio-blk over the MMIO transport, which is simpler than PCI for a first pass. Emulate the local APIC timer or use the VMX preemption timer so the guest schedules, and reach a shell.
5. Time-slice two guests, each with its own EPT and serial console, and show that a guest overwriting all of its physical memory does not disturb the other.
6. Count exits per reason, print them on your console, and compare them with KVM's from Lab 4.4.1.

DONE WHEN

- [ ] Linux boots to a shell on your hypervisor on the server.
- [ ] Two guests run concurrently and cannot see each other's memory, and the exit statistics are written up beside KVM's.

EVIDENCE FOR THE NOTEBOOK

- The hypervisor; the VMCS setup documented field by field; the exit statistics.

LAB 4.4.5 Firecracker, Cloud Hypervisor and a VMM API for Compute

1 weekend

1. Run Firecracker with a kernel and root file system and drive its API with `curl` over its Unix socket: `PUT /boot-source`, `/drives/rootfs`, `/network-interfaces/eth0` and `/machine-config`, then `PUT /actions` with `InstanceStart`. Time boot to shell. Pause it with `PATCH /vm`, create a snapshot, restore it in a new process with `PUT /snapshot/load`, and time the restore.
2. Read the device manager and virtio block device in Firecracker's `src/vmm` beside your own, and list what it does that you did not, and why: rate limiters, seccomp filters, the jailer, MMDS and the balloon.
3. Run the same guest under Cloud Hypervisor and compare its device model (PCI, hot-plug, virtio-fs, vDPA) and boot time.
4. Write Compute's VMM API as an OpenAPI document: instance lifecycle, device attach and detach, snapshot, migration, metrics and how the control plane authenticates. Write one page choosing the base for Compute's first release (Firecracker, Cloud Hypervisor, QEMU or your own VMM), and implement a subset of your API on your Lab 4.4.2 VMM as an HTTP service on a Unix socket.

DONE WHEN

- [ ] You can start, snapshot and restore a microVM through your own API.
- [ ] The OpenAPI document and the architecture decision are written.

EVIDENCE FOR THE NOTEBOOK

- Boot and restore timings for both VMMs; the API document; the decision page.

LAB 4.4.6 Live migration and the multi-tenant node — 2 weekends

1. Track dirty pages with `KVM_MEM_LOG_DIRTY_PAGES` and `KVM_GET_DIRTY_LOG`, or KVM's dirty ring, and implement pre-copy migration to a second VMM process over TCP. Iterate until the dirty set falls below a threshold, pause, send the vCPU state (`KVM_GET_REGS`, `SREGS`, `FPU`, `MSRS`, `LAPIC` and whatever else your guest needs) and your virtqueue indices (the disk file is shared), and resume on the destination. Migrate a guest running `iperf3` and a guest writing a counter to disk, measure the downtime and check that nothing was lost.
2. Harden the host: run each VMM in its own cgroup with `memory.max` and `cpu.max`, under a seccomp filter allowing only the system calls you traced it making, as an unprivileged user, in its own network namespace. Enable core scheduling, and pin vCPU threads to the isolated cores you configured in Chapter 4.2.
3. Measure interference: one tenant runs a memory-bandwidth hog while another runs a latency-sensitive loop. Report the latency distribution with and without pinning, and with the two tenants on sibling hyperthreads.
4. Read `/sys/devices/system/cpu/vulnerabilities/` on the server, explain each entry and what its mitigation costs, and set Compute's SMT policy for each instance tier.
5. Pass the host's attestation result from Lab 2.7.4 into a guest over virtio-vsock or an MMDS-style metadata service, so that a tenant can check it runs on an attested OMH module. Sketch how a per-guest vTPM (`swtpm` behind an emulated TIS or CRB device) would extend the chain.

DONE WHEN

- [ ] A running guest migrates with under 100 ms of downtime and its disk intact.
- [ ] The multi-tenant policy is written and the interference is measured.

EVIDENCE FOR THE NOTEBOOK

- Migration logs with downtime; the seccomp filter and cgroup settings; the interference plots; the policy page.

LAB 4.4.7 Arm, for comparison — 1 weekend

Kit. Raspberry Pi 5.

1. Port your Lab 4.4.2 VMM: `KVM_ARM_PREFERRED_TARGET` and `KVM_ARM_VCPU_INIT`, a GICv2 created with `KVM_CREATE_DEVICE` (the BCM2712's GIC-400 is a GICv2), the generic timer, a PL011 UART emulated on MMIO exits, virtio over MMIO, and an arm64 `Image` booted as `booting.rst` describes, with a device tree you generate with libfdt. Align guest memory to the host's page size.
2. On a 16 KiB-page kernel, read KVM's lines in `dmesg`, explain its warning about the GICv2 virtual CPU interface from the GIC-400's register layout, and count the exits that result.
3. Boot Linux and compare exit reasons and rates with your x86 VMM on the same workload.
4. Write one page on what a Compute module built on Arm server silicon (Ampere, or a Grace-class part) would change in your VMM and in the product.

DONE WHEN

- [ ] Linux boots in your VMM on the Pi, and the exit comparison and the page are written.

EVIDENCE FOR THE NOTEBOOK

- The port; the exit comparison table.

Problem set

1. Give three x86 instructions that Popek and Goldberg would call sensitive but that did not trap in ring 3 before VT-x, and say what each leaks or breaks.

2. A guest kernel handles a page fault with no VM exit. Explain how that is possible under EPT, and name a guest page fault that does cause an exit. Count the reads in a worst-case two-dimensional walk with four-level guest tables and four-level EPT; how do 2 MiB EPT pages change it, and how do VPIDs change how often you pay it?
3. Your virtio-blk device finds a descriptor pointing outside guest memory. What must it do, and what happens if it dereferences the address anyway?
4. Explain why an MSI from a passed-through device must be remapped by the IOMMU, and describe the attack a malicious guest could mount without remapping.
5. Firecracker boots in about 125 ms and QEMU with OVMF in a few seconds. Attribute the difference across firmware, device enumeration, kernel decompression and init.
6. An 8 GiB guest dirties 0.5 GiB/s over a 2 GiB working set and the link carries 1.1 GiB/s. With the figure's model, how many pre-copy rounds bring the remainder under 100 ms of transfer, and how long does the migration take? For a guest backed by 2 MiB pages, what granularity do you track, what does it cost in the final stop-and-copy, and when would you split pages?
7. Two tenants' vCPUs share a core's hyperthreads. Name two concrete side channels, the mitigation for each, and its cost in Compute's capacity model.
8. Your bare-metal hypervisor must handle a guest `wrmsr` to `IA32_EFER`. What should it allow, refuse and virtualise, and where in the VMCS does the answer live?
9. Confidential VMs (SEV-SNP or TDX) encrypt guest memory against the hypervisor. What can the hypervisor still do to a guest, and which OMH claim would a confidential tier support that plain Compute cannot?

Deliverables and stretch

Deliverables. The KVM VMM (direct boot, 16550, virtio-blk, virtio-net, VFIO passthrough, two vCPUs, dirty tracking and migration) with a README and test scripts; the VT-x hypervisor with its VMCS setup documented field by field and its exit statistics; the Compute VMM API and architecture decision; measurements of exit costs, boot times, migration downtime and tenant interference; the Arm port and its comparison.

Stretch. Implement virtio-vsock and a guest agent, which is how Compute will run in-guest operations such as key rotation and metrics without a network path. Add virtio-iommu so a guest can run VFIO itself, and test it with nested passthrough of a VF. Give your bare-metal hypervisor a `vmcall` interface and boot your Chapter 4.1 kernel as an L2 guest under KVM under your hypervisor. Implement a vTPM device backed by `swtpm`, so that the guest's PCRs include a measurement of the host's attestation result.

COMPLETION CHECKLIST

- ☐ **Lab 4.4.1:** a 64-bit guest from nothing, with MMIO exits and injected interrupts; the exit-cost table.
- ☐ **Lab 4.4.2:** Linux to a shell on your virtio-blk, networking over your virtio-net, two busy vCPUs, boot time recorded.
- ☐ **Lab 4.4.3:** your Chapter 4.1 kernel as a guest; a VF at over 9 Gb/s through the IOMMU; the DMA fault caught.
- ☐ **Lab 4.4.4:** Linux on your bare-metal hypervisor; two isolated guests; exit statistics beside KVM's.
- ☐ **Lab 4.4.5:** start, snapshot and restore through your own API; the decision written.
- ☐ **Lab 4.4.6:** live migration under 100 ms of downtime with the disk intact; the multi-tenant policy measured.
- ☐ **Lab 4.4.7:** Linux in your VMM on the Pi; the exit comparison written.
- ☐ **Problem set:** all nine answered, with problems 2 and 6 checked numerically.
- ☐ **You can explain**, without notes: every step of a VM exit and re-entry, the two-dimensional page walk, a virtqueue round trip, what the IOMMU does for passthrough, and how pre-copy converges or fails to.

CHAPTER 4.5

Storage and network data paths

TIME	2.5 weeks
YOU BUILD	Every layer between a write and the flash measured, with a power-cut test of acknowledged writes; SPDK and a vhost-user disk for your VMM; a Reed-Solomon stripe store at memory bandwidth; NVMe over Fabrics with reservation fencing; four network receive paths compared at line rate; a replication transport; a Blocks prototype that survives losing a node.
NEEDS	The lab server with the Intel P4510 and the sacrificial Kingston NV3, the X520-DA2 with a DAC between its ports, the Pi 5 with its NVMe drive, your Chapter 4.4 VMM, `fio`, liburing, SPDK, DPDK and ISA-L.

WHAT THIS BUYS OMH

Storage, Blocks, Tables, Queues and Relational share one data path. Bytes arrive on a NIC, are transformed, land on NVMe and are copied to other modules so that a failed drive or module loses nothing. OMH's specification sheet promises erasure coding across hot-swap bays, drive replacement without downtime and encryption with keys in Modulus. The data path decides whether they cost one core per drive or eight, through the copies per byte, the interrupts per I/O and the speed of the erasure code. This chapter measures every layer between a tenant's write and replicated flash, and builds the two primitives Storage and Blocks share: a fast local NVMe path and a fast path between modules.

SKIP TEST

1. A 4 KiB `pwrite` with `O_DIRECT` to an NVMe drive: list every copy, mode switch and interrupt to completion. Repeat for `io_uring` with `SQPOLL` and `IOPOLL`, then for SPDK.
2. Reed-Solomon over GF(2^8) with four data and two parity shards: how many bytes are read to encode a stripe and to repair one shard, and why does a systematic code matter for reads?
3. NVMe over Fabrics on TCP and on RDMA: what does each cost the initiator's CPU per I/O?
4. Why does DPDK or AF_XDP beat the kernel at 64-byte packets, and what does it give up?
5. An enterprise drive advertises power-loss protection. What exactly is protected, and what does a consumer drive do with your `fsync`?

If all five are easy, do Labs 4.5.2, 4.5.4 and 4.5.5.

Core ideas

The storage path in Linux. A buffered `write` copies into the page cache and returns. Writeback later builds bios, the filesystem decides layout and order, and blk-mq moves each request from a per-CPU software queue to a hardware queue mapped to an NVMe submission queue. The driver writes a 64-byte command and rings the doorbell; the controller fetches it, moves the data by DMA and posts a completion, usually with an MSI-X interrupt. `O_DIRECT` skips only the page cache. `fsync` forces the journal and sends a Flush, or relies on FUA writes. Each layer costs microseconds, which is negligible for a hard disk and a large share of an NVMe drive's tens of microseconds. The generic device `/dev/ng0n1` with `io_uring` passthrough keeps the driver and skips the block layer.

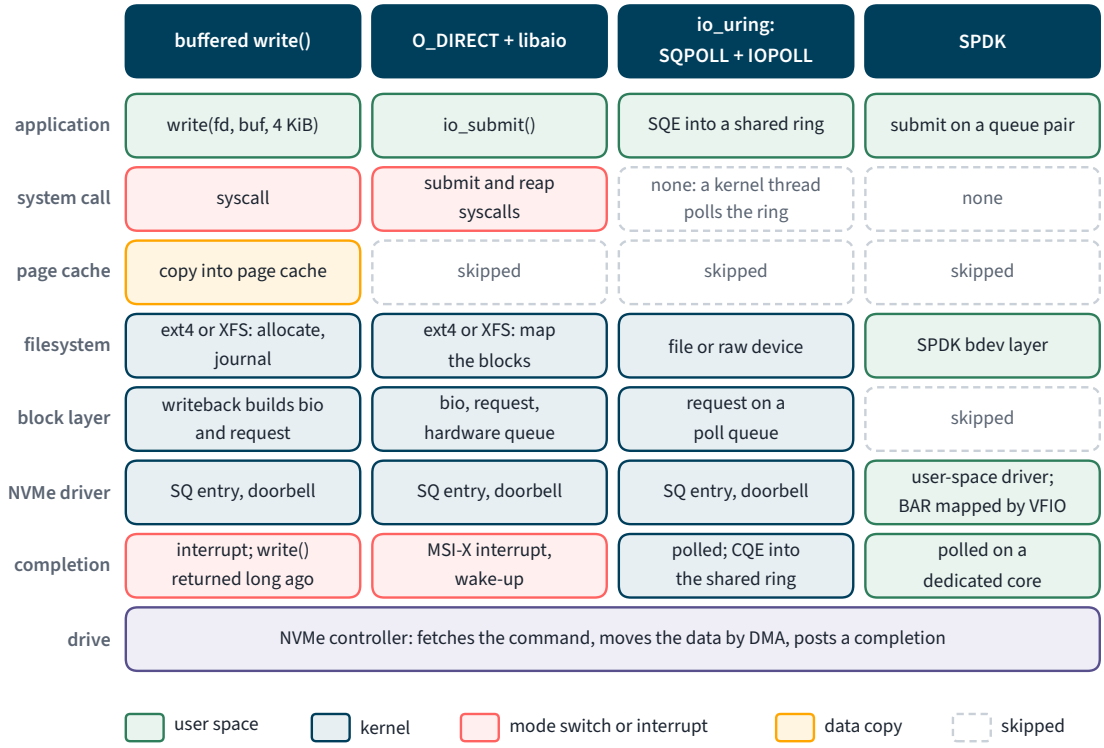


Figure 4.5.1. Four ways from a 4 KiB write to flash. Labs 4.5.1 and 4.5.2 measure what removing each layer saves.

Asynchronous I/O and polling. libaio is asynchronous only with `O_DIRECT`. `io_uring` shares submission and completion rings with the kernel. With `SQPOLL` a kernel thread watches the submission ring, so a busy application makes no system calls; with `IOPOLL` completions are polled from the driver, which needs NVMe poll queues (`nvme.poll_queues`). Registered buffers skip per-I/O pinning, and linked requests order a write before its flush. An interrupt and wake-up cost several microseconds, much of a fast drive's latency at queue depth 1. Polling removes them and spends a core (hybrid polling sleeps through part of the wait first), and Blocks picks a point on that curve per volume tier.

SPDK. SPDK takes the drive from the kernel through VFIO, maps its BARs into the process and runs a lock-free poll-mode driver on a dedicated core with pinned hugepage buffers. With no interrupts, system calls or copies, one core completes a million or more 4 KiB I/Os per second. Its bdev layer stacks RAID, encryption and logical volumes, and its NVMe-oF and vhost-user targets export them. The cost is a core that always spins and the loss of kernel tooling.

Erasure coding. Three-way replication survives two failures at three times the capacity; a (k, m) Reed-Solomon code survives any m lost shards at $(k+m)/k$. Encoding multiplies the data by a generator matrix over GF(2^8), where addition is XOR. A systematic generator stacks the identity on an $m \times k$ matrix C, so data shards are stored as they are and healthy reads touch only them. With a Cauchy matrix, $c_{ij} = 1/(x_i + y_j)$, every square submatrix is invertible, so any k survivors rebuild the stripe: take their rows, invert, multiply. The field multiply sets the speed. Nibble lookups with `vpshufb` handle 32 bytes per instruction on AVX2 and reach memory bandwidth. ISA-L uses the polynomial 0x11D while GFNI's `gf2p8mulb` is fixed to the AES polynomial 0x11B, so fast libraries use `gf2p8affineqb` with an 8×8 bit matrix per coefficient. Each write becomes $k+m$ writes and each lost drive k reads per stripe; that repair bandwidth sets how many drives a module can lose per hour before data is at risk.

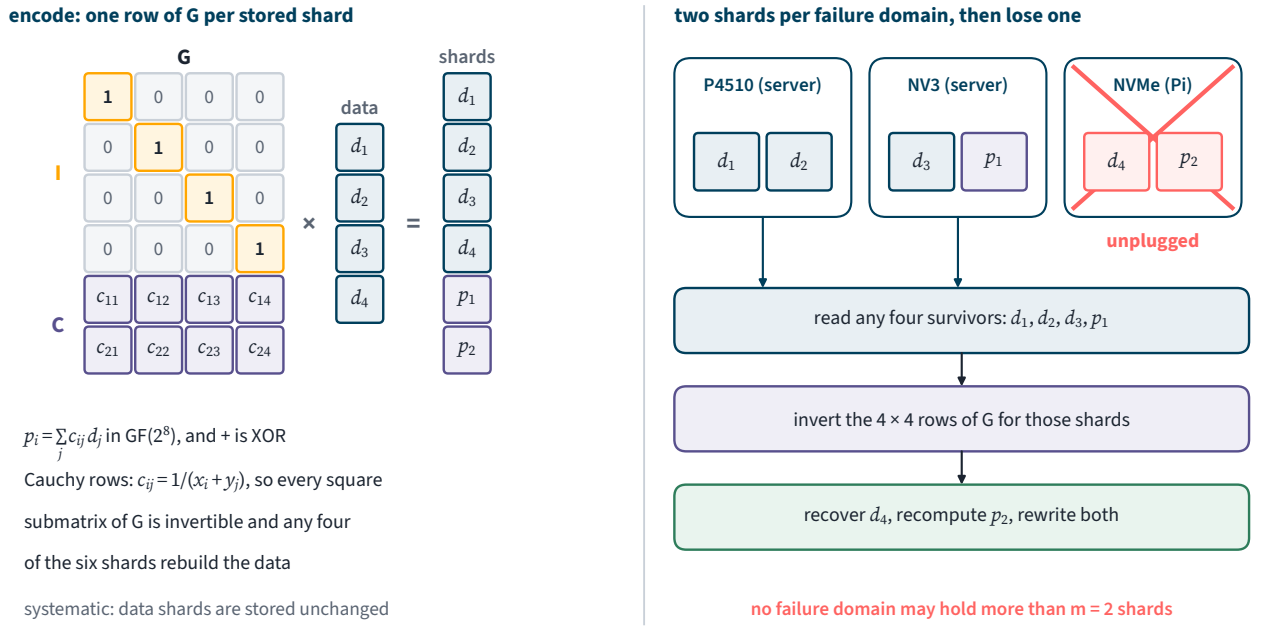


Figure 4.5.2. A (4, 2) stripe. Identity rows keep data shards readable as stored; Cauchy rows make any four survivors enough. Lab 4.5.6's failure domains are two drives and the Pi; Chapter 4.7 makes each a node.

Consistency across modules. A stripe spread over three modules is durable only if a crash at any instant leaves a recoverable state. Updating in place risks the write hole, where data and parity disagree after a crash; intent logs, two-phase commit per stripe, or append-only stripes under a versioned index avoid it, and the last makes snapshots nearly free. A Raft group above the stripes holds placement, which Chapter 4.7 builds.

NVMe over Fabrics. NVMe-oF carries the NVMe command set over RDMA, TCP or Fibre Channel. A target exports namespaces and the initiator presents them as local drives. NVMe/TCP needs no special NIC and costs the initiator a TCP stack and a copy per I/O. RDMA places data straight into registered memory and adds under 10 µs to a remote read for almost no CPU. The X520 has no RDMA engine, so the labs use soft-RoCE (`rdma_rxe`), which runs the protocol on the CPU. It teaches the verbs, but its latency is no guide to RDMA hardware. NVMe/TCP can run over TLS 1.3 with the handshake in user space (`tlshd`). Blocks serves volumes as NVMe-oF namespaces, so choosing a volume tier means choosing a transport.

Reservations and fencing. Hosts register keys under their host identifiers, and one holder acquires a reservation such as Write Exclusive. Reservation Acquire with Preempt, or Preempt and Abort, removes another host's registration, and that host's next write fails with a reservation conflict. This is Blocks' multi-attach fence. Without it, two sleds writing one volume after a partition corrupt it. Linux's kernel target supports reservations when a namespace's `resv_enable` is set, and so does SPDK; on a target without them nothing stops the second writer.

The network path. The NIC DMAs a frame into a receive ring and raises a moderated interrupt. NAPI polls the ring, the driver builds an `sk_buff`, GRO merges segments, netfilter, IP and TCP run, and `recv` copies the payload out, at a few thousand cycles per packet. XDP runs eBPF in the driver before any `sk_buff` exists and returns drop, transmit, pass or redirect. AF_XDP redirects frames to a user-space socket whose UMEM the driver fills directly, and `ixgbe` supports its zero-copy mode. DPDK polls the NIC from user space through `vfio-pci`; RDMA writes into application memory with no CPU. Offloads (checksum, segmentation, RSS, kTLS) and DPUs move work into the NIC. A 64-byte frame occupies 84 bytes of wire time, so 10 Gbit/s line rate is $10^{10}/(84 \times 8) \approx 14.88$ million packets per second, which is 67 ns per packet, about 175 cycles at 2.6 GHz.

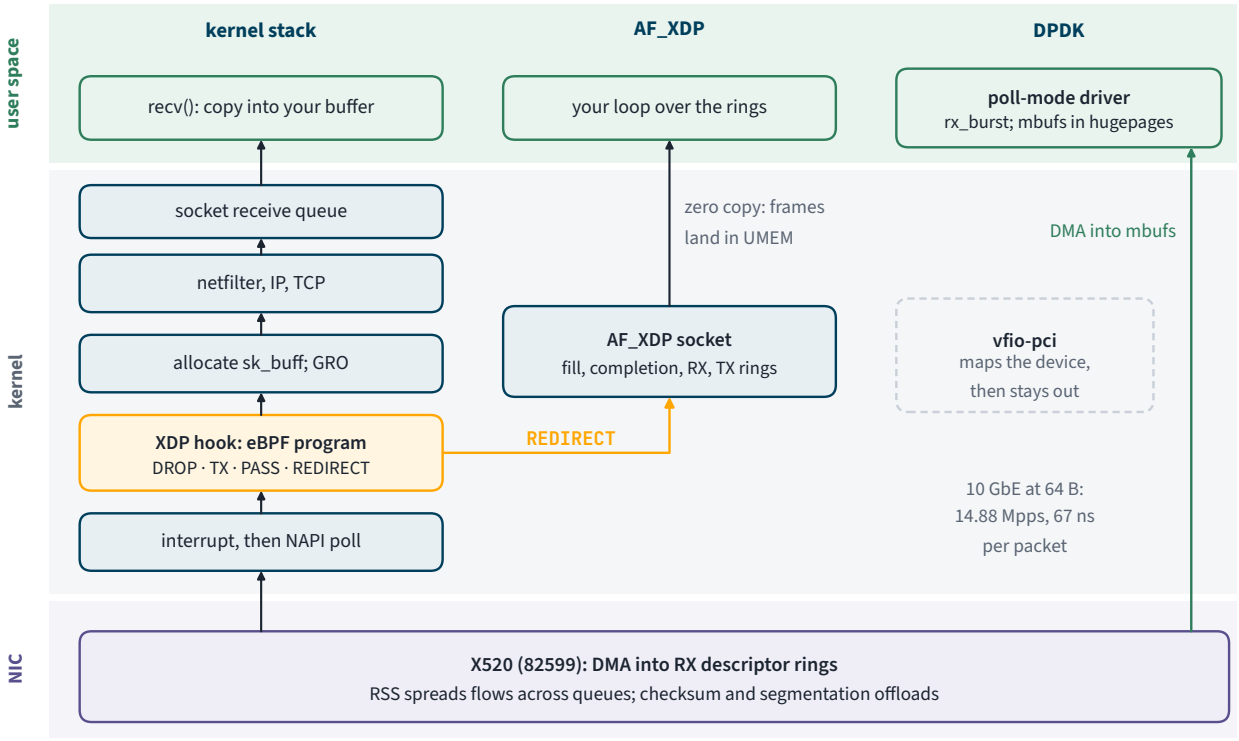


Figure 4.5.3. The receive path and its exits, each measured per core in Lab 4.5.5.

Copies, cores and capacity. The lab server's eight DDR4-2400 channels peak near 150 GB/s, and a copy costs twice its size. Zero copy means the NIC DMAs into the buffer the encoder reads and the drive DMAs from the one it writes; each copy avoided saves part of a core at 25 Gbit/s and a whole core at 100.

Encryption in the path. Use AES-XTS at rest, with keys unwrapped from Modulus into memory only, and AES-GCM in flight. AES-NI runs at several GB/s per core, so the decision is where to encrypt, or whether to trust the drive's TCG Opal instead.

Durability. Power-loss protection keeps a drive alive long enough to save its write cache. A drive without it reports a volatile write cache and may hold acknowledged data in DRAM until a Flush, and one that acknowledges a Flush early is found only by cutting its power. Per-block checksums (CRC32C, xxHash) verified on read catch silent corruption that erasure codes cannot see, and a scrubber finds it before a second failure makes it permanent. Endurance also limits the design. The NV3 500 GB is rated for 160 TB written, which at a write amplification of 3 allows about 49 GB of host writes a day for three years.

Reading

- NVMe Base Specification 2.x on queues, Identify Controller, the SMART log, reservations and Fabrics; the NVM Command Set and NVMe/TCP Transport specifications.
- Plank's 1997 Reed-Solomon tutorial with its 2005 correction; Plank, Greenan and Miller, "Screaming Fast Galois Field Arithmetic", FAST 2013; ISA-L's `erasure_code/`.
- Axboe, "Efficient IO with io_uring"; the liburing man pages; kernel `blk-mq.rst` and `ublk.rst`.
- SPDK's NVMe, bdev, NVMe-oF and vhost pages; the DPDK guide on poll-mode drivers and mbufs; "The eXpress Data Path", CoNEXT 2018; kernel `af_xdp.rst`.
- The Raft paper (Ongaro and Ousterhout, 2014) for Chapter 4.7; the GFS and Ceph papers for placement.
- Pillai et al., "All File Systems Are Not Created Equal", OSDI 2014, on what `fsync` guarantees.

Labs

Every lab but Lab 4.5.3 destroys data on the NV3: keep the operating system on another disk, and save each drive's `nvme smart-log` first.

LAB 4.5.1 Measure the local storage path — 1 weekend

Goal. measure what each layer costs, and whether each drive keeps acknowledged writes through a power cut.
Kit. The server, both drives, the Pi.

1. Boot with `nvme.poll_queues=4`. Record `nvme id-ctrl` for both drives, including VWC, and predict their behaviour under `fsync`.
2. Run `fio` with `direct=1` on the raw NV3: `psync`, `libaio`, `io_uring` (plain, `sqthread_poll`, then `hipri` with `fixedbufs`) and `io_uring_cmd` on `/dev/ng1n1`, at 4 KiB and 128 KiB, depths 1 to 128, random read and write. Record IOPS, bandwidth and p50 to p99.9 latency. Repeat on ext4, on XFS, with `fsync=1`, and on the P4510.
3. Explain each row against the last. Where Little's law (Chapter 2.4) fails, find the bottleneck with `perf` and `bpftrace` on the block tracepoints.
4. Write a C++ liburing harness with registered buffers, `IOPOLL`, a fixed pool of in-flight requests and a latency histogram, and match `fio`'s best rows.
5. Cut the power. Write sequenced, CRC-checked 4 KiB blocks with `fdatasync`, send each acknowledged sequence number by UDP to the Pi, and cut the server mid-run from the Pi with a Redfish `ForceOff` to the iDRAC (Lab 2.7.7). Count acknowledged blocks lost. Make five cuts per drive, then five without `fdatasync`.

> **SAFETY**
> `ForceOff` cuts power to the whole host. Keep only the boot disk mounted read-write, and nothing on the server that you need.

DONE WHEN

- [ ] The matrix has a sentence per row, and your harness is within 10 % of `fio`'s best IOPS at depth 32.
- [ ] Twenty power cuts are logged with the acknowledged blocks lost per drive and mode.

EVIDENCE FOR THE NOTEBOOK

- `fio` output; the harness; the power-cut log with each drive's VWC field.

LAB 4.5.2 SPDK, and a vhost-user disk for your VMM — 1 weekend

Goal. take the kernel out of the storage path.

1. Build SPDK, bind the NV3 with `scripts/setup.sh`, run `spdk_nvme_perf` and `bdevperf`, and explain the gap to your best `io_uring` row, with CPU per I/O from `perf stat`.
2. Write an SPDK application that attaches the controller, allocates a queue pair, reads into `spdk_dma_zmalloc` memory and polls for completions. Compare it with your Chapter 4.1 NVMe driver.
3. Stack a crypto bdev (`AES_XTS`, key from `accel_crypto_key_create`) and a logical volume store, and measure encryption's cost per core.
4. Create a controller in SPDK's `vhost` target and implement the vhost-user front end in your Chapter 4.4 VMM: shared hugepage memory, `VHOST_USER_SET_MEM_TABLE` with descriptors passed over the socket, the vring messages, and eventfds bound to `KVM_IOEVENTFD` and `KVM_IRQFD`. Compare guest `fio` with your file-backed virtio-blk.

DONE WHEN

- [ ] A guest reaches within 20 % of host SPDK throughput at 4 KiB random read, depth 32, and the cost table includes CPU per I/O and AES-XTS per core.

EVIDENCE FOR THE NOTEBOOK

- The SPDK rows; a vhost-user message trace of one guest boot.

LAB 4.5.3 Reed-Solomon in C++ at memory bandwidth

1 weekend

Goal. write an encoder fast enough that the network sets the limit.

1. Implement GF(2^8) with 0x11D, tested against a slow carry-less reference; a Cauchy generator; a systematic encoder; and a Gaussian-elimination decoder tested on every combination of m lost shards for (4, 2) and (10, 4).
2. Vectorise with nibble tables and `_mm256_shuffle_epi8`. Benchmark (10, 4) encode and repair with 1 MiB shards on one core against ISA-L's `ec_encode_data`, then read ISA-L's AVX2 kernel and close the gap. The E5-26xx v4 lacks GFNI; on a workstation that has it, add a `gf2p8affineqb` kernel.
3. Build a stripe store: each shard in its own file behind a header (stripe ID, shard index, version, CRC32C), a reader that rebuilds from any k and verifies, and a scrubber.
4. Write 10 GiB as (4, 2) across both drives, delete two shard files, read back and time the repair. Flip a byte in one shard and show the checksum catching it.

DONE WHEN

- [ ] The encoder is within a factor of two of ISA-L and passes every loss pattern; the store returns every byte after two lost shards and a silent corruption.

EVIDENCE FOR THE NOTEBOOK

- The benchmark table; the failure-test log.

LAB 4.5.4 NVMe over Fabrics between two nodes

1 weekend

Goal. serve a volume across the network, and fence a host.

Kit. The server, the Pi, the X520 and its DAC.

1. Export an NV3 partition with `nvmet-tcp` through `/sys/kernel/config/nvmet`. From the Pi, `nvme connect` and run `fio` over 1 Gbit/s.
2. For 10 Gbit/s, put the X520's second port in a network namespace and connect from it with `ip netns exec`; the host driver opens its sockets in the caller's namespace, so traffic crosses the DAC. Rerun `fio` and trace the target's TCP send path with `bpftrace`.
3. Serve the namespace from SPDK's `nvmf_tgt` and compare CPU per I/O. Build `rdma_rxe` into both kernels, run `rdma link add rxe0 type rxe netdev <port>`, connect with `-t rdma` and compare depth-1 latency with TCP.
4. Fence. Register and acquire Write Exclusive from the Pi with `nvme resv-register` and `nvme resv-acquire`; show the server's initiator gets a reservation conflict; preempt the Pi from the server. Repeat on SPDK and with `resv_enable` unset, and write the fencing protocol for a partitioned sled on one page.
5. Reverse the roles and boot a guest in your VMM from the Pi's drive.

DONE WHEN

- [ ] A guest boots from a remote volume, and reservations fence the second host wherever the target implements them.
- [ ] The transport table gives depth-1 latency and CPU per I/O for kernel TCP at 1 and 10 Gbit/s, SPDK and soft-RoCE.

EVIDENCE FOR THE NOTEBOOK

- The target scripts; the reservation transcript; the fencing page.

LAB 4.5.5 The network path: kernel, XDP, AF_XDP and DPDK

1 weekend

Goal. measure the cost per packet at each exit, and build the replication transport.

Kit. The X520's ports joined by the DAC, one in a network namespace; the Pi.

1. Baseline with `iperf3`, then 64-byte frames from `pktgen` (`samples/pktgen`): packets per second and CPU per packet against 14.88 Mpps, with `ethtool -S` and `perf top`.
2. XDP: drop one UDP port with `ip link set dev <port> xdpdrv object drop.o section xdp`, then reflect frames with `XDP_TX` and compare with a user-space `recvmmsg` echo.
3. AF_XDP: bind a UMEM and its rings with libxdp's `xsk` API and `XDP_ZEROCOPY` to one queue, steer the flow there with `ethtool -N`, and echo.
4. DPDK: bind both ports to `vfio-pci`, generate with `dpdk-testpmd` in `txonly` mode on one, and forward with your own `rte_eth_rx_burst` loop on the other, as two processes with separate `--file-prefix` values. Aim for line rate on one core.
5. Build Chapter 4.7's replication transport: CRC32C-checked frames over TCP with `io_uring` and `IORING_OP_SEND_ZC`. Measure GB/s per core at 1 MiB across the DAC and to the Pi.

DONE WHEN

- [ ] The four paths are compared per core at 64 bytes and at 1 MiB, and the transport moves shards to the Pi with every checksum verified.

EVIDENCE FOR THE NOTEBOOK

- The programs; the comparison table.

LAB 4.5.6 A Blocks prototype — 2 weekends

Goal. build a volume a guest boots from, which survives a lost node.

1. Build a volume of 1 MiB stripes coded (4, 2), with two shards each on the P4510, the NV3 and the Pi's drive, the last through your transport. Writes append new stripe versions under an offset index, and a compactor reclaims old ones.
2. Expose it with `ublk` (ublksrv's library, or your own `io_uring` loop on `/dev/ublkc0`) and boot a guest in your VMM from `/dev/ublkb0`.
3. Run `fio` in the guest, then pull the Pi's cable mid-write: I/O continues on four shards, and the store repairs when the cable returns.
4. Snapshot by copying the copy-on-write index, and boot a second guest from a clone.
5. Measure cores per GB/s of erasure-coded writes, and memory bandwidth with `pcm-memory`.

DONE WHEN

- [ ] The guest sees no I/O errors when the Pi disappears mid-write; repair completes and a scrub verifies every checksum.
- [ ] A clone boots a second guest, and cores per GB/s is recorded as the first line of Chapter 4.6's cost model.

EVIDENCE FOR THE NOTEBOOK

- The cable-pull script; guest results, healthy and degraded.

Problem set

1. Compute overhead, repair reads and failures tolerated for 3× replication, (4, 2), (10, 4) and (17, 3). Which suits a six-bay module, and which a rack of ten?
2. A stripe write to three modules stops when one loses power after four of six shards are written. Give two designs that keep the volume consistent and their write-path costs.
3. From Lab 4.5.3, how many cores does encoding 10 GB/s of (10, 4) take, and how much memory bandwidth with one copy in and one out? Where does the 150 GB/s server hit its ceiling, and what removes the copies?
4. `io_uring` with `SQPOLL` and `IOPOLL` still costs something per I/O. List what, and estimate each cost in nanoseconds.

5. Why does the NVMe/TCP initiator copy each read completion, and how does RDMA avoid it? What must the RDMA NIC know about the buffers, and who tells it?
6. A consumer drive acknowledges flushes it has not completed. Design a test that proves it in one power cycle, and say what Blocks must do on such drives.
7. Two sleds attach one Blocks volume; sled A loses the control plane and keeps its path to the storage module. Walk the fencing protocol and name the NVMe command and action that stop A's writes.
8. Should Lab 4.5.6 encrypt in the guest, the VMM, the stripe store before encoding, each shard, or the drive? Compare key custody, repair, deduplication and CPU.
9. A bit flips silently in a shard. Trace its detection on read and on scrub, and what parity without a checksum would do.

Deliverables and stretch

Deliverables. The explained measurement matrix; the power-cut log; the Reed-Solomon library and benchmarks; the stripe store, transport and Blocks prototype with a cable-pull script; the fencing page and the encryption decision.

Stretch. Make the stripe store an SPDK bdev module and compare cores per GB/s. Add local reconstruction codes (Huang et al., USENIX ATC 2012) and measure repair traffic. Build Storage's skeleton, an S3 subset (PUT, GET, multipart upload, range reads, ETags), on the stripe store. Move the transport to RDMA verbs on soft-RoCE.

COMPLETION CHECKLIST

- ☐ **Lab 4.5.1:** the matrix explained; the harness within 10 % of `fio`; twenty power cuts logged.
- ☐ **Lab 4.5.2:** SPDK rows and application; a vhost-user guest within 20 % of host SPDK.
- ☐ **Lab 4.5.3:** encoder within 2× of ISA-L; the stripe store survives two lost shards and a silent corruption.
- ☐ **Lab 4.5.4:** a guest boots from a remote volume; reservations fence a second host; the transport table.
- ☐ **Lab 4.5.5:** four receive paths compared per core; the transport verified end to end.
- ☐ **Lab 4.5.6:** a guest survives losing the Pi mid-write; a snapshot boots; cores per GB/s recorded.
- ☐ **Problem set:** all nine answered.
- ☐ **You can explain**, without notes: the copies, mode switches and interrupts in a 4 KiB write on each path, how a Cauchy matrix lets any k shards rebuild a stripe, what a reservation conflict proves, and why only a power cut shows that a drive acknowledges flushes it has not completed.

CHAPTER 4.6

Product engineering

TIME	2 weeks
YOU BUILD	Module Zero's specification, threat model, firmware update architecture (exercised on real hardware), manufacturing and test plan (with Board A's self-test and provisioning run), compliance plan (with a pre-compliance scan) and cost model.
NEEDS	Your notebook, a spreadsheet, Boards A and B, the lab server, your Lab 2.7.4 verifier, the tinySA Ultra with the Tekbox near-field probes, a spare SparkFun ATECC608A breakout.

WHAT THIS BUYS OMH

A product is a prototype plus the documents that let someone else build it, test it, sell it legally, support it and price it, and the decisions that keep those documents consistent. Module Zero needs a specification, a threat model, an update architecture that fixes it in the field without bricking a rack, a test plan that turns boards into working units, a compliance plan that keeps it legal to sell, and a cost model that says whether the company survives. In this chapter you write all six from your notebook. Some of it is already law. Since 11 September 2026 the EU has required manufacturers to give a 24-hour early warning of any actively exploited vulnerability in their products.

SKIP TEST

1. What must Module Zero meet to be sold in Great Britain, the EU and the US, and which test does a server usually fail first?
2. Define roots of trust for update, measurement and reporting, and name the component of Module Zero that provides each.
3. An update reaches 40 of 100 modules before a bug halts the rollout. What state must the rack be in, and what put it there?
4. Give a 1U server's cost structure (materials, assembly, logistics, warranty, support) as shares of unit cost, and the gross margin a hardware company needs.
5. What is a failure mode and effects analysis, and what would the top three rows of one for Module Zero say?

If all five are easy, skip the reading and go straight to the labs.

Core ideas

The specification. It states what the module does, for whom, at what performance and reliability, through which interfaces, and what it does not do; every later document points back at it. Good specifications are short, numeric and testable ("sustains 2 GB/s of erasure-coded writes at 8 W per TB with two drives failed" in place of "high performance") and name the standards they adopt: Redfish, NVMe-oF, DC-SCM, Open Rack.

Threat modelling. List the assets (tenant data, keys in Modulus, firmware integrity, availability) and the adversaries (a tenant, a rack neighbour, someone with physical access, a supply-chain interposer, a compromised update server, an insider). The trust boundaries are those earlier chapters built: the hypervisor between tenants (Chapter 4.4), the IOMMU between devices and memory, the BMC between the management network and the host, measured boot between firmware versions (Chapter 2.7), and the TPM around the module's secrets. STRIDE at each boundary (spoofing, tampering, repudiation, information disclosure,

denial of service, elevation of privilege) yields the threats. Each threat gets a mitigation, an accepted risk or a gap, and the gaps form the roadmap.

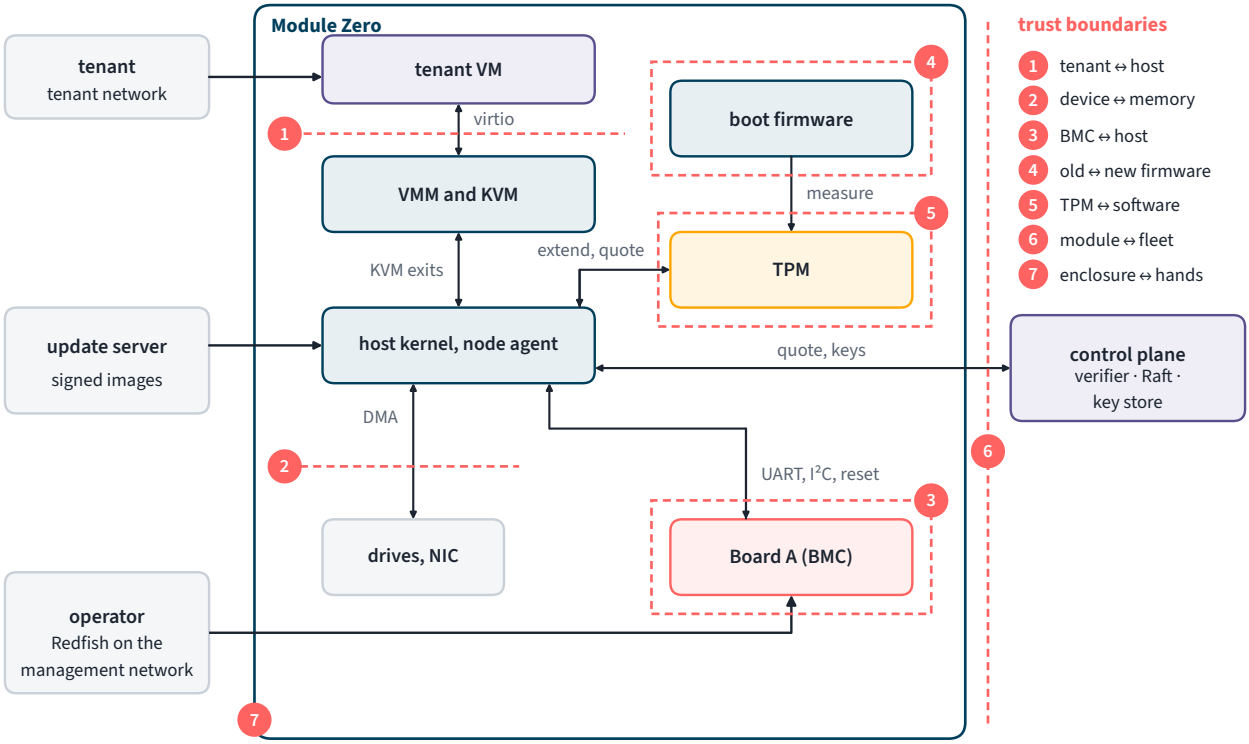


Figure 4.6.1. Module Zero's data flows and trust boundaries. Lab 4.6.2 applies STRIDE at each numbered boundary.

Roots of trust. A root of trust is assumed correct because nothing beneath it can check it. TCG names roots for measurement, storage and reporting; NIST SP 800-193 adds update, detection and recovery, and asks of each firmware component whether it is protected, checked and recoverable. In Module Zero, a CPU's boot ROM verifies the first firmware only where signed boot is fused and enabled. The server's TPM and Board B's SLB 9672 measure, store and report. Your Lab 2.6.6 bootloader, in write-protected flash, verifies Board A's firmware. Identity is held in Board A's ATECC608 and Board B's TPM endorsement key. OCP's Caliptra shows what a mature design puts in silicon.

Firmware update architecture. Every programmable part will need a new image: the server's UEFI, Board B's CM5 bootloader EEPROM and U-Boot, Board A's bootloader and application, and drive, NIC and TPM firmware. A locked ATECC608 configuration never changes. Each image needs a signed format, verification from a root of trust, A/B slots or recovery, anti-rollback, a delivery path (Redfish `UpdateService`, PLDM over MCTP behind the BMC, `fwupd` on the host) and a place in a dependency order. A rack updates one module at a time, each drained, updated, attested against the release's reference values and readmitted; a failure halts the rollout in a known mixed state. The Uptane standard is the best-documented design for signed multi-component updates; borrow its roles and metadata.

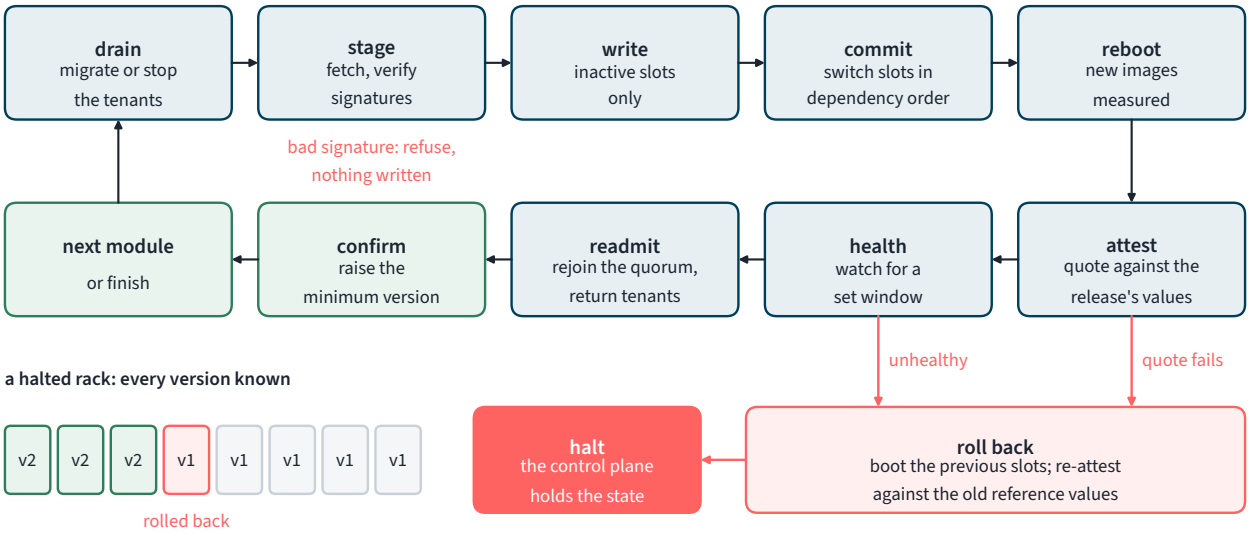


Figure 4.6.2. The per-module update sequence, and what a halted rollout leaves behind.

Reliability. Failure rates of parts in series add, and the annualised failure rate follows from the MTBF:

$$\lambda_{\text{module}} = \sum_i \lambda_i, \quad \text{AFR} = 1 - e^{-8760/\ \text{MTBF}}$$

so a 1,000,000-hour MTBF fails 0.87 % of units a year. Field data such as Backblaze's drive statistics is more reliable than datasheets, and redundancy belongs where the arithmetic demands it. A failure mode and effects analysis lists how each part fails, the effect, its detection and its mitigation, ranked by severity times occurrence times detectability. ASHRAE class A2 allows 10 to 35 °C at the inlet; add humidity, altitude and transit shock. Burn-in finds infant mortality before a customer does.

Manufacturing and test. A contract manufacturer builds from your design package, BOM and test procedures, and design for test gives each board a fixture that programs, tests and provisions it in minutes. Provisioning creates each unit's identity. The ATECC608 generates its private key internally, and the fixture records the public key and serial, installs the first firmware and enrols the identity in the fleet verifier. Flying probe finds opens and shorts before power, functional test runs self-test firmware against limits, and system test runs the real firmware, attests to a factory verifier and burns in. First-pass yield decides whether the cost model's price can be met.

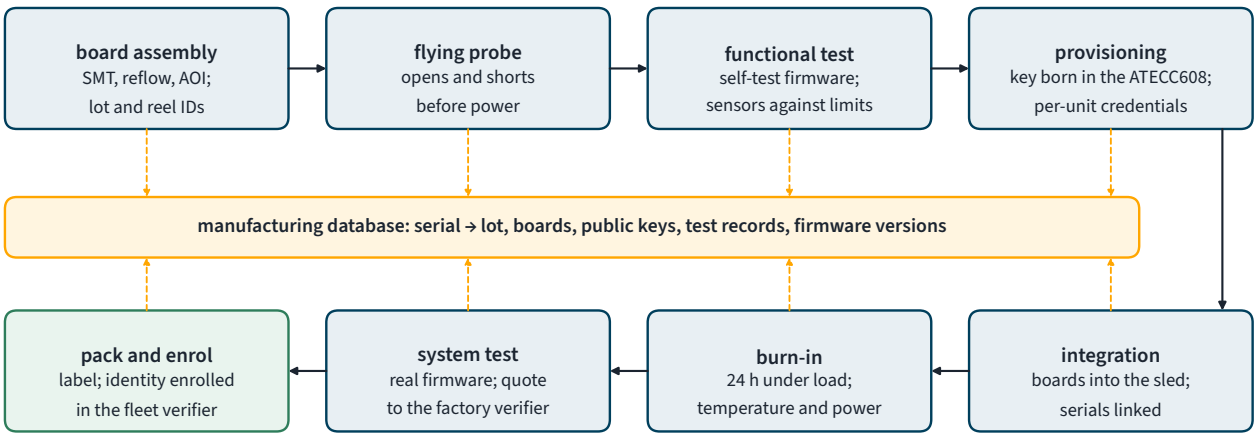


Figure 4.6.3. From bare board to enrolled module. Every station writes to the manufacturing database.

Product compliance. Module Zero is IT equipment with no radio. In Great Britain and the EU it needs safety to IEC 62368-1 (the fourth edition, 2023, is current; check which edition is harmonised and designated before testing) under the Low Voltage Directive and the UK Electrical Equipment (Safety) Regulations 2016.

It also needs EMC compliance under Directive 2014/30/EU and the UK EMC Regulations 2016, tested to EN 55032 for emissions and EN 55035 for immunity. Great Britain accepts the UKCA or CE mark indefinitely, so one technical file serves both. RoHS limits ten substances to 0.1 % of each homogeneous material (cadmium 0.01 %), with files kept for ten years. REACH makes you declare any substance of very high concern above 0.1 % by weight and, in the EU, notify it to ECHA's SCIP database. UK WEEE producers register yearly (through a compliance scheme above five tonnes a year), mark the crossed-out wheeled bin and keep records for four years. The EU's server ecodesign regulation, 2019/424, requires secure data deletion and firmware and security updates until eight years after a model's last unit is placed on the market, unless an exclusion applies. In the US, FCC Part 15 Subpart B Class A applies, by Supplier's Declaration of Conformity or certification, and the cryptography needs an export classification (ECCN 5A002, or 5A992 as mass-market). Radiated emissions fails most often; a bench near-field scan finds most problems first.

Security regulation. The EU Cyber Resilience Act, Regulation (EU) 2024/2847, entered into force on 10 December 2024. Since 11 September 2026 manufacturers must report actively exploited vulnerabilities and severe incidents through ENISA's single reporting platform: an early warning within 24 hours, a notification within 72, and a final report 14 days after a fix for a vulnerability or a month after notification for an incident. From 11 December 2027 the whole Act applies, with conformity assessment, CE marking and updates for a declared support period. Radio equipment has had cybersecurity rules since 1 August 2025 under Delegated Regulation (EU) 2022/30, with EN 18031-1 to −3 listed, with restrictions, as its standards. Delegated Regulation (EU) 2026/339 repeals 2022/30 from 11 December 2027, when the CRA takes over. Module Zero has no radio, so it is outside 2022/30, which in the UK applies only in Northern Ireland. Great Britain has PSTI, in force since 29 April 2024 for consumer connectable products. It bans universal default and guessable passwords, and requires a published way to report security issues, with response times, and a published minimum update period. Whether a module sold only to businesses is covered needs legal advice; the three requirements cost little.

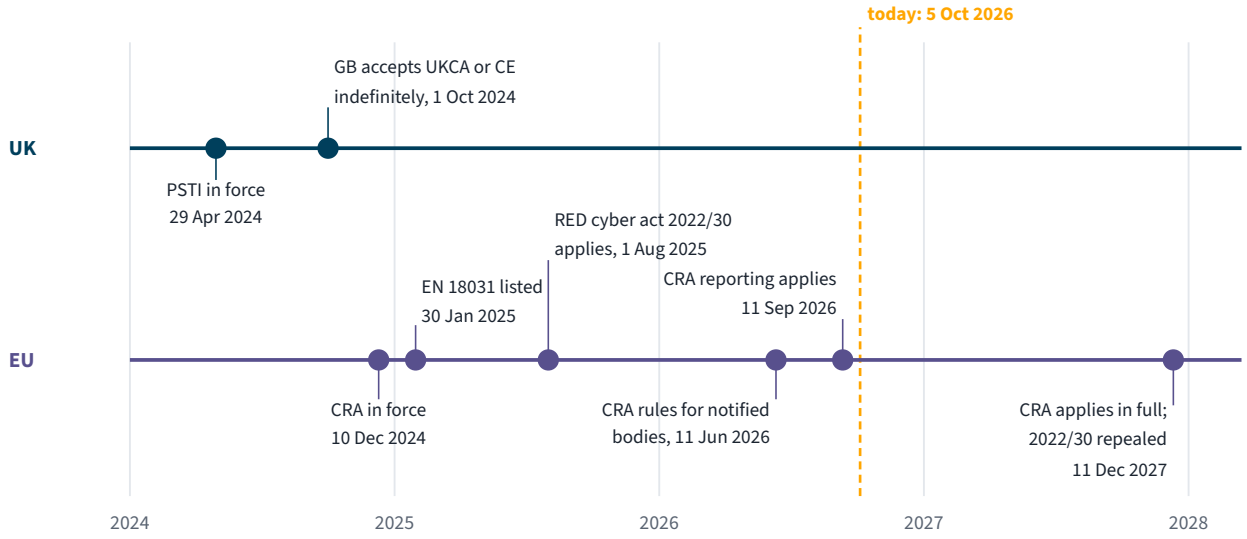


Figure 4.6.4. UK and EU dates that bear on Module Zero. CRA reporting applies now, the rest of the Act from December 2027.

Cost model. Unit cost is the BOM at volume, with price breaks and exchange rates, plus labour, yield loss, logistics and duties, a warranty reserve, support, compliance upkeep and the cash tied up in inventory; gross margin is price minus all of it. Hardware companies that last run 35 to 60 % gross margin with services on top, which for OMH are the AWS-class services. Respins, compliance tests, tooling and this year are non-recurring costs.

Operations and documentation. A module in a customer's rack reports health without tenant data and is replaced from a shipped spare without an OMH engineer; the FMEA says which service levels you can afford. Guides, the Redfish schema, release notes and the disclosure policy are much of what an on-premises customer buys.

Reading

- NIST SP 800-193, all of it; RFC 9334 again, with Module Zero as attester and the quorum as relying party.
- OCP Caliptra's overview and boot flow; the Uptane Standard's roles and metadata; DMTF DSP0267 (PLDM for Firmware Update) and DSP0236 (MCTP).
- Shostack, *Threat Modeling*, chapters 1 to 5 and 7; Arthur and Challener, *A Practical Guide to TPM 2.0*, chapters 9, 12 and 22; OCP Yosemite v3 and DC-SCM 2.0.
- Regulation (EU) 2024/2847, Articles 13 and 14 and Annex I; the Commission's CRA reporting page; GOV.UK's PSTI and UKCA guidance.
- Ott, *Electromagnetic Compatibility Engineering*, chapter 18 (pre-compliance); FCC Part 15 Subpart B.
- Backblaze's drive statistics; Schroeder and Gibson, FAST 2007, on real disk failure rates; Ben Einstein's "Hardware by the Numbers".

Labs

Each lab is a document written from your notebook. Mark every number you had to guess. Those are the measurements you still owe, and Chapter 4.7 collects them.

LAB 4.6.1 The Module Zero specification 8 h

Goal. say what Module Zero is, in numbers you can test.

1. Write a page of positioning: who buys Module Zero and what they stop doing. Map each claim on the OMH site to the mechanism in this course that delivers it.
2. Fix the configuration: sled envelope (Yosemite v3, Open Rack v3 or your own), compute, memory, drives (M.2, U.2 or E1.S), NICs, Board A's successor, the TPM, power input and fans.
3. Define the interfaces: management (Board A's Redfish subset, console, join protocol), data (Ethernet, NVMe-oF) and the service APIs.
4. Set numeric targets, each with its measurement: storage throughput and latency (Lab 4.5.6), VMs per module (Chapter 4.4), boot to joined (Lab 2.7.6), power, noise, inlet range, MTBF and the security-update period you will publish.
5. List what Module Zero does not do, and why.

DONE WHEN

- [ ] Every claim on the OMH hardware page maps to a specification line and a mechanism, or is marked unsupported; every target names its measurement and says whether it was measured or guessed.

EVIDENCE FOR THE NOTEBOOK

- The specification; the claim-to-mechanism table.

LAB 4.6.2 The threat model 8 h

Goal. know what Module Zero must never do, and what stops it.

1. Draw the full data-flow diagram, extending Figure 4.6.1 with Modulus and every network, and mark each trust boundary.
2. Apply STRIDE at each boundary and produce the threat table.
3. Record for each threat the mitigation you built (chapter and lab), one designed but not built, or an accepted risk with its reason. Cross-tenant memory reads, for example, meet Chapter 4.4's EPT isolation, core scheduling and side-channel mitigations, with a residual risk accepted for future speculative-execution bugs.
4. Physical attack: say what the TPM and encryption at rest protect, what an SPI flash interposer defeats, and what a Caliptra-class root of trust would add.

5. Supply chain: a board returns from the manufacturer with modified BMC firmware. Say what detects it, and when.

DONE WHEN

- [ ] No row lacks a mitigation or an explicit acceptance, and the top ten gaps are ranked as a roadmap.

EVIDENCE FOR THE NOTEBOOK

- The diagram; the threat table; the ranked gaps.

LAB 4.6.3 The firmware update architecture

1 weekend

Goal. update a module in the field and survive a bad release.

Kit. Boards A and B, the server, your Lab 2.7.4 verifier.

1. Inventory every programmable part with its update path, format, signing and recovery, including the drives (`nvme fw-download`, `nvme fw-commit`) and the X520.
2. Design the image format and key hierarchy: an offline root, per-component keys, rotation, revocation, and where each public key lives (Board A's bootloader, the Secure Boot db, U-Boot's FIT key). Reuse your Lab 2.6.6 header where it fits.
3. Write the per-module sequence and the rack rollout as state machines: what halts a rollout, the halted state, and how an operator resumes or reverts.
4. Implement it. Board A takes its image through its own Redfish `UpdateService` over lwIP into the inactive slot of your Lab 2.6.6 bootloader, Board B's host image goes through `fwupd` or your own agent, and the verifier is the gate. Update, attest and readmit, then release a broken image and show the rollback and halt.
5. Map it onto the update duties of the CRA, PSTI and ecodesign 2019/424, and list the gaps.

DONE WHEN

- [ ] A real module rolls forward and back under the written architecture, with the verifier as the gate and the control plane reporting the halt.

EVIDENCE FOR THE NOTEBOOK

- The state machines; the update, rollback and halt logs.

LAB 4.6.4 The manufacturing and test plan

1 weekend and a 24 h burn-in

Goal. produce tested, identified units by following the procedure alone.

Kit. Every Board A you built, Board B, the Debug Probe, the spare ATECC608A breakout.

1. Write the process flow from bare board to packed module, naming each station and what it records.
2. Specify functional test for Boards A and B with limits and a time budget. Write Board A's self-test as a test mode of your Chapter 2.6 firmware, entered over the debug UART: the INA228, the TMP117s against each other, both fan tachometers, the ATECC608, an Ethernet ping and the host connector through a loopback plug. Run it on every Board A and record first-pass yield.
3. Specify provisioning (a key generated inside the ATECC608, per-unit Redfish credentials with no default password, the database schema, the label, enrolment before shipping) and implement it with your Lab 2.7.4 verifier as the database.
4. Burn Board B in for 24 hours under `stress-ng` and `fio`, logging temperatures and power through Board A against written criteria.
5. Write the traceability procedure from a module serial to its boards, lots and test records.

SAFETY

Locking the ATECC608's configuration and data zones is permanent. Rehearse provisioning on the spare breakout before you lock a Board A.

DONE WHEN

- [ ] The self-test has run on every Board A with yield recorded, one board is provisioned and enrolled by the procedure alone, and the burn-in log is complete.

EVIDENCE FOR THE NOTEBOOK

- The plan; the self-test firmware; the yield record; the burn-in log.

LAB 4.6.5 The compliance plan

8 h and a bench scan

Goal. list every mark, declaration and report, with its cost and time.

Kit. Boards A and B, the tinySA Ultra, the near-field probes.

1. List every requirement for sale in Great Britain, the EU and the US with its test or declaration, a laboratory, cost, duration and the design features that address it.
2. Safety: walk IEC 62368-1's energy-source classification for Module Zero (mains in the power supply, the 12 V bus, hot surfaces, fans, any battery) and list the safeguards each needs.
3. EMC: name the likely sources (regulators, PCIe, Ethernet, any FPGA clock) and the Chapter 3.4 layout measures against each. Scan Boards A and B with the near-field probes and the tinySA, and attribute the five loudest frequencies.
4. Materials: supplier RoHS and REACH declarations, SCIP notification, WEEE registration and marking, and what leaded prototype solder changes.
5. Security and export: map PSTI and the CRA's reporting duty onto your design and processes, write the 24-hour and 72-hour reporting runbook, and classify Module Zero for export.

DONE WHEN

- [ ] The plan has a costed line per requirement with its design feature, the scan's loudest frequencies are attributed, and the runbook has been rehearsed against a mock vulnerability inside 24 hours.

EVIDENCE FOR THE NOTEBOOK

- The compliance table; the scan; the runbook and the walk-through notes.

LAB 4.6.6 The cost model

8 h

Goal. find out whether the company can survive on Module Zero's margins.

1. Build the BOM at 10, 100 and 1,000 units from your Chapter 3.4 BOMs with distributor price breaks and estimates for drives, NICs and assembly, marking every guess.
2. Add labour from Lab 4.6.4's time budgets at a contract manufacturer's hourly rate, and yield loss from your first-pass yield.
3. Add logistics, tariffs, the warranty reserve from the FMEA, support and compliance upkeep, then the non-recurring costs: respins, Lab 4.6.5's testing, tooling and the engineering year.
4. Set the price from the margin you need, and compare three years of Module Zero with the equivalent AWS services for one typical customer. The OMH pitch depends on that comparison.
5. Model the cash flow, and how many units you can build before revenue.

DONE WHEN

- [ ] Every input is labelled measured, quoted or guessed, and a one-page summary states the unit price, the margin and the cash needed for the first 100 units.

EVIDENCE FOR THE NOTEBOOK
- The spreadsheet; the summary page.

Problem set

1. Module Zero's BMC signing key leaks. Walk revocation and recovery across 1,000 modules, naming what in your architecture makes it possible.
2. Can tenant data be recovered from a returned failed drive? Answer for your Chapter 4.5 encryption placement, and show how you meet ecodesign 2019/424's secure-deletion requirement.
3. A fan failure at 35 °C inlet throttles the module in four minutes. Design detection, mitigation, notification and the SLA. With two 70,000 h MTBF fans per module, how many fail a year across 100 modules?
4. Compare an OMH management board with a commercial DC-SCM card on cost, time, control and the threat model.
5. A run of 100 yields 78 % at first pass; rework at £40 a unit saves 80 % of failures, and the rest are scrapped at a £600 BOM. Find the yield cost per good unit and the Chapter 3.4 decisions to revisit.
6. List Module Zero's Redfish resources and actions, justifying each omission against customers' tooling.
7. A tenant says another read their VM's memory. What can Module Zero prove, and what not?
8. Map the CRA onto your update architecture and PSIRT, and use Annexes III and IV to classify a module that ships a hypervisor and a boot manager. What follows for its conformity assessment?
9. The model shows 28 % gross margin at 100 units. Name three levers that lift it above 45 %, and their cost in time or risk.

Deliverables and stretch

Deliverables. The six documents; Board A's self-test and provisioning, exercised; a recorded update and rollback; the ranked guesses that become Chapter 4.7's measurement plan.

Stretch. Write the PSIRT policy and disclosure page, with a mailbox, a PGP key and a `security.txt` (RFC 9116). Model a ten-customer fleet's support cost over three years, and make the manufacturing database a service.

COMPLETION CHECKLIST
- ☐ **Lab 4.6.1:** every site claim mapped to a mechanism or marked unsupported.
- ☐ **Lab 4.6.2:** every threat mitigated or accepted; the top ten gaps ranked.
- ☐ **Lab 4.6.3:** a real module rolled forward and back with the verifier as the gate.
- ☐ **Lab 4.6.4:** self-test yield recorded; a board provisioned by procedure; burn-in logged.
- ☐ **Lab 4.6.5:** a costed line per requirement; the scan attributed; the reporting runbook rehearsed.
- ☐ **Lab 4.6.6:** unit price, margin and cash for 100 units, with every input labelled.
- ☐ **Problem set:** all nine answered.
- ☐ **You can explain**, without notes: what each of Module Zero's roots of trust assumes, the state a halted rollout leaves, which marks and reports it needs, and where the margin goes.

CHAPTER 4.7

Capstone: Module Zero

TIME	6 weeks at least; 8 is realistic
YOU BUILD	Module Zero: the x86 server and Board B under Board A, with the Pi as a third voter, powering on through Redfish, booting measured, attesting to one another, forming a quorum, running a tenant's VM on your VMM with its volume erasure-coded across all three nodes, surviving a pulled node, and taking a rolling firmware update while the tenant works, all recorded in one take.
NEEDS	Everything: the lab server, Boards A and B, the Pi 5, the TL-SG108E, a laptop on the management VLAN, your VMM, stripe store, transport, verifier, node kernels and Board A firmware, a camera and a screen recorder.

WHAT THIS BUYS OMH

The earlier chapters produced the parts, and Module Zero assembles them. Two engineering-sample nodes of different architectures, the x86 server and the arm64 Board B, power on, attest their firmware, form a quorum and accept a tenant. They run its VM on your hypervisor, store its data erasure-coded across the rack, survive a node being pulled and take a rolling firmware update without the tenant noticing, and they show all of it through the interface a customer would use. The software must not depend on the architecture it runs on. When the demonstration runs from a cold rack with no keyboard attached, every claim on the OMH hardware page has a small working implementation behind it. The recording is the first thing you show an investor, a customer or a hire.

NO SKIP TEST

If you have done the Handbook and Units 1 to 4 you are ready. If you have not, the capstone will show you which chapter you skipped.

The demonstration

Write this down first and build toward it. Each step is a done-when criterion, and the acceptance checklist below turns each one into a check you tick from a cold start.

1. **Cold start.** Every node is off and Board A has standby power. From a laptop on the management VLAN, a Redfish `ComputerSystem.Reset` to Board A, served by its own firmware over lwIP, raises Board B's power-enable line and waits for power-good. A second call to the iDRAC powers the server, and the Pi starts with its supply. Board B boots through verified U-Boot and a signed FIT, the server through Secure Boot with your keys, both measured into their TPMs.
2. **Attest and join.** Each node boots the OMH node image, attests with a fresh nonce, and receives its credentials and disk key only on a passing quote. The nodes find one another on the management VLAN with no configuration and form a quorum with the Pi. The control plane shows every node joined within a time you record.
3. **A tenant arrives.** A tenant is created as a key pair and a policy. It calls the Compute API you designed in Chapter 4.4 to create an instance with a Lab 4.5.6 volume, whose key is unwrapped from your Modulus stand-in and never written to disk. The instance boots where the scheduler puts it, attaches its volume over NVMe-oF, and the tenant logs in over the tenant VLAN.

4. **A node is lost.** The tenant writes a known dataset and you pull the network cable, or the power, from a node that is not running the instance. The tenant's I/O continues, the control plane shows the degraded state and the repair plan, and after you reconnect, repair completes and every checksum verifies.
5. **A rolling update.** You release new firmware for Board A and a new host image. The Lab 4.6.3 rollout drains each node, or live-migrates its instances with the migration you built in Chapter 4.4, then updates, reboots, attests against the new reference values and readmits it before moving on. The tenant's instance stays reachable throughout. A deliberately bad image then halts the rollout and rolls back.
6. **Observe.** One dashboard fed by the management interface shows each node's temperatures and fan speeds, power from the INA228s and the server's power supplies, per-volume IOPS and latency, per-instance CPU and attestation state, and `curl` reaches the Redfish subset from your Lab 4.6.1 specification.
7. **Recording.** Record one take with a camera on the rack and a screen capture, narrated, under twenty minutes, with the cable pull on camera.

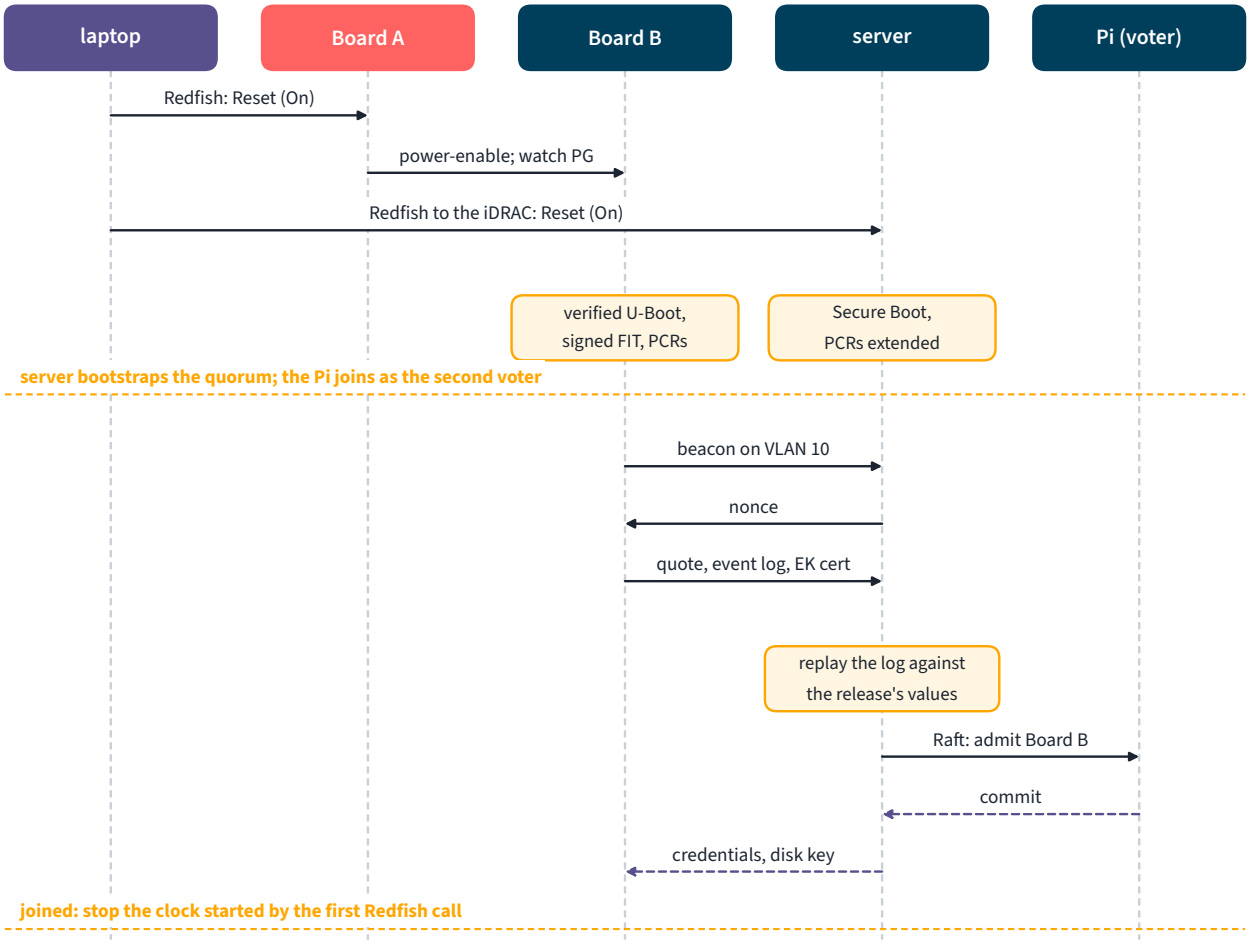


Figure 4.7.1. The cold start, from the first Redfish call to Board B's admission. The time between the first message and the last is the boot-to-joined figure your specification guessed.

Architecture

Decide these in the first week and write each as a short design note in the labs repository: the decision, the alternatives, and the lab that produced the evidence.

Node image. The node image is a Buildroot or Debian-based image holding the node kernels from Chapter 4.2, your VMM, the stripe store and the node agent, and nothing else, built for x86-64 and arm64 from one tree. The differences are the boot path and the kernel's platform code. Build it reproducibly. When two builds are identical, the reference values for the image's measurements can be computed from the build and published with the release, before any node boots it.

Identity and attestation. Each node's identity is its TPM endorsement-key certificate, enrolled by the Lab 4.6.4 procedure; Board A's is its ATECC608 key. The Lab 2.7.4 verifier becomes a quorum service: each node checks its peers' quotes against the signed reference values of the current release, and a node whose quote fails is refused. The Modulus stand-in is a key store sealed to each node's PCRs and replicated through the quorum, unwrapping volume keys for the VMM on request.

Membership and consensus. A Raft group holds the cluster state: nodes and their attestation status, tenants and policies, instances and placement, volumes and stripe maps, the firmware inventory and the active rollout. Implement Raft in C++ from the Chapter 4.5 reading, or take a library and write down why. Two voters cannot keep a quorum through a partition, so the Pi is the third voter and the third storage failure domain; record the trade-off.

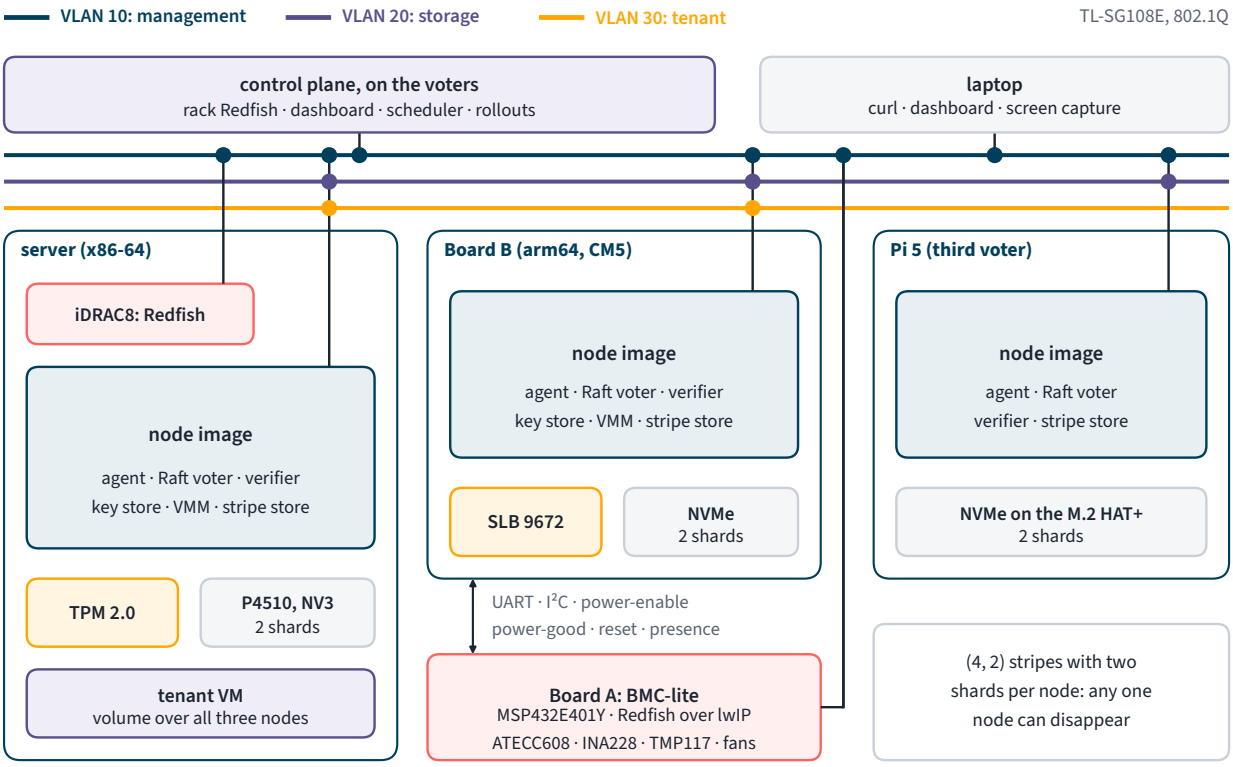


Figure 4.7.2. Module Zero on the bench: three nodes on three VLANs, with Board A on Board B's host connector and on the management VLAN through its own port. Every stripe keeps two shards on each node.

Discovery and join. Nodes announce themselves on the management VLAN with mDNS or a multicast beacon carrying their attestation evidence. The first node bootstraps the quorum alone; each later node is admitted when its quote verifies. There is no installation step, which is the site's zero-touch claim.

Compute. Compute runs your Chapter 4.4 VMM, or the Firecracker or Cloud Hypervisor decision you made there, behind the API you designed. It runs one process per instance under the Chapter 4.4 tenant hardening, with the arm64 port on Board B. A simple scheduler places instances. Each instance gets virtio-net on a per-tenant VLAN and a virtio-blk or vhost-user attachment to its volume.

Blocks. Blocks runs the Lab 4.5.6 stripe store as one process per node, with stripe maps in the Raft state, per-volume encryption keyed from the Modulus stand-in, NVMe reservations for fencing and snapshots as index copies. Place each (4, 2) stripe with two shards on each of the three nodes, so that any one node can disappear; a node holding more than m shards of a stripe turns its own loss into an outage. Repair starts when the quorum marks a node unhealthy, and scrub runs continuously at low priority.

Management. The server's iDRAC serves its vendor Redfish. Board A's firmware serves the Redfish subset directly over lwIP on its own Ethernet port: the service root, a `ComputerSystem` for Board B whose `Reset` action drives power-enable and reset, a `Chassis` with the TMP117, INA228 and fan sensors, a `Manager` for Board A itself and an `UpdateService` for its firmware. Serve it over TLS with the private key generated inside the ATECC608, and record the RAM high-water mark, because lwIP, TLS and JSON buffers share 256 KB. The

control plane aggregates both behind one Redfish endpoint for the rack, the site's "one endpoint" claim, and adds OMH's resources: tenants, instances, volumes, attestation and rollouts. If you respun Board A with the ECP5 and your Chapter 2.2 register-map peripheral (Chapter 3.4's stretch), that peripheral becomes the backplane controller, driving presence and identify LEDs over SPI as in Lab 2.6.3.

Networks. The TL-SG108E carries three VLANs: management (Board A, the iDRAC, node agents, the control plane), storage (NVMe-oF and replication) and tenant (instance traffic), configured on each node from its join response. The switch has no SFP+ ports, so every cross-node path runs at 1 Gbit/s, about 117 MB/s of payload; measure cores per GB/s on the server's DAC loop and treat 117 MB/s as the ceiling for every cross-node number.

Telemetry. Node agents publish sensor and performance data into a time series, and a small dashboard renders it. This is a small version of Signals.

Weekly plan

Six weeks is the minimum and eight is realistic. Each week is a lab that ends in a recorded partial demonstration.

LAB 4.7.1 Week 1: node image and join — 1 week

Goal. build one image for both architectures and attest both nodes.

1. Build the node image for x86-64 and arm64 from one tree with the Chapter 4.2 node kernels. Build it twice and compare the hashes.
2. Boot the server through Secure Boot with your keys (Lab 2.7.3) and Board B through verified U-Boot, a signed FIT and U-Boot's measured boot into the SLB 9672 (Labs 2.7.5 and 3.4.7).
3. Run the Lab 2.7.4 verifier on the server and attest both nodes to it.
4. Write the design notes.

DONE WHEN

- [ ] Both nodes attest and appear in a list, and two builds of the image are identical.

EVIDENCE FOR THE NOTEBOOK

- The design notes; the image hashes; the week's recording.

LAB 4.7.2 Week 2: quorum and identity — 1 week

Goal. make the quorum admit nodes by attestation.

1. Bring up Raft, yours or a library's, holding node state, with the Pi as third voter.
2. Implement discovery and admission: a beacon on the management VLAN, a nonce, a quote, and a Raft entry that admits the node.
3. Move the verifier into the quorum, with reference values signed per release.
4. Build the Modulus stand-in: a key store sealed to each node's PCRs and replicated through Raft.

DONE WHEN

- [ ] A node is refused after a change to its kernel command line and admitted once the reference values are updated.

EVIDENCE FOR THE NOTEBOOK

- The admission log for both cases; the Raft log; the week's recording.

LAB 4.7.3 Week 3: compute — 1 week

Goal. run a tenant's instance on either architecture.

1. Deploy your VMM and its API on both nodes, and write the scheduler.
2. Configure tenant VLANs from the join response.
3. Boot an instance on each node from a local image.

DONE WHEN

- [ ] A tenant creates an instance through the API and logs in over the tenant VLAN, on each node in turn.

EVIDENCE FOR THE NOTEBOOK

- The API transcript; the week's recording.

LAB 4.7.4 Week 4: Blocks — 1 week

Goal. give tenants volumes that survive a lost node.

1. Run the stripe store on all three nodes with stripe maps in Raft and keys from the Modulus stand-in.
2. Attach volumes over NVMe-oF and boot an instance from one.
3. Run `fio` in the guest and pull a node's cable; reconnect, repair and scrub.

DONE WHEN

- [ ] The guest sees no I/O errors through the cable pull, and repair completes with every checksum verified.

EVIDENCE FOR THE NOTEBOOK

- The guest `fio` log across the pull; the repair and scrub logs; the week's recording.

LAB 4.7.5 Week 5: updates and management — 1 week

Goal. run a rolling update under load and observe it through one endpoint.

1. Drive Board A's firmware update and both host image updates from the Lab 4.6.3 state machine.
2. Build the rack Redfish endpoint that aggregates Board A and the iDRAC and adds OMH's resources, and check it with DMTF's Redfish Service Validator.
3. Build the dashboard.

DONE WHEN

- [ ] A rolling update completes during tenant I/O, a bad image rolls back, and the rack endpoint passes the validator for every resource it implements.

EVIDENCE FOR THE NOTEBOOK

- The rollout log; the validator report; the week's recording.

LAB 4.7.6 Week 6: integration and recording — 1 week

Goal. run and record the whole demonstration from a cold start.

1. Run the full demonstration from cold five times, and fix what breaks each time.
2. Record it.
3. Write the retrospective.

DONE WHEN

- [ ] Five consecutive cold runs pass every step, and the recording and the retrospective exist.

EVIDENCE FOR THE NOTEBOOK

- The run log; the recording; the retrospective.

Acceptance criteria

DONE WHEN, FROM COLD

Module Zero is done when every line holds on a cold start with no keyboard attached to any node.

- [ ] Both nodes power on through Redfish, Board B through Board A and the server through its iDRAC, and boot with Secure Boot or verified boot and your keys enrolled.
- [ ] Every node attests with a fresh nonce, and a node with modified firmware or kernel is refused.
- [ ] The quorum forms without configuration, and the control plane lists every node with its attestation state and firmware versions.
- [ ] A tenant creates an instance with a volume through the API and reaches it over the tenant VLAN; the volume's key is unwrapped from the sealed store and appears in no file.
- [ ] Pulling any one node during tenant writes causes no I/O error in the guest, and after repair a full read verifies every checksum.
- [ ] A rolling update of Board A and both host images completes with the tenant's instance reachable throughout, and a bad image halts and rolls back.
- [ ] The dashboard shows live sensors, INA228 power and per-volume and per-instance metrics for every node; `curl` against the rack endpoint returns Systems, Chassis, Managers and the OMH resources, and Board A answers Redfish on its own Ethernet.
- [ ] **Labs 4.7.1 to 4.7.6:** six weekly demonstrations recorded.
- [ ] The recording exists, is under twenty minutes and shows the cable pull, and the retrospective is written.

Measurements to record

Measurement	Method
Cold power-on to every node joined, split into firmware, boot, attestation and admission	Timestamps from the first Redfish call, the consoles and the Raft log
Instance create to login, and boot time inside the VMM	API timestamps and the guest console
Volume throughput and latency at queue depths 1 and 32, healthy and degraded; repair throughput	`fio` in the guest; the stripe store's repair log
Cores per GB/s of erasure-coded writes, and the memory bandwidth used	`perf stat` and `pcm-memory`, as in Lab 4.5.6
Rolling update duration per node and in total; the tenant's latency during it	Rollout timestamps; a latency probe in the guest
Idle and loaded power per node; thermal steady state at the lab's inlet temperature	The INA228s on Boards A and B; the iDRAC's power and temperature readings; the TMP117s
Attestation cost: quote generation and verification time, and evidence size	Timers in the agent and the verifier
Board A's RAM high-water mark and Redfish response time	A stack and heap watermark; `curl -w` timings

These are the numbers your Lab 4.6.1 specification guessed. Record each with its method, and replace the guess in the specification and the cost model.

Retrospective

Two to four pages, written in the week after the recording.

1. What each claim on the OMH hardware page now rests on, and how far each is from something a customer could rely on. Be specific about the distance.
2. The three hardest problems, and what each taught you about the product.
3. What you would build differently, in hardware and in software, knowing what you know now.
4. The road from Module Zero to a saleable module: the threat model's gaps, the respin from Lab 3.4.8, the measurements still missing, the compliance plan's timeline and the cost model's remaining guesses.
5. Who you need to hire first, and what in this course they should already know.

Deliverables and stretch

Deliverables. The recording; the design notes; the six weekly demonstrations; the measurements with their methods; the retrospective; the labs repository tagged at the recorded release.

Stretch. Replace the server's vendor BMC firmware with OpenBMC if OpenBMC supports your server's BMC (check the openbmc repository's machine list before you flash anything), so every node runs management firmware you built. Run the demonstration with a real network partition and show the minority side fenced off the volumes by NVMe reservations. Add live migration to every step of the rolling update so no instance is drained, and measure the tenant's downtime. Put the S3 front end from Chapter 4.5's stretch on the stripe store and write and read an object through the rack. Present the recording and the retrospective to three people who build hardware for a living, and record their objections as the starting list for OMH's second year.

After the course

The labs repository is now a prototype of the company. Use the notebook as its engineering blog, the six documents from Chapter 4.6 as its first business plan and the retrospective as its roadmap. Keep working the loop Chapter 3.4 taught: specify, build, review, fabricate, bring up, find the mistake, respin.

UNIT 4

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and labs repository, and you can do each review task below without notes. Date each line as you finish it. If someone who builds hardware for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
4.1	Write a kernel	☐	☐	
4.2	Linux kernel internals	☐	☐	
4.3	Linux device drivers	☐	☐	
4.4	Virtualisation and hypervisors	☐	☐	
4.5	Storage and network data paths	☐	☐	
4.6	Product engineering	☐	☐	
4.7	Capstone: Module Zero	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook.

- ☐ Trace a 4 KiB read from a guest's `read(2)` through your VMM, the host kernel and the NVMe drive to the flash and back, naming every queue, copy, exit and interrupt.
- ☐ Walk a page fault through your kernel and then through Linux, and explain each difference.
- ☐ Write from memory the probe function of an I²C IIO driver like your OPT4048 driver, and explain each call and what device-tree binding it relies on.
- ☐ Explain how Module Zero admits a node from power-on to joining the quorum, and what happens when a node's firmware changes.
- ☐ Present the Module Zero recording and your retrospective to someone who builds servers, and defend the cost model against their questions.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____